TRABAJO DE GRADO

INGENIERIA DE SISTEMAS

Estrategia de automatización de pruebas funcionales y de regresión para la mejora de la calidad del software en entornos cloud

ENTREGABLE

TUTORA: Licenciada Natalia Mira

ALUMNOS:

- Luciano Castro
- Matías Primitz

Índice

1. Problemática 5
1.1. Contexto general de la problemática 5
1.2. Evidencia concreta de la problemática 6
1.2.1. Fallos Funcionales y de Regresión en Sistemas Transaccionales 6
1.2.2. Fallas por Sobrecarga de Demanda 7
1.2.3. Reflexión Integrada 7
1.2.4. Métricas de la Problemática en la Industria 8
1.3. Impacto de la Problemática 10
1.3.1. Impacto Técnico 10
1.3.2. Impacto Comercial 10
1.3.3. Impacto Reputacional 11
1.3.4. Impacto Operativo 11
1.4. Marco Teórico 12
1.4.1. Calidad de software: Concepto general 12
1.4.2. Testing como estrategia de aseguramiento de calidad 13
1.4.3. Automatización de pruebas funcionales y de regresión 13
1.4.4. Rol de cloud computing en el aseguramiento de calidad 14
1.4.5. Tipos de pruebas automatizadas 15
1.4.6. Tipos de regresión 16
1.4.7. Criterios de cobertura funcional 16
1.4.8. Testing basado en riesgos 17
1.4.9. Herramientas tecnológicas aplicadas 17
2. Tema de Investigación y Propuesta de Solución 21
2.1. Tema de investigación 21
2.2. Justificación 21
2.3. Objetivo general 22
2.4. Objetivos específicos 22
2.4.1. Objetivo Especifico 1 – Analizar el impacto de la ausencia de pruebas automatizadas en la calidad del software 23
2.4.2. Objetivo Especifico 2 – Determinar los atributos de calidad más afectados y vinculados al testing funcional 23
2.4.3. Objetivo Especifico 3 – Diseñar una estrategia de automatización de pruebas funcionales y de regresión 23
2.4.4. Objetivo Especifico 4 – Implementar un prototipo funcional de la estrategia de testing automatizado 23

2.4.5. Objetivo Especifico 5 – Validar la efectividad de la estrategia mediante análisis comparativo y evidencia técnica 23
2.4.6. Objetivo Especifico 6 – Documentar y fundamentar la aplicabilidad del modelo a otros sistemas equivalentes 24
2.5. Hipótesis o modelo de mejora 24
2.6. Propuesta de solución 24
2.6.1. Atributos de calidad priorizados según ISO/IEC 25010 25
2.6.2. Arquitectura de herramientas y tecnologías aplicadas 26
2.6.3. Posibilidad de integración con gestión de calidad 30
2.7. Alcances y limitaciones 30
2.7.1. Alcances 30
2.7.2. Limitaciones 30
3. Metodología 32
3.1. Enfoque metodológico general 32
3.2. Fases de desarrollo 32
3.3. Técnicas e instrumentos 34
3.3.1. Cypress 34
3.3.2. GitHub Actions 34
3.3.3. GitHub (repositorio) 35
3.3.4. JavaScript 35
3.3.5. Entorno simulado / controlado 35
3.3.6. Herramientas de generación de evidencia 36
3.3.7. Sistema de gestión de calidad (GitHub Projects) 36
3.3.8. Relación entre las herramientas 36
3.4. Validación de la solución 37
3.4.1. Objetivo 37
3.4.2. Escenario de prueba 37
3.4.3. Métodos e instrumentos 41
3.4.4. Métricas por relevar 41
3.4.5. Procedimiento 41
3.4.6. Resultados esperados 42
3.5. Procesos, roles y responsabilidades en el workflow de pruebas 42
3.5.1. Workflow tradicional (manual por roles) 43
3.5.2. Workflow automatizado (estrategia propuesta) 44
3.5.3. Colaboración de la estrategia en procesos y roles 45
3.5.4. Relación con ISO/IEC 25010 45

4. Implementación técnica 46
4.1. Introducción 46
4.2. Control de versiones y repositorio 46
4.3. Desarrollo del sistema 47
4.4. Identificación de casos de prueba críticos 48
4.5. Programación de casos de prueba automatizados 48
4.6. Pipeline CI/CD con GitHub Actions 49
4.6.1. Implementación del workflow en GitHub Actions 49
4.6.2. Beneficios del enfoque 52
4.7. Deploy en GitHub Pages 53
4.8. Diseño y ejecución de pruebas E2E con Cypress 53
4.8.1. Estrategia de pruebas 53
4.8.2. Arquitectura de pruebas 53
4.8.3. Diseño de casos representativos 54
4.8.4. Buenas prácticas aplicadas 54
4.8.5. Evidencias y trazabilidad 54
4.8.6. Métricas observadas 55
4.9. Cypress Cloud 55
4.10. Integración con Gestor de Proyecto 56
4.11. Resultado 57
5. Resultados y análisis 59
5.1. Objetivo 59
5.2. Descripción del prototipo ejecutado 59
5.3. Ejecución de pruebas manuales 59
5.4. Ejecución de pruebas automatizadas 59
5.5. Comparación de escenarios (Manual vs Automatizada)- Tabla 1 61
5.5.1. Registro de resultados y métricas (comparación Manual vs E2E) 62
5.5.2. Ejecuciones Manuales – Tabla 2 63
5.5.3. Ejecuciones Automatizadas – Tabla 3 65
5.5.4. Mochawesome - Reporte Ejecución de pruebas automatizadas – Figura 17 66
5.5.5. Cypress Cloud - Reporte Ejecución de pruebas automatizadas – Figura 18 67
5.6. Cierre del ciclo CI/CD 68
5.7. Análisis de resultados 70
5.7.1. Métricas observadas 70

5.7.2. Relación con atributos ISO/IEC 25010 70
5.8. Síntesis 71
6. Conclusiones y Recomendaciones 72
6.1. Conclusiones Generales 72
6.2. Contribuciones del Trabajo ..... **¡Error! Marcador no definido.**
6.2.1. A nivel académico ..... **¡Error! Marcador no definido.**
6.2.2. A nivel técnico ..... **¡Error! Marcador no definido.**
6.2.3. A nivel profesional ..... **¡Error! Marcador no definido.**
6.3. Recomendaciones 73
6.4. Líneas Futuras de Mejora 74
6.5. Cierre Final 74
7. Bibliografía 76
ANEXO I – ISO/IEC 25010:2011 77
ANEXO II - Plan y Guía de Pruebas – Plataforma de venta de entradas (Manual y Referencia a E2E) 82
1. TC-001 — Login correcto / inválido 83
2. TC-002 — Happy Path de compra 85
3. TC-003 — Pago rechazado - reintento aprobado 86
4. TC-004 — Carrito: agregar, ver totales y vaciar 88
5. TC-005 — Catálogo disponible luego del login 90
6. TC-006 — Sesión expirada y recuperación del flujo 91
ANEXO III - Workflow de Integración Continua en GitHub Actions 92
ANEXO IV – CASOS DE PRUEBA - Cypress 96
1. Login - auth cy.js 96
2. Checkout Happy - checkout.happy.cy.js 97
3. Carrito -cart.cy.js 99
4. Catalogo -cart.cy.js 100
5. Sesión expirada – sesión.expire.cy.js 101
6. Reintento de pago exitoso – sesión.expire.cy.js 103
ANEXO V – Paquete de replicación técnica 106
1. Introducción 106

1. Problemática

1.1. Contexto general de la problemática

La calidad del software se ha transformado en un factor estratégico para las plataformas de comercio electrónico, especialmente aquellas que operan en segmentos de alta transaccionalidad como la venta de entradas online (e-commerce). Estos sistemas no sólo deben garantizar la funcionalidad básica de compra, sino también soportar variaciones bruscas en el volumen de usuarios, brindar una experiencia de usuario segura, estable y fluida, y adaptarse rápidamente a cambios o actualizaciones.

En este tipo de plataformas, el fallo de cualquier componente crítico, como el proceso de login, selección de entradas o pasarela de pago, impacta de manera inmediata y directa tanto en el usuario final como en la imagen comercial de la empresa. Problemas funcionales, caídas de servicio o inconsistencias en la validación de datos generan pérdidas económicas cuantificables, daño reputacional y pérdida de confianza de los usuarios.

Tradicionalmente, el control de calidad en este tipo de sistemas se ha apoyado en pruebas manuales, ejecutadas antes de lanzamientos relevantes o durante mantenimientos programados. Sin embargo, este enfoque ha mostrado limitaciones significativas:

- Escasa cobertura de pruebas sobre escenarios posibles.
- Lentitud en la validación de cambios o actualizaciones frecuentes.
- Elevado riesgo de errores no detectados hasta producción.
- Dificultad para garantizar la estabilidad ante variaciones de carga.

Adicionalmente, la falta de integración con entornos de ejecución en la nube limita la posibilidad de obtener trazabilidad, repetibilidad y escalabilidad en los procesos de prueba. Esto no solo retrasa la validación de nuevas versiones, sino que también impide generar evidencias objetivas de cumplimiento de calidad ante auditorías internas o externas.

Frente a esta realidad, resulta indispensable avanzar hacia esquemas de validación automatizada, apoyados en tecnologías de integración continua (CI) y ejecución en entornos cloud. No hacerlo representa no solo una ineficiencia operativa, sino también una desventaja competitiva en un mercado donde la agilidad y la confiabilidad son claves para la permanencia.

1.2. Evidencia concreta de la problemática

La falta de testing funcional automatizado en las fases de desarrollo de sistemas críticos de transacciones online genera errores funcionales y regresiones que impactan negativamente en la calidad del servicio, la experiencia del usuario y los resultados comerciales.

La identificación pública de este tipo de fallos, sin embargo, no siempre resulta sencilla: Debido al impacto que los errores tienen en la imagen corporativa y en la confianza de los usuarios, muchas empresas optan por no divulgar formalmente las incidencias o bien minimizar su exposición a través de canales de atención interna.

Así, la evidencia concreta de fallos funcionales suele surgir más comúnmente de reportes en comunidades técnicas, foros de usuarios y medios especializados, donde los propios afectados comparten experiencias reales de disfuncionalidad.

A continuación, se presentan casos recientes y verificables que ilustran claramente la problemática.

1.2.1. Fallos Funcionales y de Regresión en Sistemas Transaccionales

Caso 1: Problemas de carrito de compras en Shopify (foro oficial, 2023)

En el foro oficial de Shopify Community, usuarios reportaron incidentes donde, tras actualizaciones en la plataforma, los carritos de compra no retenían correctamente los productos seleccionados, generando pérdidas de ventas y frustración de clientes.

Tipo de falla: Error funcional en flujo crítico de compra.

Fuente: Shopify Community - https://community.shopify.com/c/preguntas-y-respuestas-t%C3%A9cnicas/problemas-con-el-carrito-de-compras/m-p/1928450

Impacto: Abandono de compras, deterioro de experiencia del usuario.

Caso 2: Fallos de acceso en Openbank (ADSLZone, 2023)

Según reportes de ADSLZone, Openbank enfrentó problemas de acceso a cuentas de usuarios, impidiendo el uso normal de sus plataformas digitales durante horas.

Tipo de falla: Falla funcional crítica en login de usuarios.

Fuente: ADSLZone - https://www.adslzone.net/noticias/fintech/problemas-acceso-cuenta-openbank-solucion/

Impacto: Interrupción de servicios bancarios esenciales, pérdida de confianza.

Caso 3: Fallas funcionales en banco digital (Brubank, 2023)

En informes de Ámbito Financiero y redes sociales, usuarios de Brubank reportaron errores en transferencias bancarias, como saldos incorrectos o movimientos no reflejados, tras actualizaciones recientes.

Tipo de falla: Falla de actualización de datos críticos.

Impacto: Riesgo financiero y pérdida de credibilidad.

Caso 4: Errores de checkout en CyberMonday (CACE/Infobae, 2022)

Durante el CyberMonday 2022, múltiples usuarios reportaron fallos en carritos de compra y procesos de checkout, imposibilitando completar transacciones en sitios de e-commerce relevantes.

Tipo de falla: Regresión funcional en procesos de pago.

Fuente: Infobae

Impacto: Pérdida de ventas, reputación afectada.

1.2.2. Fallas por Sobrecarga de Demanda

Caso 5: Mercado Libre – Hot Sale Argentina (Infobae, 2023)

En el marco del Hot Sale 2023, Mercado Libre experimentó demoras y caídas parciales de servicio, afectando operaciones de compra y pago de usuarios.

Tipo de falla: Problemas de performance ante alta demanda.

Fuente: Infobae

Impacto: Pérdida de operaciones en momentos críticos.

1.2.3. Reflexión Integrada

Los casos expuestos muestran que las plataformas de venta online, fintech y sistemas transaccionales enfrentan riesgos de diversa naturaleza:

- Errores funcionales no detectados en la etapa de desarrollo.
- Fallos de regresión introducidos por actualizaciones insuficientemente validadas.
- Caídas por sobrecarga, aunque menos frecuentes, también afectan gravemente la continuidad de los negocios.

La falta de implementación sistemática de pruebas funcionales automatizadas y de regresión, integradas en entornos de desarrollo y controlados mediante herramientas de integración continua, es un factor clave que explica la persistencia de estos problemas.

La problemática no solo persiste, sino que se agrava en un mercado donde la agilidad, la confiabilidad y la experiencia de usuario son cada vez más exigidas.

1.2.4. Métricas de la Problemática en la Industria

La problemática derivada de la falta de automatización de pruebas funcionales y de regresión no sólo se evidencia a través de casos aislados, sino que también ha sido sistemáticamente documentada mediante estudios de alcance global.

Diversos informes recientes destacan el impacto medible de no incorporar estrategias adecuadas de testing automatizado en los procesos de desarrollo de software.

A continuación, se presentan métricas relevantes:

Demoras en entregas y liberaciones

Según el **World Quality Report 2023-2024** (Capgemini, Sogeti, Micro Focus):

- **61%** de las organizaciones reconocen que su capacidad de liberar versiones de software se ve limitada por la falta de automatización de pruebas en su ciclo de desarrollo.
- El **Lead Time for Changes** (tiempo desde un cambio en el código hasta su puesta en producción) es en promedio **2,5 veces mayor** en proyectos que dependen de testing manual.

Incremento de defectos en producción

El **State of Testing Report 2023** (PractiTest) revela que:

- **49%** de las empresas admiten haber incrementado la cantidad de defectos en ambientes productivos como consecuencia directa de no contar con una cobertura adecuada de testing automatizado.
- El **defect density** (número de defectos por unidad funcional) es aproximadamente **35% mayor** en desarrollos que carecen de suites automatizadas de regresión.

Incremento de costos de corrección de errores

Según la **ISTQB Software Testing Survey 2023**:

- El costo promedio de corregir un defecto detectado en producción es entre **4 y 6 veces mayor** que el costo de corregirlo en etapas tempranas de desarrollo.
- **70%** de las organizaciones considera que el testing automatizado contribuye directamente a la reducción del costo total de calidad.

Impacto en agilidad y DevOps

El **World Quality Report** también destaca que **58%** de las empresas manifiestan que su estrategia actual de testing no soporta adecuadamente las necesidades de agilidad e integración continua exigidas por entornos DevOps y despliegue continuo.

Las métricas analizadas confirman de manera objetiva que la falta de automatización de pruebas no solo genera impactos técnicos visibles (errores funcionales, regresiones), sino que también:

- Retrasa entregas, afectando la competitividad.
- Eleva costos operativos, incrementando el costo total de calidad.
- Reduce la fiabilidad del producto final, impactando la experiencia del usuario.

Por tanto, la necesidad de adoptar estrategias de testing automatizado no responde simplemente a una tendencia tecnológica, sino a una exigencia concreta de la industria para sostener calidad, eficiencia y competitividad en proyectos de software modernos.

1.3. Impacto de la Problemática

La ausencia de estrategias sistemáticas de automatización de pruebas funcionales y de regresión durante las fases de desarrollo genera consecuencias severas tanto en el plano técnico como en los ámbitos comercial, reputacional y operativo de las organizaciones.

1.3.1. Impacto Técnico

La falta de testing automatizado conlleva un incremento significativo de errores funcionales no detectados en etapas tempranas, que luego se trasladan a los ambientes de producción. Esto da lugar a:

- Aumento de bugs en producción, afectando funciones críticas como login, procesos de compra y validaciones de pago.
- Errores de regresión, donde funcionalidades previamente estables fallan tras cambios o actualizaciones no suficientemente validadas.
- Inestabilidad del sistema, con comportamiento errático que degrada la experiencia del usuario.
- Dificultad para implementar cambios rápidos y seguros, ya que la validación manual resulta lenta e incompleta.

Desde el punto de vista de la ingeniería del software, esta situación rompe con los principios de calidad, mantenibilidad y confiabilidad exigidos en sistemas de alta disponibilidad.

1.3.2. Impacto Comercial

Las deficiencias técnicas se traducen en impactos económicos concretos, tales como:

- Pérdida directa de ventas, debido a procesos de compra fallidos, abandonos de carritos o interrupciones de servicio.
- Incremento de costos asociados a soporte técnico, atención al cliente y re-procesamiento de operaciones fallidas.
- Reducción de tasas de conversión, especialmente crítica en eventos de alta demanda (Hot Sale, preventas, entre otros).
- Desventaja competitiva frente a empresas que aseguran flujos de compra ágiles, confiables y libres de errores.

En mercados donde la agilidad y la satisfacción del usuario son diferenciales estratégicos, estos problemas pueden implicar pérdidas de mercado significativas.

1.3.3. Impacto Reputacional

La confianza del usuario es un activo intangible fundamental en el comercio electrónico. Las fallas funcionales recurrentes o visibles provocan:

- Pérdida de confianza del consumidor en la plataforma.
- Daño a la imagen de marca, amplificado por el efecto de las redes sociales y los sitios de opinión.
- Mayor sensibilidad a la competencia, dado que los usuarios perciben alternativas más confiables como preferibles.
- Riesgo de crisis públicas en casos de fallas masivas o reiteradas, como se ha observado en incidentes recientes en plataformas de venta online y servicios financieros.

La recuperación de la reputación dañada requiere inversiones de marketing, relaciones públicas y programas de fidelización, cuyos costos son muchas veces superiores a los que implicaría una estrategia adecuada de prevención.

1.3.4. Impacto Operativo

En términos operativos, la falta de automatización eficiente provoca:

- **Aumento de los tiempos de entrega de nuevas versiones**, debido a la necesidad de pruebas manuales extensivas.
- **Sobrecarga de los equipos de QA**, que deben validar repetidamente funcionalidades estables ante cada cambio, generando desgaste y errores humanos.
- **Baja eficiencia en la gestión de releases**, afectando la capacidad de responder a cambios del mercado o nuevas necesidades del negocio.
- **Dificultades para escalar el sistema**, ya que cada modificación representa un riesgo elevado de fallas.

Desde una perspectiva de desarrollo ágil y DevOps, estas ineficiencias contradicen los principios de integración continua, entrega y mejora continua.

1.4. Marco Teórico

1.4.1. Calidad de software: Concepto general

La calidad de un producto de software no puede definirse únicamente en términos de ausencia de errores, sino que se evalúa a partir de un conjunto de características normalizadas internacionalmente. En este sentido, el modelo de calidad definido por la norma ISO/IEC 25010:2011, perteneciente a la familia SQuaRE (Software product Quality Requirements and Evaluation), establece un marco conceptual que descompone la calidad en ocho características principales: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.

Cabe señalar que en 2023 se publicó una actualización (ISO/IEC 25010:2023) que introduce ajustes terminológicos y conceptuales para alinearse con entornos de desarrollo modernos (DevOps, cloud, IA). Sin embargo, la estructura conceptual se mantiene equivalente, por lo que la versión 2011 continúa siendo plenamente aplicable y es la adoptada en este trabajo como referencia principal.

Cada característica se subdivide en subccaracterísticas que permiten especificar, medir y evaluar la calidad del software (ver ANEXO I). Así, por ejemplo, la Adecuación funcional contempla la corrección y completitud de las funciones implementadas, mientras que la Fiabilidad incluye aspectos como tolerancia a fallos, recuperabilidad y resistencia.

En el contexto de este proyecto de grado, el modelo ISO/IEC 25010:2023 ofrece la justificación teórica para los objetivos de automatización de pruebas, ya que los ensayos funcionales con Cypress permiten verificar la Adecuación funcional; la integración en pipelines de CI/CD contribuye a la Fiabilidad y la Mantenibilidad; los reportes automáticos aportan a la Analizabilidad; y las validaciones de la interfaz de usuario impactan directamente en la Capacidad de interacción.

De esta manera, se demuestra que las acciones de calidad implementadas en este trabajo no responden únicamente a criterios prácticos, sino que se encuentran alineadas con un estándar internacionalmente reconocido que define y estructura la calidad de software.

La calidad de software es definida como el conjunto de características de un producto de software que determina su capacidad para satisfacer necesidades explícitas o implícitas bajo condiciones específicas de uso.

Autores como Pressman (*Ingeniería de Software: Un enfoque práctico, 2015*) y Sommerville (*Ingeniería del Software, 2017*) enfatizan que la calidad no debe ser un proceso posterior al desarrollo, sino un objetivo transversal a todas las etapas del ciclo de vida del software.

Además, el World Quality Report 2023-2024 (Capgemini, Sogeti, Micro Focus) señala que en el entorno actual de transformación digital, la calidad de software es un factor estratégico que impacta directamente en la fidelización del cliente y la competitividad de las organizaciones.

1.4.2. Testing como estrategia de aseguramiento de calidad

Dentro de las prácticas que permiten asegurar atributos de calidad, el testing de software constituye un pilar esencial.

Según la definición de ISTQB (International Software Testing Qualifications Board, 2023), testing es:

"El proceso de evaluación de un producto de software para detectar diferencias entre las condiciones existentes y las requeridas (defectos) y para evaluar las características del software."

Se distinguen dos tipos principales relevantes para este proyecto:

- Testing funcional: Verifica que cada funcionalidad cumpla con los requisitos especificados.
- Testing de regresión: Valida que los cambios introducidos no afecten negativamente funcionalidades ya existentes.

El testing permite garantizar atributos como funcionalidad correcta, fiabilidad bajo cambios, y eficiencia operativa.

Sin embargo, es importante destacar sus limitaciones:

- El testing funcional automatizado no cubre atributos como usabilidad o experiencia del usuario (por ejemplo, estética visual, facilidad intuitiva).
- Tampoco detecta todos los problemas de seguridad a nivel de arquitectura o configuración de red.
- Por ello, debe ser complementado por otras estrategias de calidad como revisiones de código, auditorías de seguridad, y pruebas de experiencia de usuario.

1.4.3. Automatización de pruebas funcionales y de regresión

La automatización del testing permite superar las restricciones del testing manual, especialmente en proyectos que requieren entregas frecuentes, cambios dinámicos y despliegues continuos.

El World Quality Report 2023 identifica la automatización como uno de los factores clave que permiten sostener la calidad en entornos DevOps, donde la velocidad de cambios es alta.

Entre las principales ventajas de la automatización se destacan:

- Repetibilidad: posibilidad de ejecutar las mismas pruebas cada vez que se realiza un cambio.

- Cobertura ampliada: se puede testear más funcionalidad en menos tiempo.
- Agilidad: disminuye los tiempos del ciclo de prueba en comparación con testing manual.
- Trazabilidad y documentación automática: generación de reportes, capturas de errores, y métricas.
- Detección temprana de errores: Prevención de defectos en etapas iniciales del ciclo de vida.
- Reducción de costos a largo plazo: especialmente en proyectos con alta tasa de cambio.

En particular, la automatización de pruebas de regresión resulta crítica: toda modificación en el código fuente puede introducir defectos imprevistos, y la única forma de detectar estos problemas de manera efectiva y sostenible es a través de suites de prueba ejecutadas automáticamente tras cada modificación relevante.

1.4.4. Rol de cloud computing en el aseguramiento de calidad

La adopción de plataformas de ejecución en la nube aporta beneficios sustanciales al proceso de testing automatizado:

- Ejecución escalable: Capacidad de lanzar pruebas en paralelo en múltiples entornos simultáneamente.
- Reproducibilidad: Ambientes homogéneos que garantizan consistencia de resultados.
- Trazabilidad: Registro automático de logs, resultados, y métricas de pruebas.
- Disponibilidad continua: Posibilidad de ejecutar pruebas en cualquier momento sin necesidad de infraestructura propia.

Herramientas como Cypress y plataformas de CI/CD como GitHub Actions permiten coordinar todo el ciclo de testing en la nube, integrándolo directamente con los repositorios de código y los flujos de despliegue continuo.

En este contexto, la nube no sólo representa una ventaja operativa, sino también un pilar para sostener la calidad en esquemas de integración y entrega continua.

1.4.4.1. Relación con el proyecto

El presente proyecto de grado se centra en el diseño y aplicación de una estrategia de testing funcional automatizado y de regresión, ejecutada en entornos cloud, como mecanismo para mejorar atributos esenciales de calidad de software: funcionalidad, fiabilidad y eficiencia.

Se buscará:

- Validar flujos críticos de operación de manera automatizada.
- Prevenir errores de regresión ante cambios.
- Aumentar la cobertura de validación en entornos controlados y reproducibles.

De este modo, el trabajo no se limita a una demostración técnica, sino que se inserta en una perspectiva integral de mejoramiento de la calidad de software, alineada con las mejores prácticas y tendencias actuales de la industria.

1.4.5. Tipos de pruebas automatizadas

Las pruebas automatizadas pueden clasificarse de acuerdo al nivel de abstracción y la capa del sistema que validan:

- Pruebas unitarias: se enfocan en componentes individuales del código fuente (funciones, métodos, clases). Generalmente son escritas y mantenidas por desarrolladores. Herramientas comunes: JUnit, NUnit, pytest.
- Pruebas de integración: verifican la correcta interacción entre diferentes módulos o componentes del sistema. Son esenciales para detectar fallos en flujos de datos y dependencias entre servicios o API.
- Pruebas end-to-end (E2E): simulan el comportamiento del usuario final. Validan flujos funcionales completos desde la entrada hasta la salida del sistema. Son especialmente útiles en aplicaciones web y móviles. Cypress, Selenium y Playwright son ejemplos destacados de herramientas E2E.

La pirámide de testing (Cohn, 2009) sugiere que la mayoría de pruebas deberían ser unitarias, seguidas por pruebas de integración y, en menor medida, E2E. Sin embargo, en sistemas donde lo crítico está en la interacción completa (como una plataforma de venta online), el énfasis en pruebas E2E es totalmente justificable.

En este proyecto se adopta el enfoque E2E con Cypress, priorizando flujos como login, selección de entradas, proceso de compra y confirmación, ya que representan el núcleo funcional del sistema.

Si bien en el presente proyecto se han limitado las pruebas automatizadas a la capa de aplicación del sistema controlado, resulta pertinente señalar que la validación puede extenderse también al nivel de base de datos. La verificación de registros mediante consultas SQL permite corroborar que las operaciones realizadas en la interfaz de usuario se reflejen de manera consistente en la capa de persistencia, garantizando la integridad de la información. Este enfoque complementario, común en entornos de aseguramiento de la calidad, asegura que los datos almacenados mantengan coherencia con la lógica de negocio aplicada. En el marco de esta tesis, se ha optado por restringir el alcance a la interacción controlada con la aplicación; sin embargo, se reconoce que la incorporación

de pruebas automatizadas a nivel de base de datos constituye una línea de mejora y extensión futura de la presente investigación.

1.4.6. Tipos de regresión

Las pruebas de regresión se orientan a validar que las funcionalidades previamente estables no se vean afectadas por nuevos desarrollos, correcciones o mejoras.

Los tipos más comunes de regresión incluyen:

- Regresión funcional: ocurre cuando una funcionalidad que antes funcionaba correctamente comienza a fallar tras un cambio. Es la forma más común y crítica.
- Regresión visual o de UI: errores de diseño, desplazamientos en la interfaz, estilos alterados o pérdida de accesibilidad. Herramientas como Percy, Applitools o Cypress Visual Testing permiten detectarlas.
- Regresión de datos: implica cambios inesperados en la persistencia o presentación de datos, como cálculos erróneos, datos que no se actualizan o se eliminan incorrectamente. Este tipo de regresión es más difícil de detectar sin validaciones automáticas post-ejecución.
- Regresión de rendimiento (no cubierta en este proyecto): implica una degradación en los tiempos de respuesta, consumo de recursos o escalabilidad.

Detectar regresiones requiere mantener una suite de pruebas automatizadas actualizada, que actúe como una red de seguridad ante cada modificación del sistema.

1.4.7. Criterios de cobertura funcional

La cobertura funcional se refiere al grado en que las funcionalidades del sistema han sido verificadas mediante pruebas. A diferencia de la cobertura de código (que mide líneas o ramas ejecutadas), la cobertura funcional se orienta a los requerimientos y a los casos de uso esperados.

Los criterios más utilizados para establecer la cobertura funcional son:

- Cobertura por requerimientos: mide qué porcentaje de los requerimientos funcionales están cubiertos por pruebas automatizadas. Herramientas como Xray o TestRail permiten mapear requerimientos con casos de prueba.
- Cobertura por criticidad o riesgo: prioriza funcionalidades cuyo fallo tendría mayor impacto, como login, pagos, seguridad de datos o flujos de compra. Este criterio es el adoptado en este proyecto.

- Cobertura por frecuencia de uso: funcionalidades más utilizadas tienen mayor prioridad. Por ejemplo, en un sistema de venta de entradas, el flujo de compra es más crítico que la edición del perfil del usuario.
- Cobertura por tipo de usuario o rol: se asegura que cada perfil (cliente, administrador, operador) pueda operar sin errores las funcionalidades asignadas.

Establecer estos criterios permite optimizar los recursos y centrar la automatización en las partes más sensibles del sistema.

1.4.8. Testing basado en riesgos

El testing basado en riesgos (Risk-Based Testing, RBT) es una estrategia en la cual la planificación, priorización y diseño de pruebas se fundamenta en los riesgos identificados en el sistema. Se considera tanto la probabilidad de que un defecto ocurra como el impacto que tendría si ese defecto se manifestara.

Este enfoque, formalizado por el ISTQB y aplicado en entornos industriales, se adapta perfectamente a proyectos donde no se puede testear todo, y por lo tanto se requiere tomar decisiones informadas sobre qué automatizar.

En el caso del presente proyecto:

- Las funcionalidades como login, selección de entradas y pasarela de pago presentan alto riesgo por su centralidad operativa.
- Otras funcionalidades como edición de perfil, visualización de historial o detalles de usuario tienen menor impacto ante errores y, por tanto, se testean con menor frecuencia o en forma manual.

Este enfoque permite diseñar una estrategia de testing eficiente y justificable, alineada con los objetivos de calidad, el tiempo disponible y los recursos técnicos del equipo.

1.4.9. Herramientas tecnológicas aplicadas

La elección de herramientas adecuadas para la ejecución de pruebas funcionales automatizadas y de regresión resulta crucial para garantizar la eficiencia, trazabilidad y escalabilidad del proceso de aseguramiento de calidad.

El presente proyecto adopta dos tecnologías principales, ampliamente reconocidas en la industria: Cypress y GitHub Actions.

Cypress: Automatización de pruebas funcionales

Cypress es una plataforma de testing de código abierto diseñada para validar aplicaciones web modernas. Se ejecuta directamente en el navegador, lo que le permite interactuar en tiempo real con la aplicación bajo prueba.

Según su documentación oficial (*Cypress Documentation, 2024*), se destacan las siguientes capacidades:

- **Testing End-to-End de interfaces web:** Permite validar flujos completos de usuario (login, compra, checkout, entre otras funcionalidades.).
- Testing de APIs: Cypress soporta la validación directa de APIs RESTful o GraphQL mediante comandos como cy.request() y cy.intercept(), posibilitando el testeo de servicios de back-end de manera aislada o integrada.
- Testing de bases de datos: Si bien Cypress no se conecta directamente a bases de datos, permite mediante plugins y comandos personalizados validar los efectos de acciones en la base (por ejemplo, después de un registro o compra).
- Mocking y Stub de servicios externos: Puede interceptar solicitudes HTTP y simular respuestas controladas, ideal para pruebas en entornos donde servicios dependientes aún no están disponibles o requieren aislamiento.
- Captura automática de evidencias: Genera screenshots y grabaciones de video automáticas ante fallos, facilitando la trazabilidad de errores.
- Integración con herramientas de CI/CD: Facilita la orquestación de pruebas en plataformas como GitHub Actions, GitLab CI, Jenkins, CircleCI, entre otras.
- Plugins y extensibilidad: Cypress cuenta con un ecosistema de plugins que permiten extender sus capacidades (reportes mochawesome, integración con Allure, testing visual, testing de performance, entre otros).

Estas características convierten a Cypress en una herramienta adecuada no solo para la validación de front-end, sino también para pruebas de servicios intermedios, procesos de integración y regresión automatizada.

Referencia: Cypress Documentation, 2024. https://docs.cypress.io/

GitHub Actions: Integración y automatización de testing en la nube

GitHub Actions es la plataforma de integración continua (CI) y entrega continua (CD) incorporada en GitHub, diseñada para automatizar flujos de trabajo relacionados con la construcción, prueba y despliegue de software.

De acuerdo a su documentación oficial (*GitHub Docs, 2024*), sus capacidades principales incluyen:

- **Orquestación basada en eventos:** Permite ejecutar flujos de trabajo (workflows) automáticamente en respuesta a eventos como push, pull request, creación de ramas, tags o releases.

- **Ejecución en matrices y paralelización:** Posibilita correr múltiples pruebas simultáneamente en distintos sistemas operativos, versiones de Node.js, navegadores, entre otros, reduciendo los tiempos de validación y aumentando la cobertura.
- **Escalabilidad cloud-native:** Los workflows se ejecutan en runners hospedados en la infraestructura de GitHub, garantizando disponibilidad, reproducibilidad y elasticidad en el uso de recursos.
- **Seguridad y gestión de secretos:** Integra mecanismos seguros para el manejo de variables sensibles (tokens de autenticación, claves de API, contraseñas) mediante GitHub Secrets.
- **Artefactos y reportes:** Permite almacenar y publicar artefactos generados por los jobs, como reportes de pruebas, resultados de cobertura, capturas de error, entre otros.
- **Extensibilidad mediante Marketplace:** Cuenta con un marketplace de más de 10.000 acciones preconstruidas, facilitando la integración de servicios externos (Slack, AWS, Firebase, Cypress Dashboard, entre otros).
- **Facilidad de configuración:** Los workflows se describen en archivos YAML sencillos y declarativos, lo que permite personalizar y versionar los procesos de CI/CD.

El uso de GitHub Actions, en combinación con Cypress, permite garantizar que las pruebas se ejecuten automáticamente en cada evento crítico del desarrollo, aumentando la eficiencia, la trazabilidad y la calidad del producto final.

Referencia: GitHub Actions Documentation, 2024. https://docs.github.com/es/actions

1.4.9.1. Relación con el proyecto

La adopción de Cypress y GitHub Actions en el presente proyecto responde a criterios de:

- Alineación con tendencias de la industria, según World Quality Report 2023-2024 y State of Testing Report 2023.
- Capacidad de escalar el proceso de testing, asegurando cobertura funcional robusta en entornos dinámicos y de alta demanda.
- Facilidad de integración y automatización, permitiendo incorporar testing como un componente esencial del flujo de desarrollo y entrega continua.

Estas herramientas no solo facilitan el aseguramiento de la calidad funcional, sino que también contribuyen a sostener prácticas de DevOps, mejorando la estabilidad, trazabilidad y eficiencia operativa del ciclo de vida del software.

En síntesis, los casos analizados, las métricas de la industria y el marco teórico revisado permiten concluir que la falta de pruebas automatizadas en sistemas transaccionales constituye un obstáculo crítico para la calidad del software. Este déficit se traduce en errores funcionales recurrentes, regresiones no detectadas, sobrecostos operativos y pérdida de confianza del usuario, con un impacto directo en la competitividad de las organizaciones. En consecuencia, resulta indispensable plantear una solución que atienda de forma sistemática estas limitaciones, adoptando herramientas y enfoques que garanticen repetibilidad, trazabilidad y escalabilidad en la validación de flujos críticos. Esta necesidad justifica la formulación de una propuesta de solución que responda a las debilidades identificadas, la cual se desarrolla en el siguiente capítulo.

2. Tema de Investigación y Propuesta de Solución

A partir de la problemática expuesta, este capítulo presenta la propuesta de solución: una estrategia de automatización de pruebas funcionales y de regresión en entornos cloud. La elección de este enfoque no es arbitraria, sino que responde directamente a los desafíos identificados en el capítulo anterior: la lentitud de la validación manual, la escasa cobertura de escenarios, la falta de trazabilidad de resultados y la dificultad de escalar pruebas en contextos de alta demanda. La solución se fundamenta en el uso combinado de Cypress y GitHub Actions, tecnologías que permiten validar de forma ágil y reproducible los flujos críticos de una plataforma de venta online, fortaleciendo atributos de calidad definidos por la norma ISO/IEC 25010 como adecuación funcional, fiabilidad y eficiencia de desempeño.

2.1. Tema de investigación

¿Cómo puede contribuir la automatización de pruebas funcionales y de regresión, implementada en entornos cloud mediante herramientas modernas como Cypress y GitHub Actions, a mejorar la calidad del software en sistemas de venta online con flujos críticos de operación?

Este proyecto propone investigar, diseñar y aplicar una estrategia de testing automatizado que permita validar de forma sistemática los principales flujos funcionales de una plataforma web transaccional. El foco estará puesto en el impacto que dicha automatización tiene sobre atributos como funcionalidad, fiabilidad y eficiencia, de acuerdo al modelo ISO/IEC 25010 (ANEXO I).

A través de la aplicación práctica de herramientas actuales, el proyecto busca demostrar que la incorporación estructurada del testing automatizado en el ciclo de desarrollo puede contribuir de manera significativa a la mejora de la calidad del software, a la prevención de regresiones y al fortalecimiento del proceso de entrega continua.

Asimismo, la propuesta no se limita a la automatización de pruebas funcionales, sino que integra de forma completa el proceso de aseguramiento de calidad, abarcando la construcción (*build*), despliegue (*deploy*), ejecución de pruebas, generación de reportes y gestión automatizada de incidencias dentro de un flujo continuo en la nube. Este enfoque integral permite garantizar reproducibilidad, trazabilidad y consistencia en todas las etapas del ciclo de vida del software, alineándose con las mejores prácticas de ingeniería de calidad.

2.2. Justificación

En el desarrollo de sistemas de software orientados a servicios transaccionales, como las plataformas de venta online, la calidad del producto final no puede depender

exclusivamente de pruebas manuales o validaciones ad hoc realizadas al final del ciclo de desarrollo. Este tipo de sistemas opera bajo exigencias crecientes de confiabilidad, escalabilidad, experiencia de usuario y disponibilidad continua.

La evidencia expuesta en el capítulo anterior, tanto en forma de casos reales como de métricas de industria (World Quality Report, PractiTest, ISTQB), demuestra que la ausencia de una estrategia sistemática de pruebas automatizadas conlleva errores funcionales, regresiones frecuentes y una pérdida significativa de eficiencia operativa. A su vez, la detección tardía de defectos implica un aumento exponencial del costo de corrección, comprometiendo plazos de entrega y satisfacción del usuario final.

En este contexto, resulta justificado investigar e implementar una solución que integre pruebas automatizadas desde etapas tempranas del ciclo de vida del software. La automatización de pruebas funcionales y de regresión no solo permite detectar errores de forma más ágil y repetible, sino que habilita prácticas modernas como la integración continua (CI), la entrega continua (CD) y el despliegue confiable en entornos cloud, aspectos cada vez más relevantes en organizaciones que adoptan enfoques DevOps.

Asimismo, utilizar herramientas actuales como Cypress y GitHub Actions permite garantizar trazabilidad, cobertura, escalabilidad y facilidad de mantenimiento de las pruebas, en línea con los principios de calidad definidos por la norma ISO/IEC 25010.

Este proyecto resulta pertinente y necesario, ya que contribuye a resolver un problema técnico real, con impacto directo en la calidad de sistemas de alto uso. Además, proporciona una solución aplicable a múltiples escenarios más allá del caso analizado, y refuerza una práctica profesional ampliamente demandada en la industria actual: el testing automatizado como garantía de calidad continua.

2.3. Objetivo general

Diseñar, aplicar y validar una estrategia integral de testing automatizado funcional y de regresión, ejecutada en entornos cloud, que no solo optimice la validación de flujos críticos, sino que integre de manera sistemática las fases de construcción, despliegue, ejecución, reporte y gestión de calidad, conforme a los lineamientos de la norma ISO/IEC 25010.

2.4. Objetivos específicos

Desglosamos como acciones puntuales que, al cumplirse, conducen al logro del objetivo general.

2.4.1. Objetivo Especifico 1 – Analizar el impacto de la ausencia de pruebas automatizadas en la calidad del software

Investigar, documentar y fundamentar los efectos técnicos, comerciales y operativos que genera la falta de testing automatizado en sistemas web transaccionales, con especial foco en regresiones funcionales y errores en producción.

2.4.2. Objetivo Especifico 2 – Determinar los atributos de calidad más afectados y vinculados al testing funcional

Identificar, a partir de ISO/IEC 25010:2023, los atributos de calidad relevantes (Adecuación funcional, Fiabilidad, Eficiencia de desempeño, Mantenibilidad, Capacidad de interacción, Flexibilidad, Seguridad, entre otros) susceptibles de mejora mediante automatización de pruebas.

2.4.3. Objetivo Especifico 3 – Diseñar una estrategia de automatización de pruebas funcionales y de regresión

Proponer un modelo de pruebas basado en criterios de cobertura, criticidad y riesgo, que contemple los flujos críticos del sistema bajo prueba, utilizando herramientas como Cypress y buenas prácticas de diseño de pruebas automatizadas.

2.4.4. Objetivo Especifico 4 – Implementar un prototipo funcional de la estrategia de testing automatizado

Desarrollar un entorno demostrativo en el que se ejecute una suite automatizada de pruebas funcionales y de regresión, integradas a través de GitHub Actions en un pipeline CI/CD hospedado en la nube.

2.4.5. Objetivo Especifico 5 – Validar la efectividad de la estrategia mediante análisis comparativo y evidencia técnica

Evaluar el impacto de la solución propuesta mediante métricas, reportes de ejecución y simulaciones de regresión, comparando el escenario con y sin automatización.

2.4.6. Objetivo Especifico 6 – Documentar y fundamentar la aplicabilidad del modelo a otros sistemas equivalentes

Reflexionar sobre la posibilidad de adaptar la estrategia diseñada a otros entornos de software con flujos funcionales críticos, evaluando su escalabilidad y transferibilidad como práctica recomendada.

2.5. Hipótesis o modelo de mejora

Se parte de la hipótesis de que la incorporación sistemática de pruebas automatizadas funcionales y de regresión, integradas en entornos cloud mediante herramientas actuales como Cypress y GitHub Actions, permite mejorar significativamente la calidad del software en sistemas de venta online, al reducir defectos funcionales, prevenir regresiones, y aumentar la confiabilidad de los despliegues.

Esta hipótesis se fundamenta en los siguientes supuestos técnicos y organizacionales:

1. Que los flujos funcionales críticos de un sistema web (como login, selección de productos, procesos de pago o validaciones) pueden ser formalizados, automatizados y versionados mediante suites de pruebas E2E.
2. Que la ejecución continua de dichas pruebas en entornos cloud reproducibles, a través de pipelines CI/CD, permite detectar fallas de forma anticipada y trazable ante cada modificación del sistema.
3. Que una estrategia basada en criterios de riesgo, cobertura y frecuencia de uso permite optimizar la selección de pruebas a automatizar, sin necesidad de testear exhaustivamente todo el sistema.
4. Que los resultados de la automatización pueden ser medidos mediante evidencias objetivas: reducción de tiempos de validación, detección anticipada de errores, reportes de ejecución, entre otros.

En consecuencia, se espera que la implementación de una estrategia de testing automatizado como la propuesta actúe como un modelo de mejora de la calidad, al aportar confiabilidad, eficiencia y sustentabilidad técnica a los procesos de desarrollo y entrega continua.

Este modelo será validado mediante la implementación de un prototipo funcional, que permitirá comparar escenarios con y sin automatización sobre un flujo crítico del sistema.

2.6. Propuesta de solución

La solución propuesta consiste en el diseño e implementación de una estrategia de automatización de pruebas funcionales y de regresión, orientada a validar flujos críticos

de una plataforma web de venta online. Esta estrategia se sustentará en el uso de herramientas actuales y de uso industrial, como Cypress para la construcción y ejecución de pruebas end-to-end, y GitHub Actions como plataforma de integración continua en la nube para su orquestación y trazabilidad.

La propuesta contempla automatizar los flujos funcionales que presentan mayor impacto ante fallos, tales como:

- **Inicio de sesión y autenticación de usuarios.**
- **Selección de entradas y carga en el carrito.**
- **Proceso de compra y validación de pagos.**
- **Confirmación de operación exitosa.**

Estas pruebas serán diseñadas bajo principios de testing basado en riesgos, priorizando la cobertura de funcionalidades por criticidad, frecuencia de uso y dependencia técnica. La ejecución de estas pruebas se integrará en un pipeline CI/CD, de modo que se disparen automáticamente ante cambios relevantes en el código, garantizando así un ciclo de validación continuo.

La solución no pretende abarcar la totalidad del sistema, sino actuar como modelo demostrativo, evidenciando cómo la automatización sistemática de pruebas puede:

- Prevenir errores regresivos.
- Reducir los tiempos de validación.
- Aumentar la fiabilidad del software liberado.
- Mejorar la trazabilidad de las validaciones en entornos colaborativos.

El enfoque es replicable y transferible a cualquier sistema web con flujos funcionales sensibles, por lo que esta solución actúa no solo como una intervención puntual sobre un caso de estudio, sino como una estrategia ingenieril adaptable a distintos contextos.

2.6.1. Atributos de calidad priorizados según ISO/IEC 25010

Se busca determinar los atributos de calidad más afectados y vinculados al testing funcional. De acuerdo con la norma ISO/IEC 25010:2011, el modelo de calidad del producto contempla nueve características; sin embargo, no todas son igualmente observables en el alcance del presente trabajo. Esta priorización mantiene compatibilidad con la actualización ISO/IEC 25010:2023, la cual amplía y redefine algunas subcaracterísticas sin alterar los fundamentos adoptados en este estudio. Por ello, se priorizaron aquellos atributos directamente impactados por la automatización de pruebas funcionales en sistemas transaccionales en línea: adecuación funcional, fiabilidad, eficiencia de desempeño, interoperabilidad y seguridad básica. La elección se justifica en que una falla en cualquiera de estos aspectos comprometería la operación crítica de un sitio de comercio electrónico.

Cada atributo se vincula con escenarios específicos del prototipo desarrollado. Así, la adecuación funcional se verifica mediante casos que validan login correcto/incorrecto y compra completa; La validación de reintentos idempotentes y recuperación de sesión evita la duplicación de órdenes y transmite confianza al usuario respecto de que su transacción no será procesada más de una vez; la eficiencia de desempeño se mide en tiempos de respuesta del flujo de checkout; la interoperabilidad se asegura a través de la interacción controlada con el procesador de pagos simulado; y la seguridad se aborda en el acceso a rutas protegidas y la gestión de sesiones expiradas. De esta forma, la automatización de pruebas contribuye con métricas observables y reproducibles para evaluar la calidad, en línea con el marco SQuaRE de la familia ISO/IEC 25000, que habilita la priorización de características según el contexto de uso. Asimismo, estudios de la industria (World Quality Report, PractiTest, ISTQB) coinciden en que la automatización incrementa la cobertura, reduce errores y fortalece la confiabilidad, lo que refuerza la pertinencia de centrar la evaluación en estos atributos.

2.6.2. Arquitectura de herramientas y tecnologías aplicadas

La estrategia propuesta se apoya en una arquitectura técnica moderna, compuesta por herramientas ampliamente adoptadas en la industria del software para la automatización de pruebas y la integración continua en entornos cloud. A continuación, se describe el mapa de tecnologías utilizadas, sus roles y la relación entre ellas (Ver Figura - 01):

2.6.2.1. Equipo de Desarrollo (Local)

- El **equipo de desarrollo** trabaja en nuevas funcionalidades y realiza cambios en el código fuente.
- Cada modificación se guarda mediante operaciones de **commit/push** hacia el repositorio remoto en **GitHub**.
- Esto asegura control de versiones, trazabilidad y colaboración en el equipo.

2.6.2.2. Repositorio de Código (GitHub)

- GitHub centraliza y resguarda el **código fuente**.
- Al recibir un push, se **dispara el pipeline CI/CD** configurado en **GitHub Actions**, descrito en un archivo **YAML**.
- Aquí comienza la **automatización**: no depende de intervención manual.

2.6.2.3. CI/CD en la Nube (GitHub Actions)

- **Pipeline YAML** define las etapas de integración y despliegue.
- Etapas principales:
  1. **Build App / Deploy** → compila la aplicación en Node.js + React (con Vite) y la despliega en un **entorno de pruebas**.
  2. **Run Test** → ejecuta los casos de prueba automatizados con **Cypress**.
- Esto asegura **entregas continuas**, reduciendo errores humanos y acelerando el ciclo de desarrollo.

2.6.2.4. Pruebas Automatizadas (Cypress + Mochawesome)

- **Cypress** ejecuta pruebas end-to-end sobre la aplicación desplegada en la nube.
- Los resultados se registran en múltiples formatos (**JSON, HTML, Mochawesome Report**) para su análisis.
- Los reportes pueden ser consumidos tanto por el **equipo QA** como por el **Product Owner**, facilitando la comunicación del estado de calidad.

2.6.2.5. Gestión de Incidencias (Git Hub Project)

- Ante fallos en las pruebas, el sistema reporta automáticamente a **Git Hub Project**, generando un **issue** con trazabilidad al commit y al test que falló.
- Esto introduce un **circuito cerrado de feedback**, clave en metodologías ágiles.

2.6.2.6. Equipo de QA (Local)

- El equipo de QA **desarrolla y mantiene los casos de prueba** en Cypress, también versionados en GitHub.
- Sus aportes complementan el trabajo del equipo de desarrollo.
- QA analiza los reportes de Mochawesome y valida si los errores son **defectos reales** o falsos positivos.

2.6.2.7. Product Owner (Local)

- El PO recibe visibilidad del estado de calidad a través de Jira y los reportes generados.
- Su rol clave es validar y aprobar/rechazar entregas.
- Este punto asegura la alineación con los objetivos del negocio y cierra el ciclo de calidad.

2.6.2.8. Beneficios del Flujo

- Automatización completa desde commit hasta validación.
- Trazabilidad: cada bug está ligado a un commit y a un test.
- Colaboración efectiva entre Dev, QA y PO.
- Escalabilidad: el pipeline puede extenderse a más pruebas, despliegues en producción, etc.

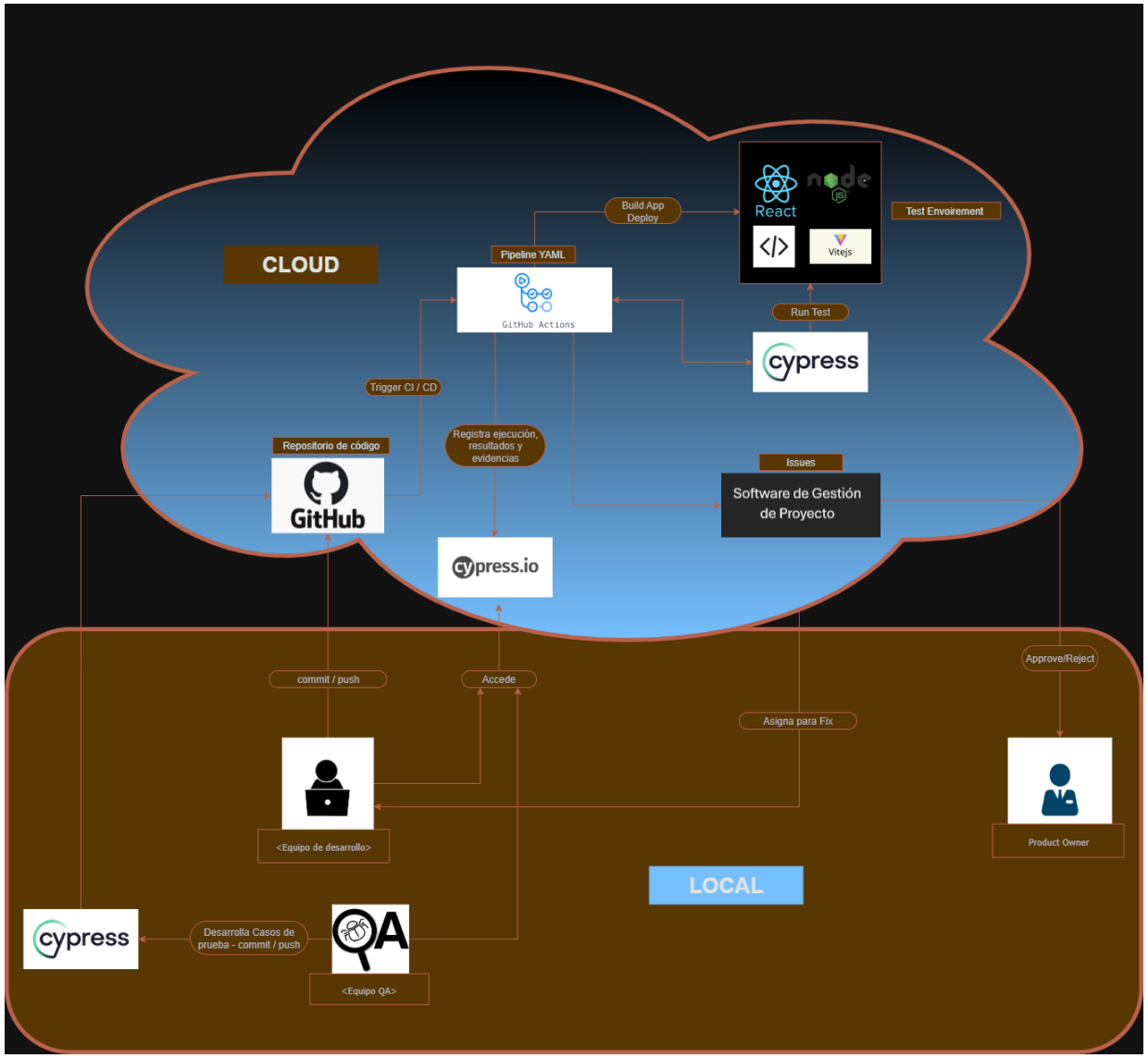

Figura 01 – Workflow e2e

2.6.3. Posibilidad de integración con gestión de calidad

Como extensión del modelo propuesto, se contempla la posibilidad de integrar los resultados de las pruebas automatizadas con una herramienta de gestión de calidad de código abierto como **Jira**, lo cual permitiría documentar automáticamente los errores detectados, vincularlos a tareas correctivas y establecer un ciclo de mejora continua basado en evidencia.

2.7. Alcances y limitaciones

2.7.1. Alcances

Este proyecto de grado se centra en el diseño, desarrollo y validación de una estrategia de pruebas automatizadas orientada a mejorar la calidad del software en sistemas de venta online. Los principales alcances son:

- Automatización de pruebas funcionales y de regresión sobre flujos críticos del sistema: login, selección de entradas, proceso de compra y confirmación.
- Implementación de un prototipo funcional de pruebas automatizadas con Cypress, ejecutadas en un pipeline CI/CD utilizando GitHub Actions.
- Evaluación del impacto de la automatización en la reducción de errores, mejora de trazabilidad y eficiencia operativa.
- Fundamento teórico basado en estándares internacionales (ISO/IEC 25010) y en prácticas actuales de calidad de software y testing.
- Generación de evidencia técnica que permita comparar escenarios con y sin automatización.
- Redacción de conclusiones y recomendaciones aplicables a sistemas similares, más allá del caso específico utilizado como referencia.

2.7.2. Limitaciones

Este proyecto no pretende:

- Implementar pruebas automatizadas sobre la totalidad del sistema o cubrir el 100% de los flujos funcionales.
- Incluir pruebas de carga, estrés, rendimiento o seguridad avanzada (no funcionales).

- Validar integraciones con servicios de terceros (por ejemplo, pasarelas de pago reales).
- Desarrollar un sistema de venta online desde cero; se parte de un entorno controlado, mockeado o parcial.
- Sustituir prácticas de testing exploratorio, revisión manual o pruebas de aceptación de usuarios, que son complementarias al testing automatizado.

La solución se acota a un enfoque de pruebas funcionales automatizadas en el contexto de validación técnica, con énfasis en la demostración del modelo de mejora y su aplicabilidad en proyectos reales.

3. Metodología

3.1. Enfoque metodológico general

El presente trabajo de grado se enmarca en una metodología de tipo proyectual, aplicada, propia de las disciplinas de ingeniería de sistemas. El objetivo no es únicamente investigar una problemática, sino diseñar, implementar y validar una solución técnica concreta basada en estándares, herramientas y buenas prácticas actuales del desarrollo de software.

Desde el punto de vista metodológico, el enfoque adoptado puede caracterizarse como:

- **Aplicativo:** se orienta a resolver un problema técnico real, mediante el desarrollo de un modelo funcional que pueda ser replicado y adaptado a otros contextos.
- **Ingenieril:** integra procesos de análisis, diseño, implementación, validación y documentación, propios del ciclo de vida del software.
- **Iterativo y controlado:** se estructura en fases consecutivas con retroalimentación entre etapas, incorporando revisión y ajuste continuo, tanto en lo técnico como en lo metodológico.
- **Empírico:** incluye la ejecución de un prototipo de pruebas automatizadas, lo que permite observar y validar los resultados esperados en un entorno controlado.

El proyecto se apoya, además, en referencias normativas (ISO/IEC 25010:2011) y en marcos prácticos reconocidos (CI/CD, DevOps, automatización con Cypress y GitHub Actions), con el objetivo de asegurar rigurosidad técnica, aplicabilidad real y valor profesional.

Este enfoque metodológico permite desarrollar una solución acotada pero robusta, que no sólo responde a una necesidad concreta sino que también aporta valor al campo profesional mediante evidencia validable y transferencia potencial a otros proyectos.

3.2. Fases de desarrollo

El presente trabajo de grado se estructura en cinco fases sucesivas, organizadas bajo un enfoque de investigación aplicada con validación técnica. Cada fase tiene como finalidad aportar progresivamente a la construcción, fundamentación y verificación de una estrategia de mejora de calidad de software mediante pruebas automatizadas, sin implicar el desarrollo completo de un sistema.

Fase 1 – Análisis de la problemática y marco teórico

- Relevamiento de antecedentes, reportes de la industria y casos reales que evidencien los efectos negativos de la ausencia de testing automatizado.

- Revisión bibliográfica y normativa, con énfasis en calidad de software (ISO/IEC 25010), metodologías de testing, automatización y CI/CD.
- Caracterización del entorno objetivo: sistemas transaccionales web con flujos críticos.

Producto: Informe teórico-técnico con fundamentación del problema, impacto, marco normativo y estado del arte.

Fase 2 – Diseño del modelo de solución

- Definición de los objetivos funcionales de la estrategia de testing.
- Selección de herramientas (Cypress, GitHub Actions) en función de criterios técnicos y de buenas prácticas.
- Diseño de casos de prueba representativos de flujos críticos (ej.: login, compra), definidos bajo un enfoque de cobertura por riesgo.
- Especificación del pipeline CI/CD a utilizar para ejecutar las pruebas en entorno cloud.

Producto: Modelo de solución documentado, con estructura de pruebas, flujos seleccionados y justificación técnica de cada componente.

Fase 3 – Construcción del prototipo demostrativo

- Implementación acotada de los casos de prueba diseñados, empleando Cypress en un entorno de pruebas controlado.
- Configuración de GitHub Actions para la ejecución automatizada de dichos casos.
- Simulación de escenarios controlados (con y sin cambios regresivos) para verificar el comportamiento del modelo propuesto.

Producto: Prototipo funcional parcial que permite visualizar la estrategia y recoger evidencia de validación. No implica el desarrollo de un sistema completo ni despliegue en producción.

Fase 4 – Validación técnica y análisis comparativo

- Ejecución del prototipo en distintos contextos de prueba.
- Recolección de métricas: detección de errores, generación de logs, tiempo de validación, cobertura alcanzada.
- Comparación objetiva de resultados entre escenarios con y sin automatización.

Producto: Informe de validación que respalda la hipótesis mediante evidencia empírica reproducible.

Fase 5 – Sistematización y conclusiones

- Consolidación de resultados obtenidos, lecciones aprendidas y hallazgos significativos.
- Análisis crítico de la aplicabilidad y escalabilidad del modelo a otros sistemas con flujos equivalentes.
- Redacción del documento final y presentación para evaluación.

Producto: Informe final del proyecto, con todos los anexos, evidencia técnica y propuesta de mejora formalizada.

3.3. Técnicas e instrumentos

El presente trabajo hace uso de un conjunto de herramientas tecnológicas y metodológicas que permiten llevar adelante el diseño, ejecución y validación del modelo de pruebas automatizadas propuesto. La selección de cada instrumento responde a criterios de eficacia, alineación con buenas prácticas de la industria, disponibilidad y capacidad de integración en entornos de entrega continua.

3.3.1. Cypress

Framework de código abierto para la automatización de pruebas **end-to-end** en aplicaciones web. Se ejecuta en el navegador, lo que permite validar flujos funcionales desde la perspectiva del usuario real.

Se eligió por su:

- Facilidad de instalación y configuración.
- Amplio soporte para pruebas funcionales y de regresión.
- Capacidad de interceptar APIs, controlar el DOM y generar evidencia automática (logs, videos, screenshots).
- Integración fluida con herramientas de CI/CD.

3.3.2. GitHub Actions

Plataforma de **automatización de flujos CI/CD** (Integración y Entrega Continua) integrada al ecosistema de GitHub.

Permite orquestar la ejecución de pruebas cada vez que se produce un evento sobre el repositorio (push, PR, etc.).

Se utiliza para:

- Automatizar la ejecución de las pruebas Cypress.
- Ejecutar en runners cloud sin necesidad de servidores locales.
- Registrar los resultados y errores de cada ejecución.
- Asegurar trazabilidad y reproducibilidad del proceso de testing.

3.3.3. GitHub (repositorio)

Repositorio central de código fuente y scripts de prueba. Además de facilitar el versionado y la colaboración, es el punto de disparo de los flujos de automatización mediante GitHub Actions.
Se utiliza también para:

- Almacenar workflows YAML.
- Configurar GitHub Secrets (tokens, claves).
- Generar trazabilidad del historial de pruebas.

3.3.4. JavaScript

Lenguaje base para la escritura de pruebas en Cypress. Es el lenguaje nativo del entorno Node.js y del navegador, lo que lo convierte en un estándar de facto para pruebas front-end.
Su flexibilidad y ecosistema lo hacen ideal para scripting de pruebas modulares y reutilizables.

3.3.5. Entorno simulado / controlado

Se emplea un sistema de venta online acotado (mock o instancia parcial) que permite reproducir flujos funcionales críticos sin necesidad de un sistema productivo.

Este entorno permite:

- Ejecutar pruebas sin riesgos operativos.
- Controlar escenarios específicos (login exitoso, error de pago, etc.).
- Validar regresiones en condiciones reproducibles.

3.3.6. Herramientas de generación de evidencia

- **Cypress Dashboard** o salida por consola/JUnit XML.
- Capturas de pantalla automáticas en caso de fallo.
- Videos de ejecución paso a paso.
- Reportes generados para análisis comparativo.

3.3.7. Sistema de gestión de calidad (GitHub Projects)

GitHub Projects es una herramienta de gestión de proyectos integrada dentro del ecosistema de GitHub, diseñada para planificar, organizar y dar seguimiento al trabajo asociado al desarrollo de software.

Funciona como un tablero dinámico (*kanban board*) que permite visualizar y controlar el estado de las tareas o incidencias mediante tarjetas (issues) distribuidas en columnas personalizables - por ejemplo: *To Do, In Progress, Done*.

Cada tarjeta puede vincularse directamente con elementos del repositorio, como commits, pull requests o workflows de GitHub Actions, lo que posibilita una trazabilidad completa entre el código, las pruebas y la gestión del proyecto.

Entre sus principales características se destacan:

- Integración nativa con GitHub Actions, lo que permite automatizar la actualización de estados o etiquetas de tarjetas según el resultado de pipelines de CI/CD.
- Colaboración en tiempo real entre desarrolladores, testers y product owners, sin salir del entorno de desarrollo.
- Visualización y priorización ágil de tareas mediante vistas tipo tabla, tablero o cronograma.
- Gestión basada en datos, gracias a filtros, automatizaciones y métricas internas.

En el contexto de esta tesis, GitHub Projects cumple el rol de software de gestión de calidad y seguimiento de incidencias, actuando como interfaz de control donde los resultados de las pruebas automatizadas se reflejan de forma automática, garantizando trazabilidad y transparencia en el proceso de aseguramiento de calidad.

3.3.8. Relación entre las herramientas

Cada tecnología cumple un rol dentro del flujo automatizado, actuando en conjunto para garantizar la validación continua del sistema:

1. Cypress define los casos de prueba funcionales.
2. GitHub aloja el código y dispara los flujos.
3. GitHub Actions despliega el codigo fuente nuevo generado por el equipo de desarrollo y ejecuta los tests en cloud cada vez que hay cambios.
4. El sistema simulado responde a los tests.
5. Los resultados se registran y analizan.
6. La integración con el software de gestión de proyectos fue implementada de manera efectiva dentro del pipeline YAML. Ante la ejecución de pruebas exitosas, el sistema automatiza el flujo de gestión moviendo el issue correspondiente a estado Done, asegurando la trazabilidad de extremo a extremo entre código, pruebas y gestión de calidad."

3.4. Validación de la solución

La validación de la propuesta se basa en la ejecución empírica de pruebas automatizadas sobre flujos funcionales críticos de un sistema de venta online, bajo condiciones controladas. Se busca comprobar que la implementación de una estrategia de testing automatizado en entorno cloud permite **mejorar atributos claves de calidad del software**, especialmente funcionalidad, confiabilidad y eficiencia operativa.

3.4.1. Objetivo

Confirmar que el modelo propuesto:

- Detecta errores funcionales y regresiones con mayor anticipación.
- Reduce el tiempo de validación frente al proceso manual.
- Genera trazabilidad, reproducibilidad y evidencia documentada.
- Puede integrarse a herramientas de gestión para un control completo del ciclo QA–Dev.

3.4.2. Escenario de prueba

Para los fines de este trabajo se desarrolló un mini sistema de comercio electrónico simulado, cuyo propósito es brindar un entorno controlado y representativo de un caso de uso real para la aplicación de prácticas de aseguramiento de la calidad de software. Este sistema no constituye un producto final orientado a la explotación comercial, sino un caso

de estudio académico que permite someter un flujo de negocio completo a pruebas automatizadas en la nube.

A nivel funcional, el sistema ofrece un catálogo de productos que pueden ser visualizados y seleccionados por el usuario, un carrito de compras que registra los ítems elegidos y sus cantidades, y un proceso de checkout que incluye la captura de datos del comprador y la simulación de un pago con tarjeta. El módulo de pago se implementa mediante un procesador falso, diseñado para emular distintos escenarios de transacción: aprobaciones, rechazos por falta de fondos y errores técnicos, lo cual resulta esencial para reproducir casos de prueba críticos. Finalmente, el sistema genera una orden de compra confirmada, que se persiste en almacenamiento local y vacía el carrito, brindando retroalimentación al usuario.

Este desarrollo cumple con el objetivo de proveer un sistema bajo prueba (System Under Test, SUT), con una complejidad suficiente para representar un flujo de negocio realista, pero al mismo tiempo acotada para mantener el control experimental. Sobre esta base se implementan y ejecutan pruebas automatizadas end–to–end con Cypress, integradas en un flujo de Integración y Entrega Continua (CI/CD) en la nube, lo cual constituye el eje central del presente trabajo de investigación.

Para comprobar la efectividad de la propuesta se definió un plan de pruebas funcionales y de regresión, ejecutado de manera automatizada con Cypress.

El plan contempla escenarios críticos del flujo de compra en el mini-sitio desarrollado:

- **Login**: validación de credenciales correctas e incorrectas.
- **Flujo de compra aprobado**: selección de productos, confirmación de orden y visualización de la pantalla de confirmación.
- **Pago rechazado y reintento**: manejo de rechazos y reintentos idempotentes.
- **Expiración de sesión**: redirección al login y recuperación del carrito tras relogin.
- **Validación de campos requeridos**: Permitir la compra luego de cumplir con información requerida

Cada caso se documenta con sus datos de prueba, pasos y resultados esperados, y fue implementado como specs E2E en Cypress para garantizar la repetibilidad y la evidencia objetiva.

El detalle completo de los casos de prueba se incluye en el ANEXO II - Plan y Guía de Pruebas – Plataforma de venta de entradas (Manual + Referencia a E2E).

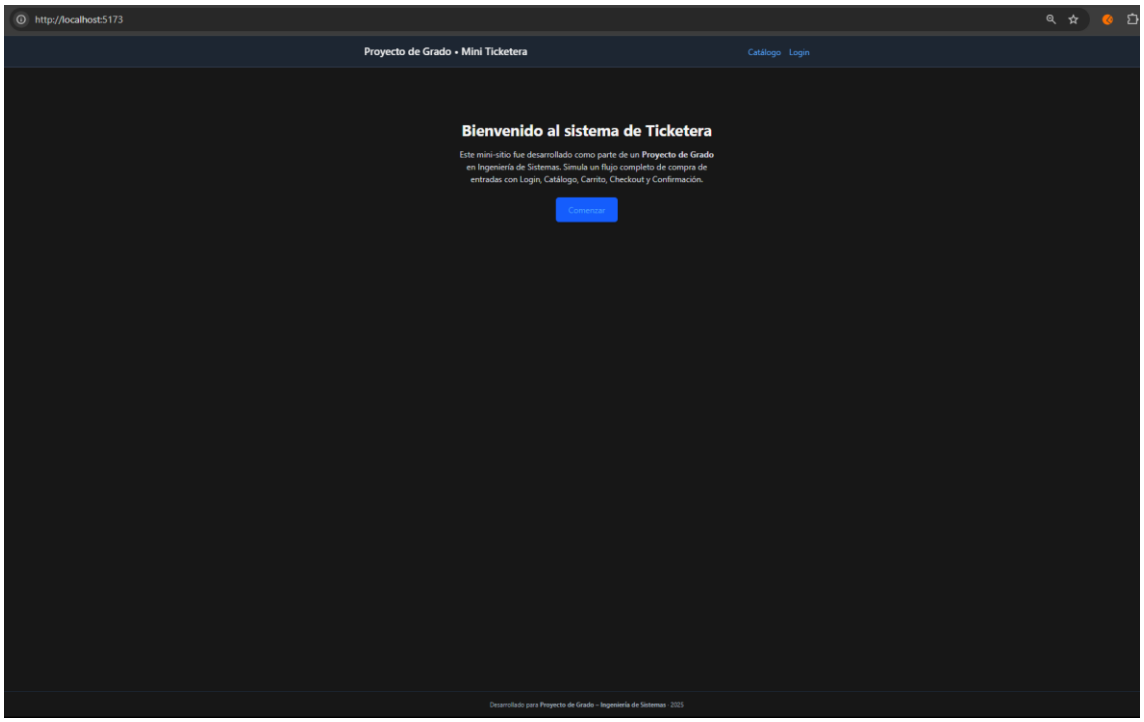

Figura 2 - index

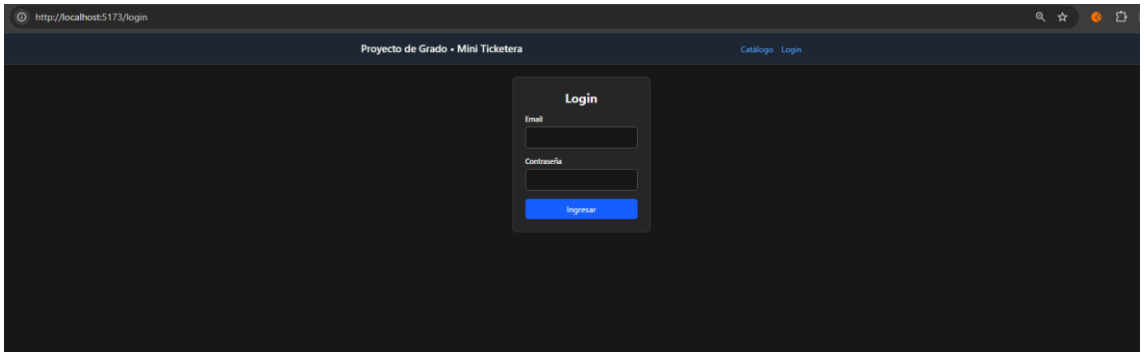

Figura 3 - Login

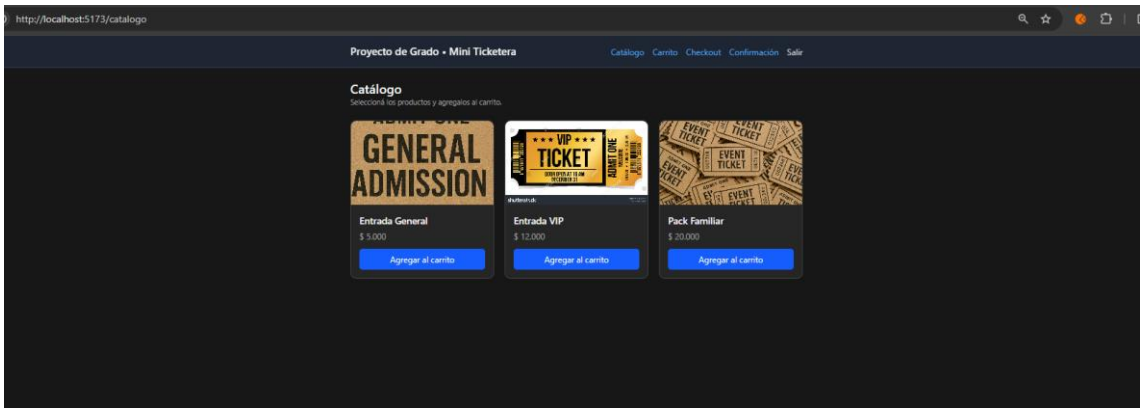

Figura 4 – Catalogo

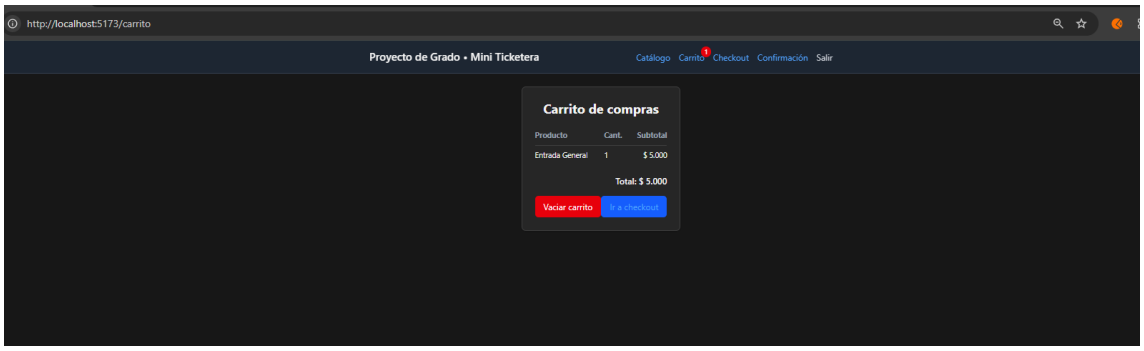

Figura 5 – Carrito de compras

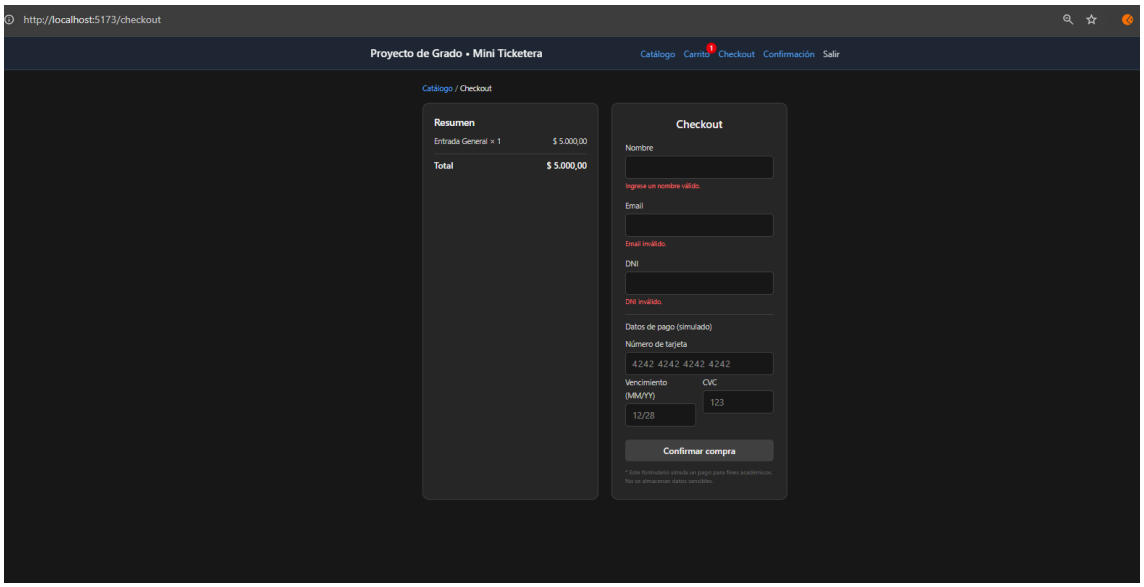

Figura 6 – Checkout

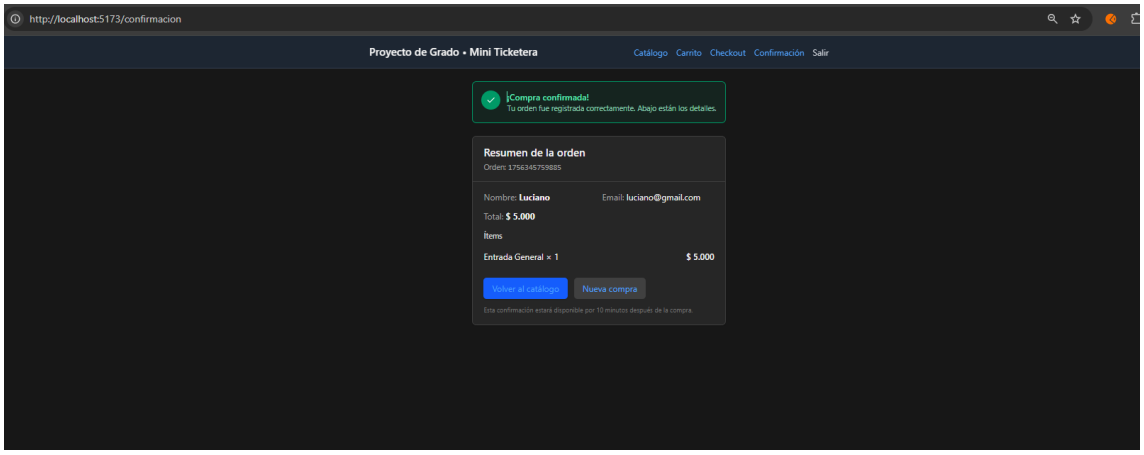

Figura 7 - Confirmación

Se utilizará un entorno simulado (mock) de sistema de venta online, donde se reproducirán flujos representativos como:

- Login de usuario.

- Selección y carga de tickets.
- Carrito de compras
- Proceso de pago.
- Confirmación de compra.

Estos flujos serán automatizados mediante Cypress y ejecutados en GitHub Actions sobre cloud cada vez que se realicen cambios en el repositorio.

3.4.3. Métodos e instrumentos

Herramienta	Rol en la validación
Cypress	Ejecución de pruebas funcionales automatizadas.
GitHub Actions	Orquestación de ejecuciones CI en entorno cloud.
Sistema simulado	Entorno de validación controlado y reproducible.
Mochawesome	Generación de reportes HTML y JSON con resultados, capturas y tiempos.
Capturas automáticas	Evidencia visual ante fallos (screenshots, video).
GitHub Proyects	Registro y trazabilidad de errores como incidencias.

3.4.4. Métricas por relevar

Métrica	Justificación
% de pruebas exitosas vs fallidas	Evalúa estabilidad del sistema frente a cambios.
Tiempos de ejecución	Mide eficiencia del proceso automatizado.
Errores detectados por regresión	Validan la hipótesis de mejora técnica.
Número de ejecuciones por commit	Asegura cobertura continua (CI).
Evidencia generada (logs, capturas)	Respaldan trazabilidad y reproducibilidad.

3.4.5. Procedimiento

La validación funcional del sistema se llevó a cabo en dos etapas:

3.4.5.1. Fase manual, que permitió identificar los flujos críticos y depurar el comportamiento de la aplicación (ver Anexo II).

3.4.5.2. Fase automatizada, implementada con Cypress (ver Anexo IV), que permitió reducir tiempos de ejecución y asegurar trazabilidad en CI/CD."

1. Se ejecutarán pruebas automatizadas ante modificaciones simuladas en el código.
2. Se documentarán los resultados de cada ejecución.
3. Se contrastarán escenarios con y sin automatización para evaluar diferencias:
    - Tiempo total de validación.
    - Detención de errores regresivos.
    - Facilidad de análisis posterior.
4. Se incluirá evidencia visual y cuantitativa en el informe final.

3.4.6. Resultados esperados

Se espera demostrar que el modelo propuesto:

- Aumenta la capacidad de detección temprana de fallos.
- Mejora la eficiencia de validación en etapas críticas del ciclo de desarrollo.
- Refuerza la trazabilidad, gracias a la integración con herramientas de reporte y gestión.
- Puede escalarse a otros entornos con estructuras funcionales similares.

3.5. Procesos, roles y responsabilidades en el workflow de pruebas

El diseño metodológico de la solución no se limita a la definición de herramientas y técnicas de validación, sino que también contempla la organización de procesos y la redistribución de responsabilidades entre los actores que intervienen en el ciclo de vida del software.

En un enfoque tradicional, los pasos de construcción, despliegue, ejecución de pruebas, reporte y gestión recaen en distintos roles (desarrolladores, equipo de operaciones, QA y

Product Owner), lo que introduce variabilidad, riesgo de errores humanos y falta de trazabilidad.

La estrategia propuesta, basada en la automatización mediante Cypress y GitHub Actions, transforma estos mismos pasos en un workflow orquestado en la nube, donde cada actividad se ejecuta de manera reproducible y controlada. De esta forma, las responsabilidades se redistribuyen: los roles humanos se enfocan en tareas de análisis, diseño y toma de decisiones, mientras que los procesos repetitivos y propensos a error se delegan al pipeline automatizado.

Este cambio metodológico permite demostrar cómo la automatización no solo agiliza la validación funcional, sino que también fortalece la fiabilidad, mantenibilidad y trazabilidad del proceso de aseguramiento de calidad, en concordancia con los atributos establecidos en la norma ISO/IEC 25010.

3.5.1. Workflow tradicional (manual por roles)

En un esquema no automatizado, los pasos críticos recaen en distintos actores:

- **Desarrollador (Dev)**
  - Compila y construye la aplicación en su entorno local.
  - Entrega artefactos manualmente al responsable de despliegue.

- **Equipo de Operaciones / DevOps**
  - Realiza el despliegue en el entorno de pruebas o producción.
  - Configura servidores y servicios dependientes.

- **Equipo de QA**
  - Ejecuta casos de prueba manualmente, siguiendo planillas de validación.
  - Documenta evidencias mediante capturas de pantalla o reportes escritos.
  - Registra defectos en el software de gestión de proyectos.

- **Product Owner (PO)**
  - Supervisa los reportes y valida manualmente el cumplimiento de requisitos.
  - Actualiza el estado de las tareas en la herramienta de gestión.

Este esquema presenta limitaciones claras: lentitud, alta dependencia de intervención humana, riesgo de omisiones, variabilidad en resultados y falta de trazabilidad consistente.

3.5.2. Workflow automatizado (estrategia propuesta)

Con la adopción de Cypress + GitHub Actions + GitHub Projects, estos mismos pasos quedan orquestados automáticamente:

- **Build y Deploy (Dev/DevOps)**
  - El pipeline CI/CD ejecuta la compilación (npm run build) y despliega automáticamente en GitHub Pages.
  - Se garantiza reproducibilidad y eliminación de errores humanos en la construcción y despliegue.

- **Pruebas (QA)**
  - Cypress ejecuta en forma automática los casos críticos (TC-001..TC-007), tanto positivos como negativos.
  - La ejecución se realiza sobre el entorno reproducible desplegado en cloud, eliminando variaciones locales.

- **Reporte (QA)**
  - Los resultados se generan en formatos estándar (Mochawesome, JUnit, Cypress Cloud) con capturas y videos incluidos.
  - Estos reportes quedan almacenados como artefactos de ejecución en GitHub Actions, accesibles para auditoría.

- **Gestión de proyectos (PO/QA)**
  - El workflow actualiza automáticamente el estado de los issues en GitHub Projects:
    - Si todas las pruebas pasan: el issue se cierra y la tarjeta se mueve a *Done*.

        - Si alguna prueba falla: el issue se etiqueta como “e2e-fail” y recibe comentarios con los enlaces a reportes y artifacts.
    - Esto asegura trazabilidad completa entre commit, ejecución de pruebas y gestión de incidencias.

- **Supervisión (PO)**
    - Accede a reportes centralizados en GitHub y Cypress Cloud, con trazabilidad objetiva.
    - No depende de reportes manuales del equipo de QA.

3.5.3. Colaboración de la estrategia en procesos y roles

La estrategia no reemplaza los roles, sino que redistribuye responsabilidades:

- Dev y DevOps dejan de ejecutar tareas rutinarias (build/deploy) y se concentran en desarrollo evolutivo.
- QA abandona la ejecución manual repetitiva y se focaliza en diseño de nuevos casos, análisis de cobertura y aseguramiento de riesgos.
- PO obtiene información confiable y en tiempo real sin necesidad de consolidar reportes manuales.

En consecuencia, se incrementa la eficiencia operativa y se reduce la dependencia de la intervención humana en pasos críticos.

3.5.4. Relación con ISO/IEC 25010

La estrategia impacta directamente en atributos de calidad definidos por la norma:

- **Adecuación funcional**: validación de los flujos críticos de negocio.
- **Fiabilidad**: ejecución reproducible y detección temprana de fallos.
- **Eficiencia de desempeño**: reducción de tiempos de validación y entrega.
- **Mantenibilidad**: pruebas versionadas junto al código, fácilmente actualizables.
- **Capacidad de interacción:** reportes claros y accesibles para todos los actores.

En síntesis, la estrategia metodológica planteada reorganiza los procesos de build, despliegue, validación y gestión, transformando un esquema manual distribuido en roles en un workflow

automatizado, reproducible y trazable. Este rediseño no elimina la participación de los actores, sino que optimiza sus responsabilidades, focalizando el esfuerzo humano en análisis y toma de decisiones. En el **Capítulo 4** se presentan las métricas y resultados obtenidos que permiten cuantificar objetivamente el impacto de esta redistribución en términos de eficiencia, fiabilidad y cumplimiento de atributos de calidad definidos por la norma ISO/IEC 25010.

* Con este desarrollo se da cumplimiento a los Objetivos Específicos N° 3 Diseñar una estrategia de automatización de pruebas funcionales y de regresión y Nº 4 - Implementar un prototipo funcional de la estrategia de testing automatizado.

4. Implementación técnica

4.1. Introducción

En esta sección se documenta la implementación técnica del sistema de e-commerce simulado desarrollado como parte del trabajo de tesis. El objetivo es mostrar, de manera ordenada y comprensible, cómo se pasó de la idea a un prototipo en producción académica, incluyendo tanto el código de la aplicación como la infraestructura de despliegue y validación automatizada.

La implementación se concibió siguiendo buenas prácticas de la ingeniería de software: control de versiones, desarrollo modular, identificación de casos de prueba críticos, automatización de pruebas y construcción de un pipeline de integración y despliegue continuo (CI/CD). Cada paso fue diseñado para demostrar en la práctica cómo se puede lograr calidad, reproducibilidad y evidencia objetiva del correcto funcionamiento de un sistema, aun en un contexto académico sin backend real.

El resultado es un MVP (Minimum Viable Product) que integra:

- Un sistema estático desarrollado con React y Vite. https://lucianocastr.github.io/ticketeraTesis/
- Pruebas automatizadas con Cypress.
- Un pipeline en GitHub Actions que compila, publica en GitHub Pages y ejecuta las pruebas end-to-end.

En lo que sigue, se describen los componentes y decisiones técnicas de manera detallada, de modo que quede clara la lógica y el valor de cada etapa en la construcción del proyecto.

4.2. Control de versiones y repositorio

El proyecto se gestionó en un repositorio de GitHub (Figura 3), que actuó como sistema de control de versiones distribuido. Esto permitió mantener un historial claro de cambios, revertir estados anteriores cuando fue necesario y garantizar trazabilidad.

Sobre este mismo repositorio se configuró GitHub Actions, el motor de integración continua de GitHub, para orquestar la construcción del sistema, su despliegue y la ejecución automatizada de pruebas end-to-end.

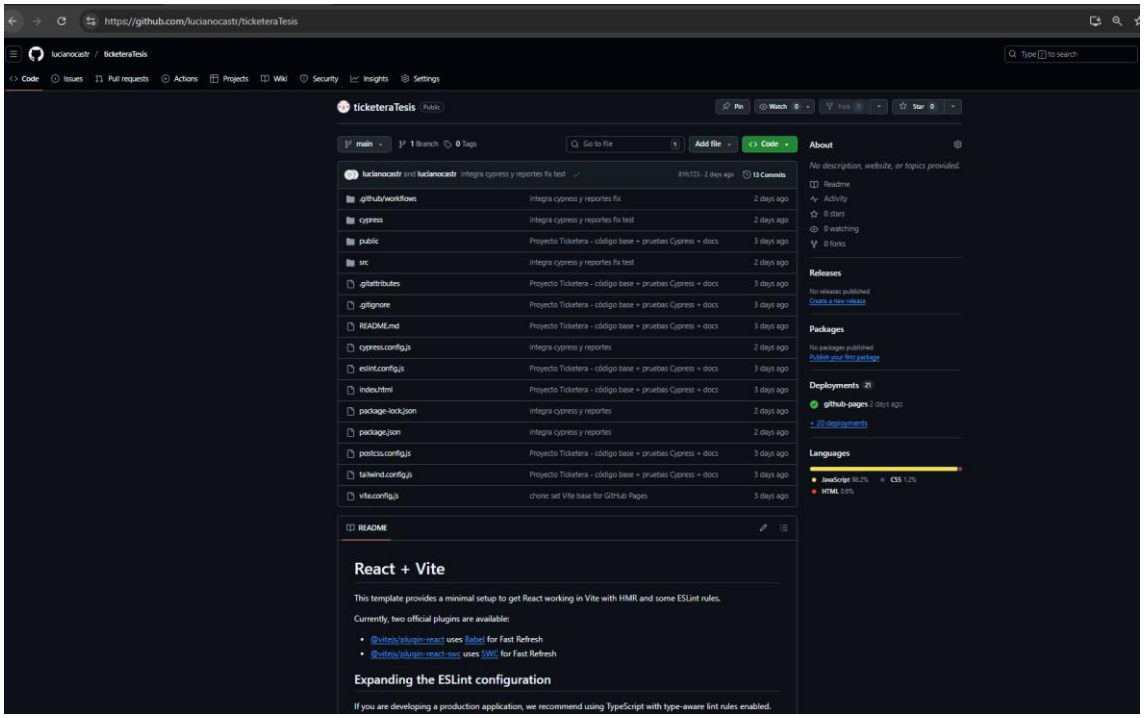

Figura 8 – Repositorio GitHub

4.3. Desarrollo del sistema

Se implementó una aplicación de e-commerce estática a modo de prototipo de plataforma de venta de entradas:

- Framework: React 18.
- Bundler: Vite 7.
- Enrutamiento: React Router v6, con guards (RequireAuth, RequireCart, RequireOrder).
- Persistencia local: uso de localStorage para simular sesiones de usuario, items en el carrito y orden de compra.
- Flujo principal: Login - Catálogo - Carrito - Checkout - Confirmación.
- Estilado: Tailwind CSS para lograr un diseño moderno y responsivo.

Al no requerirse backend, el sistema completo funciona como Single Page Application (SPA).

4.4. Identificación de casos de prueba críticos

Se definieron como críticos los siguientes casos:

- **Login**: acceso válido con usuario registrado e invalidación con credenciales incorrectas.
- **Compra de ticket**: flujo completo desde login hasta confirmación de orden.
- **Pago rechazado y reintento**: primera transacción fallida y posterior éxito.
- **Validaciones de Checkout**: habilitación/deshabilitación del botón de confirmación según formOk.
- **Carrito**: agregar ítems, visualizar totales, vaciar carrito.
- **Guards de navegación**: impedir acceso a Carrito/Checkout/Confirmación sin sesión u orden válida.

Estos casos constituyen la base de cobertura funcional del sistema.

4.5. Programación de casos de prueba automatizados

Se implementó una suite de pruebas con **Cypress** donde cada spec valida un grupo de casos de pruebas.

La suite cubre tanto pruebas positivas como negativas.

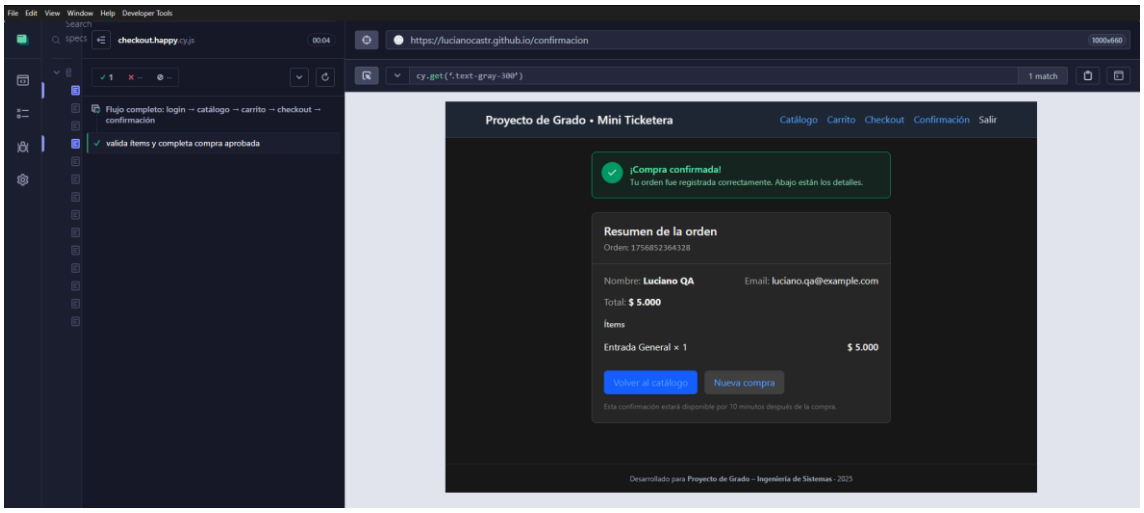

Figura 9 – Ejecución de CP automatizado Cypress modo debug

4.6. Pipeline CI/CD con GitHub Actions

4.6.1. Implementación del workflow en GitHub Actions

El núcleo de la estrategia de integración y validación automatizada se materializa en un archivo de configuración YAML (.yml) (Ver ANEXO III), alojado en el directorio .github/workflows/ del repositorio.

Este archivo representa una receta declarativa que especifica, de manera reproducible y auditable, el flujo completo de integración continua: desde la construcción del sistema hasta la ejecución de pruebas end-to-end (E2E) y la gestión de incidencias en el software de proyectos.

A continuación, se describen los principales componentes del workflow y su función:

4.6.1.1. Disparadores de ejecución

El workflow se activa automáticamente ante un evento de push sobre la rama principal (main).
Esto asegura que cada nueva versión del sistema deba ser validada antes de considerarse lista, eliminando la dependencia de acciones manuales.

4.6.1.2. Concurrencia y permisos

- **Concurrencia**: se define un grupo de ejecución que evita la acumulación de múltiples pipelines en paralelo, cancelando los anteriores si existe un nuevo commit.
- **Permisos**: se adoptan los mínimos necesarios, siguiendo el principio de privilegio reducido:
    - Lectura de contenidos del repositorio.
    - Escritura en issues (para actualizar el estado de la gestión).
    - Publicación en GitHub Pages.
    - Autenticación mediante id-token.

4.6.1.3. Job Build

Este bloque realiza la compilación del sistema (Figura 10):

- Configura el entorno Node.js (versión 20.19).
- Instala dependencias con npm ci para garantizar reproducibilidad.
- Ejecuta npm run build, generando la carpeta de distribución (dist).
- Copia index.html como 404.html, requisito técnico para aplicaciones de una sola página (SPA) en GitHub Pages.
- Almacena el resultado como artefacto, disponible para etapas posteriores.

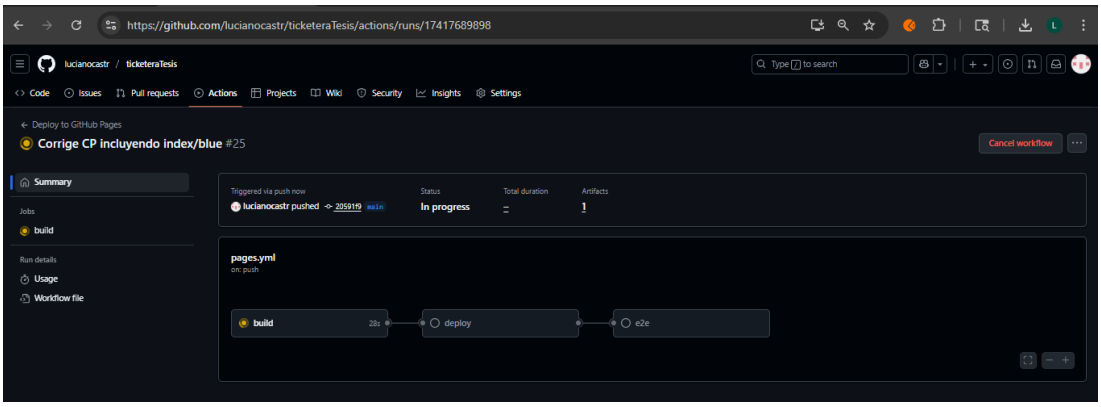

Figura 10 – Build en GitHub Actions luego de push en dev

4.6.1.4. Job Deploy

El artefacto generado se publica automáticamente en GitHub Pages, quedando disponible en un entorno cloud accesible públicamente (https://lucianocastr.github.io/ticketeraTesis/).

Este paso expone una URL de despliegue que será utilizada como base para la ejecución de pruebas automatizadas (Figura 11)..

De esta manera, las validaciones se realizan sobre un entorno real y reproducible, independiente de la máquina local del desarrollador.

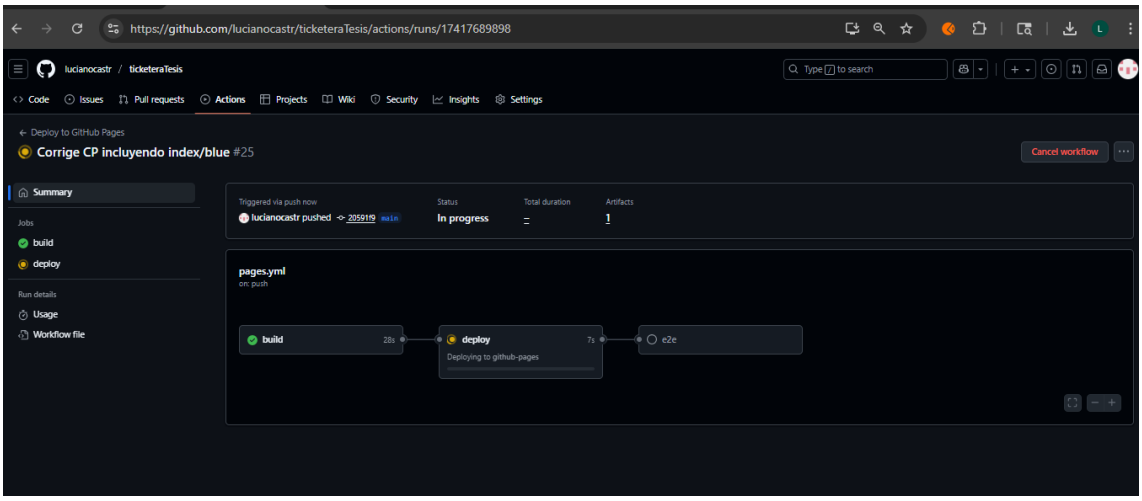

Figura 11 – Deploy en GitHub Actions luego de Build

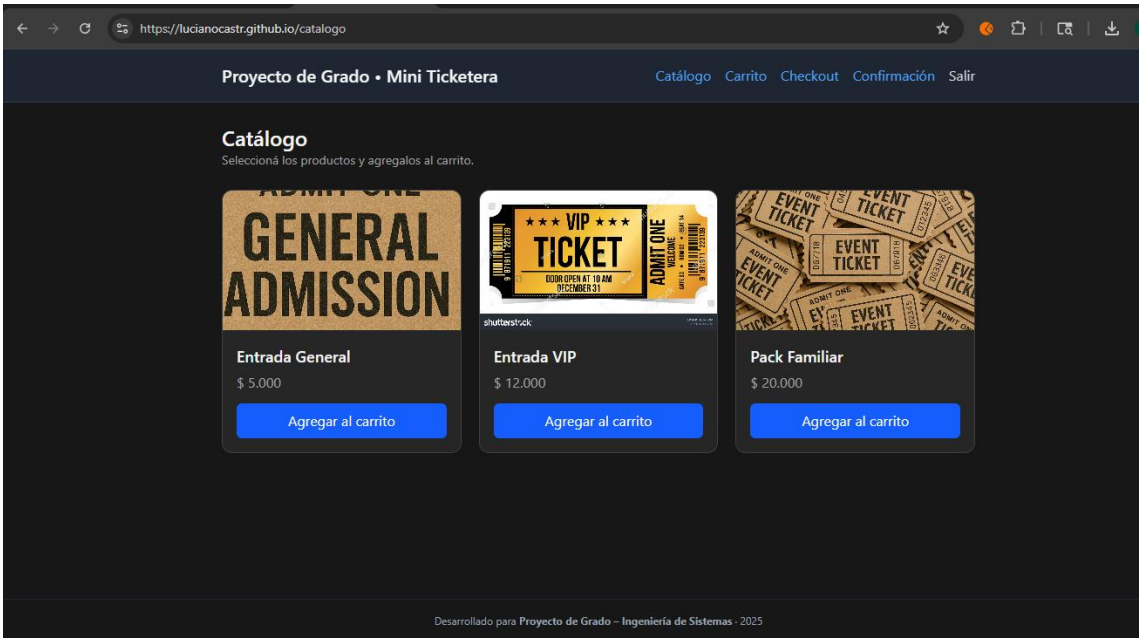

Figura 12 – Publicación en entorno de pruebas

4.6.1.5. Job E2E

En este bloque se ejecutan las pruebas funcionales automatizadas con Cypress:

- Se prepara nuevamente el entorno Node.js e instala Cypress.
- Se ejecuta la suite de pruebas E2E contra la URL desplegada en GitHub Pages.
- Los resultados generan múltiples tipos de evidencia:
    - Reportes Mochawesome (HTML/JSON).
    - JUnit (XML) para compatibilidad con otros sistemas.

- Capturas de pantalla y videos de la ejecución de pruebas.

- Los artefactos se almacenan siempre, independientemente del resultado de las pruebas, lo que asegura trazabilidad completa en caso de fallas.

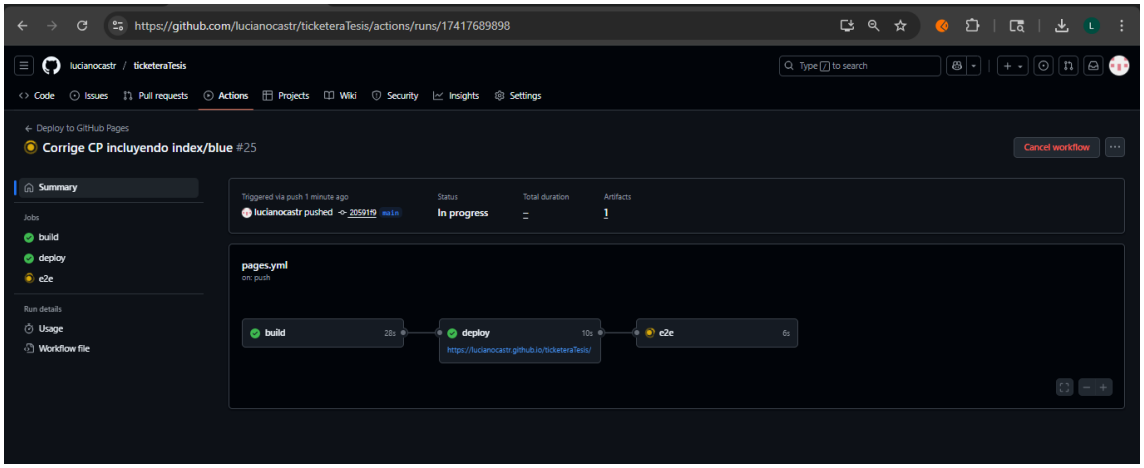

Figura 13 – Ejecución de pruebas en entorno controlado Cloud luego de Deploy

4.6.1.6. Integración con la gestión de proyectos

Un aspecto distintivo de este workflow es la integración con el sistema de gestión de proyectos (GitHub Projects/Issues):

- Si todas las pruebas se ejecutan con éxito, el workflow cierra automáticamente el issue vinculado y mueve la tarjeta a estado *Done*.

- En caso de fallos, el issue se etiqueta con “e2e-fail” y recibe un comentario automático que incluye enlaces a:

  - El run de GitHub Actions.

  - La URL del despliegue en Pages.

  - Los reportes y evidencias generadas.

Este mecanismo garantiza la trazabilidad end-to-end: desde el commit de código hasta la verificación automática y la actualización del estado en la gestión de proyectos.

4.6.2. Beneficios del enfoque

- **Reproducibilidad**: el mismo flujo corre de manera idéntica ante cada cambio.

- **Trazabilidad**: cada issue queda vinculado a commits, ejecuciones y resultados concretos.

- **Automatización**: se elimina la necesidad de intervención manual en la validación y cierre de incidencias.
- **Escalabilidad**: la arquitectura del workflow permite extenderse a nuevas pruebas, entornos o integraciones de gestión.

4.7. Deploy en GitHub Pages

Despliegue en GitHub Pages:

- En vite.config.js se definió base: '/ticketeraTesis/' para que los assets carguen correctamente bajo el subpath.
- Se configuró la copia de index.html a 404.html para que React Router pueda resolver rutas internas aunque el servidor responda 404 (mecanismo de fallback típico en Pages).

De esta forma, el sistema queda accesible en cualquier ruta (ej: /catalogo, /carrito, etc.).

4.8. Diseño y ejecución de pruebas E2E con Cypress

La validación funcional automatizada del sistema se implementó utilizando **Cypress**, herramienta de pruebas end-to-end orientada a aplicaciones web modernas. Esta elección responde a criterios de la industria vinculados con facilidad de integración en pipelines CI/CD, capacidad de interceptar peticiones de red y generación automática de evidencias (screenshots, videos, reportes).

4.8.1. Estrategia de pruebas

Se adoptó un enfoque de **testing basado en riesgos**, priorizando los flujos críticos del sistema de venta online, cuyo fallo impactaría directamente en la operación:

- Inicio de sesión y autenticación.
- Selección de entradas y carga en el carrito.
- Proceso de compra y validación de pago.
- Confirmación de la operación exitosa.

Adicionalmente, se incluyeron casos negativos (login inválido, pago rechazado) y validaciones de idempotencia (evitar duplicación de órdenes).

4.8.2. Arquitectura de pruebas

La estructura del repositorio se organizó siguiendo las convenciones de Cypress:

- cypress/e2e/: contiene los archivos de pruebas (*.cy.js).
- cypress/fixtures/: define datos estáticos de entrada.
- cypress/support/commands.js: centraliza comandos personalizados como login() o checkout().
- cypress/support/utils.js: contiene funciones auxiliares reutilizables.
- cypress.config.js: configura parámetros globales (specPattern, reporter, timeouts, integraciones).

Esta organización favorece la **mantenibilidad** y **reutilización de código**, alineada con los atributos de calidad definidos en ISO/IEC 25010.

4.8.3. Diseño de casos representativos

Cada caso se estructuró siguiendo criterios de Given/When/Then:

- Caso positivo (Happy Path): login válido, selección de producto, checkout exitoso y confirmación.
- Caso negativo: login con credenciales inválidas → mensaje de error esperado.
- Caso de idempotencia: reintento de pago con verificación de no duplicación.
- Caso de interoperabilidad: interceptación de respuestas del gateway de pago (cy.intercept()) para validar escenarios controlados.

4.8.4. Buenas prácticas aplicadas

- Uso de selectores estables (data-cy) para evitar fragilidad en las pruebas.
- Comandos custom reutilizables (cy.login(), cy.addToCart()) que simplifican el código y evitan duplicación.
- Configuración de retries automáticos en caso de tests inestables (flaky tests).
- Separación entre tests contra entorno real (deploy en Pages) y tests con mocking (interceptar API).

4.8.5. Evidencias y trazabilidad

La ejecución de Cypress genera automáticamente:

- Screenshots y videos de cada prueba.
- Reportes Mochawesome en formato JSON y HTML.
- Registros en Cypress Cloud, vinculados al commit y al run de CI/CD.

Estas evidencias son almacenadas como artefactos en GitHub Actions y comentadas en el issue correspondiente, asegurando trazabilidad de extremo a extremo.

4.8.6. Métricas observadas

El prototipo permitió cuantificar:

- Reducción del tiempo de validación: de minutos (manual) a segundos (automatizado).
- Incremento de la cobertura: validación sistemática de flujos críticos en cada cambio.
- Disminución del riesgo de regresiones: suites corren en cada push, evitando liberar cambios sin pruebas.

En consecuencia, Cypress se consolidó como herramienta clave para verificar adecuación funcional, fiabilidad e interoperabilidad, atributos centrales del modelo ISO/IEC 25010.

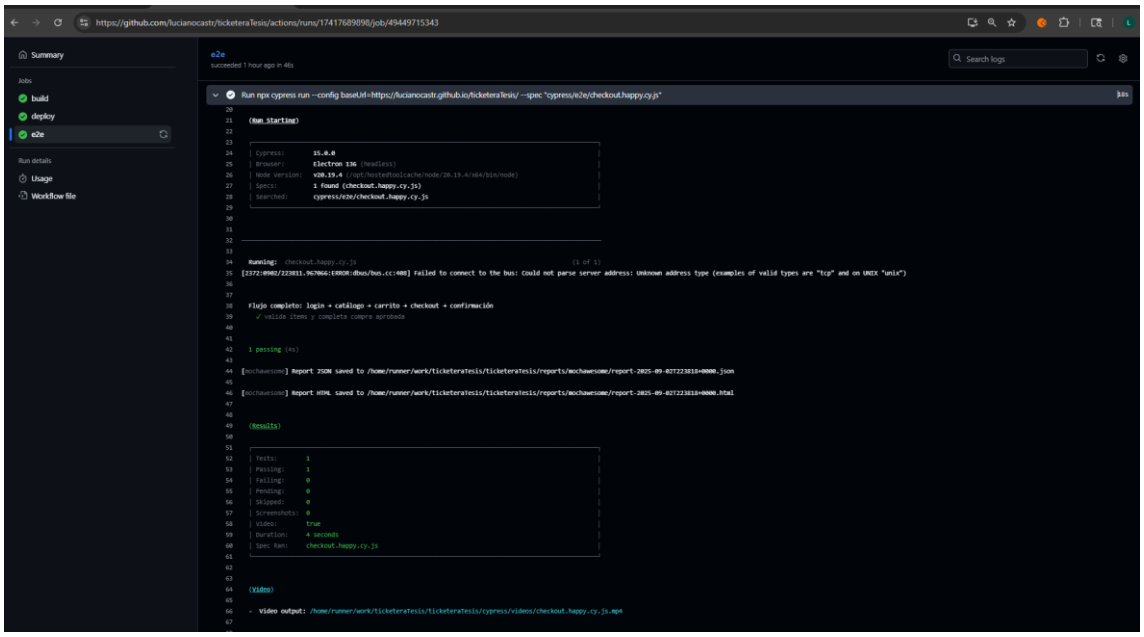

Figura 14 – Ejecución de Caso de prueba

4.9. Cypress Cloud

Cypress Cloud constituye la extensión en la nube del framework Cypress y cumple un rol clave en el proyecto, ya que aporta capacidad de registro, orquestación y trazabilidad avanzada de las pruebas automatizadas.

Entre sus funciones principales se destacan:

- **Registro centralizado de ejecuciones**: cada corrida queda asociada al commit que la originó, facilitando el análisis histórico y la comparación entre versiones.
- **Visualización de resultados en tiempo real**: la plataforma ofrece dashboards donde se consultan métricas, capturas, videos y logs de cada prueba.
- **Detección y análisis de fallos**: Cypress Cloud almacena los contextos completos de cada ejecución fallida, lo que facilita la identificación de la causa raíz y evita falsos positivos.
- **Integración con CI/CD**: se vincula directamente con GitHub Actions a través de CYPRESS_RECORD_KEY, permitiendo que las pruebas corran automáticamente en cada push y queden trazadas en el sistema de gestión de calidad.

En este proyecto, Cypress Cloud funciona como repositorio de evidencias objetivas de calidad, asegurando que cada flujo crítico validado sea reproducible, auditable y verificable frente a terceros (equipo de desarrollo, stakeholders). De esta manera, complementa la ejecución local y en pipeline, elevando la confiabilidad y la transparencia del proceso de testing automatizado.

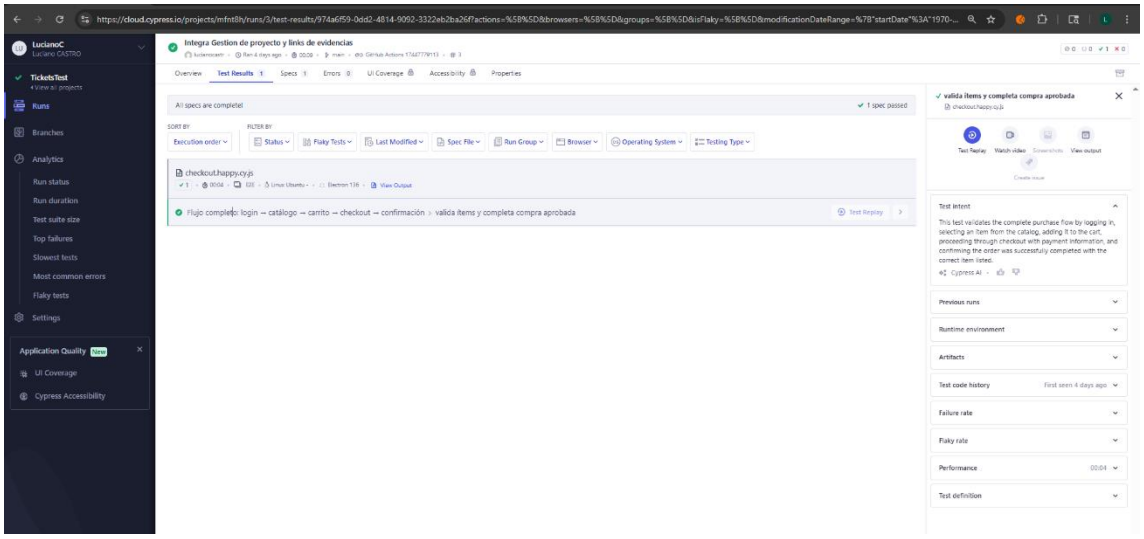

Figura 15 – Métricas, Resultados y Evidencias Cypress cloud

4.10. Integración con Gestor de Proyecto

Como parte de la solución, se implementó la integración de las pruebas automatizadas con un software de gestión de proyectos, lo que permite cerrar el ciclo de calidad de manera automática. En este proyecto se eligió GitHub Projects, aprovechando que se encuentra dentro del mismo ecosistema que el repositorio y el pipeline de CI/CD, lo que facilita la trazabilidad end-to-end sin depender de plataformas externas.

Con esta integración, cada corrida del workflow en GitHub Actions actualiza automáticamente el estado de un issue asociado:

- Si todas las pruebas pasan, el issue se cierra y la tarjeta vinculada en el tablero de Projects se mueve al estado *Done*.
- Si ocurre una falla, el issue permanece abierto y recibe comentarios automáticos con los reportes y artefactos de la ejecución.

Este mecanismo asegura que la gestión de incidencias refleje en tiempo real el estado de calidad del software, eliminando la necesidad de actualizaciones manuales y reduciendo la posibilidad de error humano.

Cabe destacar que, aunque la integración en este proyecto se realizó sobre GitHub Projects por conveniencia y coherencia tecnológica, la misma lógica puede aplicarse a otras herramientas de gestión ampliamente utilizadas en la industria, como Jira, Trello, Asana o Redmine, mediante el consumo de sus API REST. Esto evidencia que la propuesta no está restringida a una plataforma específica, sino que constituye un modelo adaptable a distintos entornos de desarrollo y gestión de calidad.

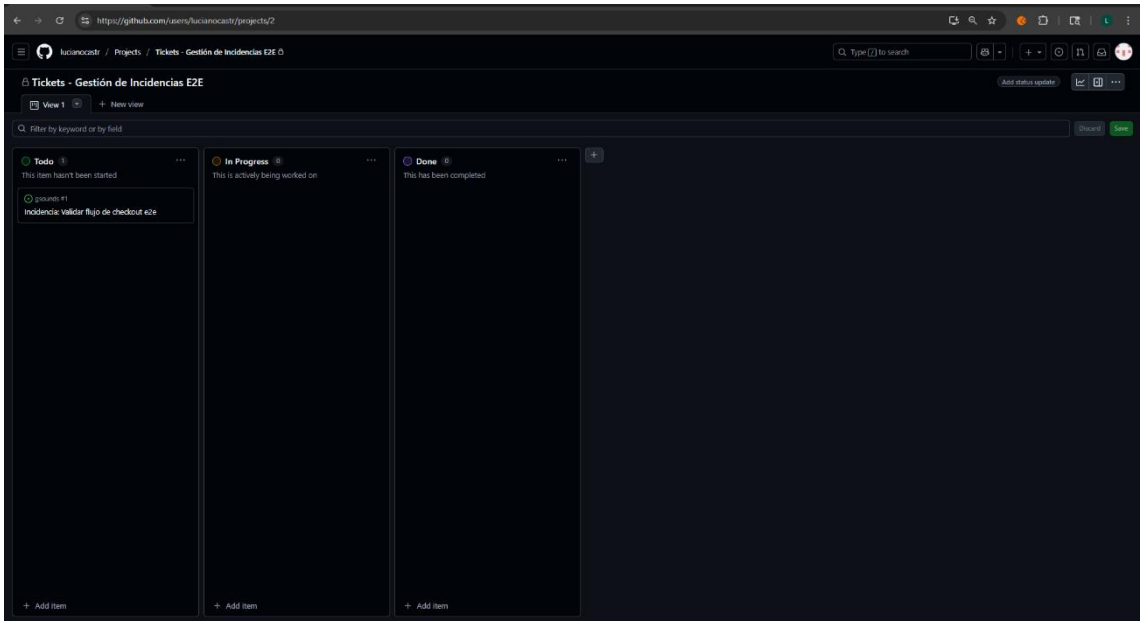

Figura 16 – Software de gestión de proyecto – Automatización de estados de issues

4.11. Resultado

Con esta implementación se logró una solución que integra:

- Desarrollo de un sistema e-commerce estático.
- Automatización de pruebas críticas con Cypress.
- Pipeline CI/CD completo en GitHub Actions.
- Deploy continuo en GitHub Pages.
- Validación end-to-end automatizada con reportes y trazabilidad en la gestión de proyectos.

La propuesta desarrollada presenta un conjunto de fortalezas que la distinguen y refuerzan su aporte al ámbito de la Ingeniería de Sistemas. En primer lugar, la automatización de pruebas en un entorno cloud controlado y reproducible asegura condiciones constantes de ejecución, eliminando las variaciones propias de entornos locales y permitiendo validar la calidad de manera confiable en cada iteración. A ello se suma la fiabilidad funcional percibida por el usuario, alcanzada mediante mecanismos como la idempotencia en los reintentos de pago, la recuperación de sesión y el manejo adecuado de casos de stock insuficiente, que transmiten confianza y reducen la incertidumbre en operaciones críticas.

Asimismo, la solución se apoya en un pipeline de integración y despliegue continuo (CI/CD) con GitHub Actions, que actúa como quality gate: únicamente se libera el prototipo cuando todos los casos de prueba E2E han sido superados, garantizando así la entrega de un sistema consistente. La integración con GitHub Projects asegura que los resultados de cada ejecución se reflejen automáticamente en la gestión de incidencias, cerrando el ciclo de calidad con trazabilidad de extremo a extremo.

De esta forma, los resultados obtenidos responden directamente a la problemática identificada en el Capítulo 1, caracterizada por errores funcionales no detectados, regresiones frecuentes y ausencia de trazabilidad en pruebas manuales. La solución propuesta demuestra que la automatización de pruebas en entornos cloud constituye un mecanismo eficaz para superar estas limitaciones y reducir el riesgo de fallos en producción.

Finalmente, los resultados alcanzados se encuentran alineados con los atributos de calidad definidos en la norma ISO/IEC 25010, evidenciándose mejoras en:

- Adecuación funcional: validación sistemática de flujos críticos.
- Fiabilidad: ejecuciones reproducibles y controladas en entornos cloud.
- Eficiencia de desempeño: reducción de tiempos de validación en comparación con pruebas manuales.
- Mantenibilidad: reutilización de pruebas, configuración declarativa del pipeline y generación automática de evidencias.

En conjunto, estas características consolidan una solución que combina rigor metodológico, aplicabilidad práctica y beneficios tangibles, constituyendo un modelo viable de mejora de calidad de software sustentado en estándares internacionales y aplicable a contextos reales de la industria.

5. Resultados y análisis

5.1. Objetivo

- Demostrar con ejecuciones reales del prototipo cómo funciona la estrategia de testing automatizado.
- Comparar escenarios (manual vs automatizado).
- Mostrar métricas, evidencias y análisis crítico.

5.2. Descripción del prototipo ejecutado

El prototipo corresponde a una plataforma web de venta de entradas, desarrollada en React + Vite, con los flujos críticos de login, selección de entradas, compra y confirmación.
Las pruebas se ejecutaron mediante Cypress integrado a GitHub Actions y desplegado en GitHub Pages, permitiendo validar la aplicación directamente en un entorno cloud reproducible.
El pipeline definido en el archivo .yml coordina automáticamente las etapas de build, deploy, testing y reporting, generando evidencia de cada ejecución.

5.3. Ejecución de pruebas manuales

Las pruebas manuales se realizaron sobre el mismo flujo funcional, registrando tiempos y observaciones.

- **Flujos cubiertos:** login, compra, checkout y confirmación.
- **Tiempo promedio:** entre **5 y 7 minutos por ciclo** completo.
- **Riesgos observados:** omisiones por error humano, variabilidad entre testers, y tiempos adicionales para capturas y registro de incidencias.

5.4. Ejecución de pruebas automatizadas

La estrategia automatizada replicó los mismos flujos mediante **Cypress E2E**, orquestado en GitHub Actions:

- Tiempo de ejecución promedio: 30 a 40 segundos por ciclo completo.

- Cobertura: casos críticos (TC-001 a TC-007) con validación de flujos positivos y negativos.
- Evidencia generada: reportes HTML Mochawesome, capturas automáticas y videos, almacenados como artefactos en Actions y Cypress Cloud.
- Integración con gestión: al finalizar, el pipeline actualiza automáticamente el estado del *issue* correspondiente en GitHub Projects.

5.5. Comparación de escenarios (Manual vs Automatizada)- Tabla 1

Aspecto	Manual (sin automatización)	Automatizado (Cypress + GitHub Actions + GitHub Projects)
Tiempo por ejecución	~5–7 min por ciclo completo (login + compra + checkout).	~30–40 seg por ciclo en pipeline cloud.
Cobertura	Casos básicos definidos por QA, limitados por disponibilidad de tiempo.	Casos críticos + regresiones + flujos negativos, ejecutados en cada commit.
Reproducibilidad	Baja: depende del tester, entorno local y disciplina.	Alta: entorno cloud controlado, ejecución idéntica en cada corrida.
Evidencia	Capturas manuales y reportes escritos a mano.	Screenshots, videos y reportes automáticos (Mochawesome, JUnit, Cypress Cloud).
Trazabilidad	Escasa: QA debe registrar manualmente defectos/incidencias.	Completa: commit → ejecución → reporte → issue/tarjeta en Projects.
Esfuerzo humano	Alto: QA ejecuta casos, documenta y actualiza gestión.	Bajo: QA diseña casos, pipeline ejecuta y reporta automáticamente.
Defectos detectados	Riesgo de omisiones; defectos regresivos suelen escapar a producción.	Detección temprana de regresiones en cada cambio, antes de liberar.
Confiabilidad de resultados	Variable: errores humanos en ejecución o registro.	Constante: evidencias objetivas almacenadas en cloud y auditables.

Tabla 1 – Contraste entre ejecución manual y automatizada de pruebas

5.5.1. Registro de resultados y métricas (comparación Manual vs E2E)

Con el objetivo de cuantificar la mejora obtenida a partir de la automatización de las pruebas, se registraron los resultados de ejecución correspondientes a las modalidades **manual** y **end-to-end (E2E)** automatizada mediante **Cypress**.

Para cada modalidad se midieron los siguientes indicadores:

- Tiempo total de ejecución
- Cobertura funcional de flujos críticos
- Tasa de éxito (pass rate)
- Número de evidencias generadas
- Intervención humana requerida

Los valores promedio se obtuvieron a partir de cinco corridas consecutivas (n = 5) en cada modalidad, ejecutadas en entornos equivalentes.

5.5.2. Ejecuciones Manuales – Tabla 2

ID	Caso Manual	Tiempo Prom. Ejecución	Tiempo Prom. Registro Evidencia	Total Prom. por Caso	Ejecuciones	Resultado	Evidencia
TC-001	Login correcto / inválido	3 min	4 min	7 min	3	OK	Capturas Catálogo / Error
TC-002	Happy path de compra	3 min	4.4 min	7.4 min	5	OK	Capturas Carrito / Confirmación
TC-003	Pago rechazado → reintento aprobado	3.2 min	4.5 min	7.7 min	5	OK	Capturas Error / Confirmación
TC-004	Carrito: agregar, totales, vaciar	2.1 min	4.5 min	6.6 min	5	OK	Capturas Carrito lleno / vacío
TC-005	Catálogo disponible luego del login	1.3 min	3.5 min	4.8 min	5	OK	Capturas Catálogo
TC-006	Sesión expirada y recuperación	4 min	3 min	7 min	5	OK	Capturas Login / Checkout

Tabla 2 – Detalle de ejecuciones manuales

5.5.2.1. Totales aproximados Ejecución Manual

- Ejecuciones: 28
- Tiempo total estimado: ≈ 140 min
- Pass rate: 100 %
- Falsos positivos/negativos: 0
- Media total por caso: 6.7 min

5.5.3. Ejecuciones Automatizadas – Tabla 3

ID	Caso	Archivo Cypress	Tiempo Prom. (min)	Pass Rate	Reducción vs. Manual	Evidencias
TC-001	Login válido/inválido	auth.cy.js	0.63	100%	80%	Screenshots + video
TC-002	Happy path compra	checkout.happy.cy.js	2.25	100%	82%	Video completo
TC-003	Pago rechazado/reintento	payment.decline-retry.cy.js	1.6	100%	79%	Screenshots + video
TC-004	Carrito agregar/vaciar	cart.cy.js	0.87	100%	83%	Screenshots
TC-005	Catálogo luego de login	catalog.cy.js	0.66	100%	70%	Screenshot
TC-006	Sesión expirada y recuperación	checkout.validations.cy.js	1.3	100%	85%	Screenshots + video

Tabla 3 – Detalle de ejecuciones automatizadas

5.5.4. Mochawesome - Reporte Ejecución de pruebas automatizadas – Figura 17

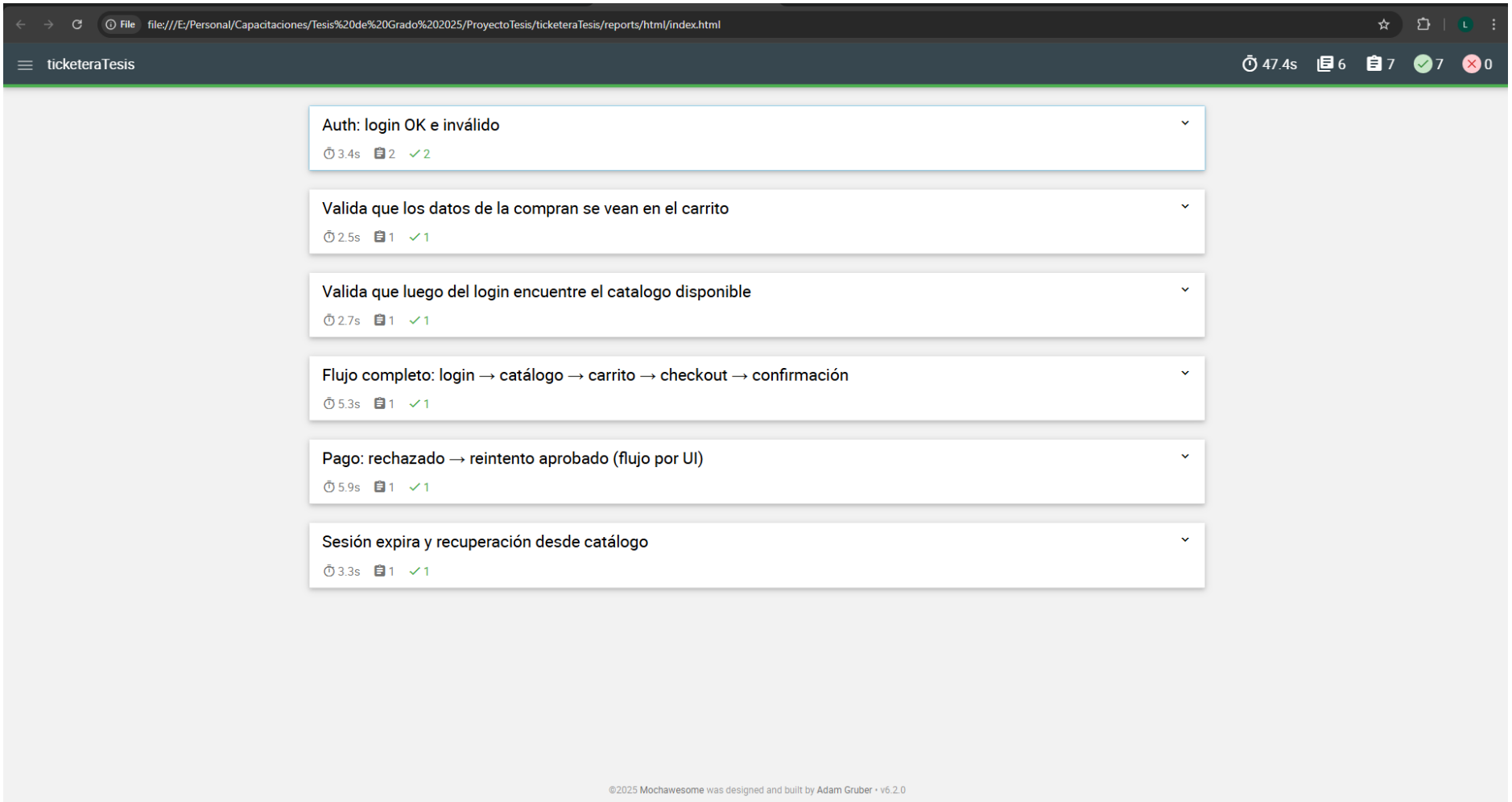

Figura 17 – Reporte Mochawesome

5.5.5. Cypress Cloud - Reporte Ejecución de pruebas automatizadas – Figura 18

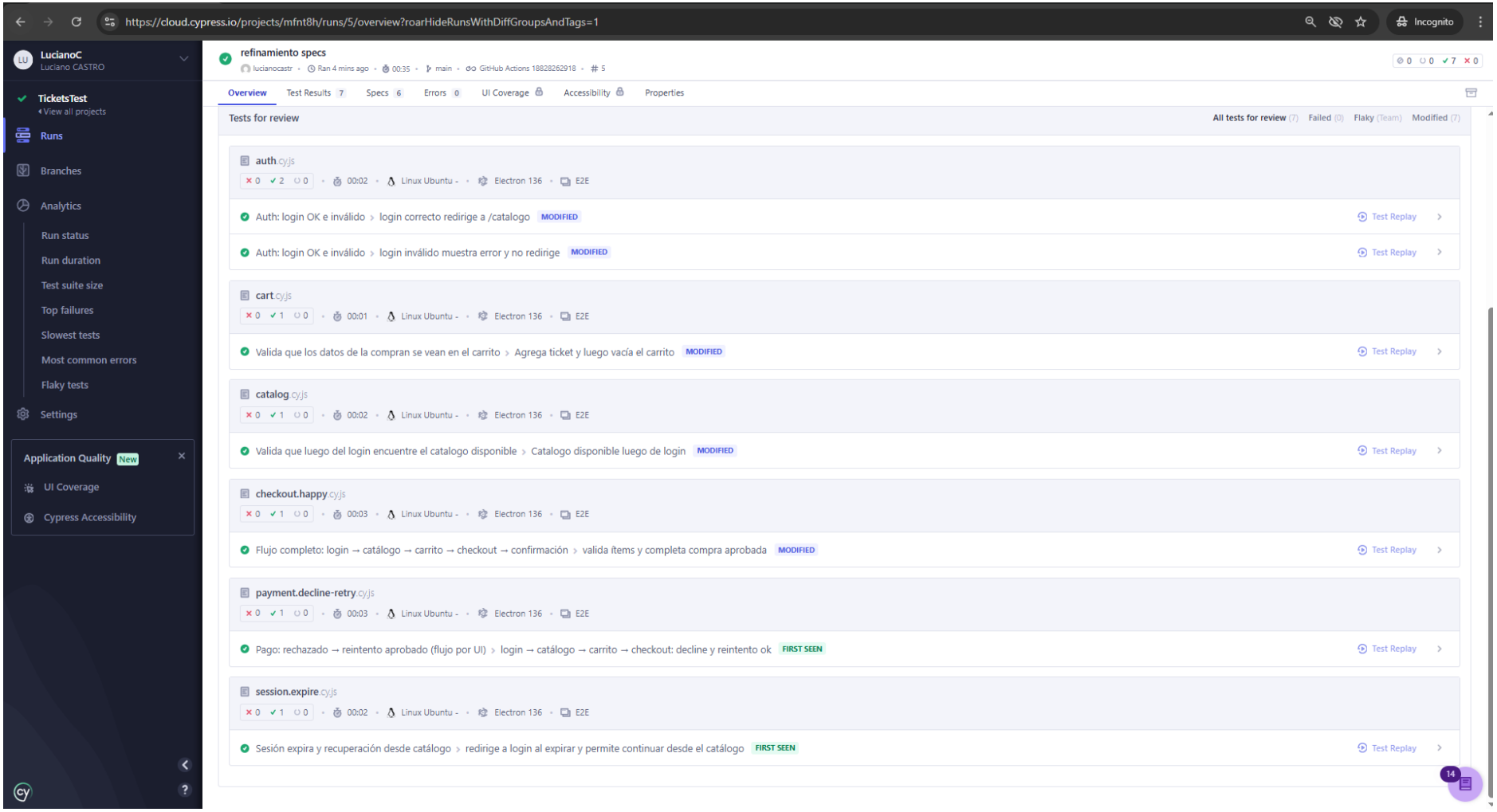

Figura 18 – Reporte Cypress Cloud

5.5.5.1. Totales aproximados ejecución automatizada

- Tiempo total manual (6 casos, promedio 140 min).
- Tiempo total automatizado (6 casos, promedio 36 min).
- **Reducción global: ≈ 74 %**.
- Pass rate promedio: **100 %**.
- Falsos positivos/negativos detectados: **0**.
- Evidencias generadas automáticamente: **30+ artefactos (videos, screenshots, reportes)**.

La comparación entre la ejecución manual y la automatizada de pruebas permite observar diferencias significativas en los aspectos más relevantes del aseguramiento de calidad. En términos de tiempo de validación, mientras que un ciclo completo de pruebas manuales (login, compra y checkout) demanda varios minutos de ejecución y registro, la estrategia automatizada logra validar el mismo flujo en cuestión de segundos dentro del pipeline cloud, garantizando un ahorro sustancial de tiempo y esfuerzo.

Otro aspecto crítico es la cobertura. En el enfoque manual, la ejecución suele limitarse a casos básicos definidos por QA, lo que deja expuestas funcionalidades críticas ante regresiones. La automatización, en cambio, permite ejecutar de forma sistemática los flujos centrales y sus variantes negativas, incorporando además pruebas de regresión que se disparan en cada commit. Esto incrementa la capacidad de detección temprana de defectos y fortalece la fiabilidad del sistema.

En cuanto a reproducibilidad y trazabilidad, la automatización elimina las variaciones entre testers y entornos, generando evidencia objetiva (screenshots, videos y reportes) integrada automáticamente en GitHub Projects. De este modo, el proceso gana consistencia, trazabilidad completa y capacidad de auditoría.

La reducción del esfuerzo humano es igualmente notoria: los testers dejan de ejecutar manualmente casos repetitivos y documentar resultados, para concentrarse en el diseño de nuevos escenarios y el análisis estratégico de cobertura. En conjunto, los resultados reflejan un cambio cualitativo en la forma de validar la calidad: de un proceso secuencial y dependiente de personas, a un esquema automatizado, reproducible y sustentado en evidencia.

5.6. Cierre del ciclo CI/CD

Tras la finalización de cada corrida del pipeline de integración continua (CI/CD), el sistema GitHub Actions emite una notificación automática que documenta el resultado de la ejecución end-to-end (E2E).

La Figura 19 muestra la evidencia correspondiente a la finalización de un ciclo de pruebas automatizadas, donde el bot github-actions[bot] registra en el *Issue* asociado del proyecto (*RunSpecsAll – Issue #3*) la confirmación del éxito de la ejecución, la generación de artefactos de reporte y el cambio de estado de la tarjeta a “Done”.

Esta notificación confirma la correcta sincronización entre el pipeline, los resultados de las pruebas y la gestión de tickets en GitHub Projects, evidenciando la trazabilidad completa del proceso de validación automatizada.

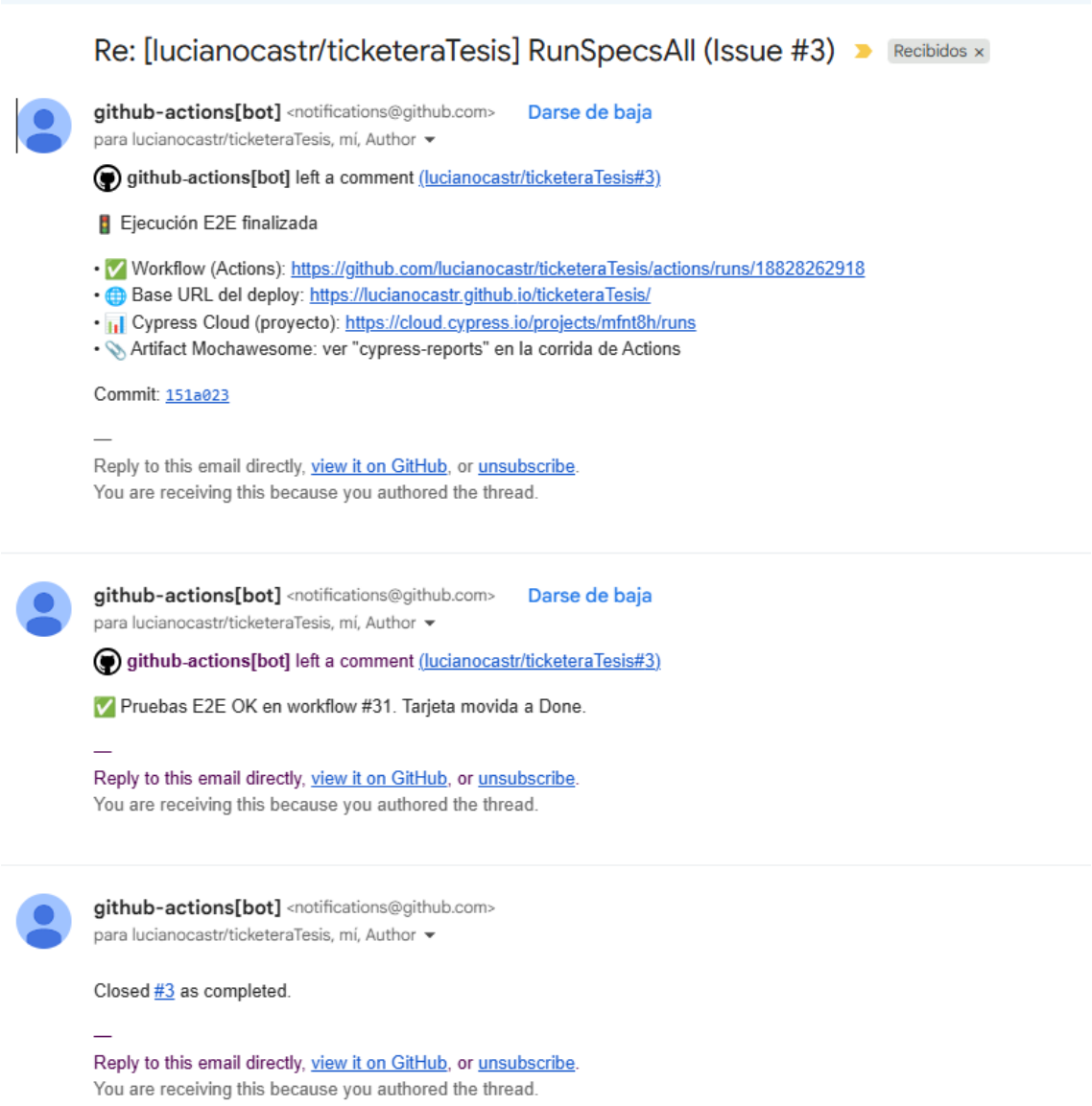

Figura 13 – Aviso vía mail con resultado de ejecución de pruebas

5.7. Análisis de resultados

A partir de los resultados comparativos, se evidencia una relación directa entre la problemática identificada en el Capítulo 1 y la efectividad de la estrategia implementada. La ausencia de pruebas automatizadas en escenarios manuales generaba lentitud en la validación, escasa cobertura de casos y falta de trazabilidad en los resultados, lo que derivaba en defectos no detectados y costos elevados de corrección.

La estrategia de automatización, validada mediante el prototipo desarrollado, demostró su capacidad para reducir tiempos de ejecución, ampliar la cobertura funcional, garantizar reproducibilidad y proveer trazabilidad completa entre el código, las pruebas y la gestión de incidencias. Estos logros responden directamente a los desafíos planteados en la problemática inicial:

- La lentitud se supera con ejecución automática en cloud, que valida en segundos lo que antes insumía minutos.
- La escasa cobertura se reemplaza por una ejecución sistemática de pruebas críticas y regresivas en cada commit.
- La falta de trazabilidad se resuelve mediante la integración con GitHub Projects, donde cada ejecución actualiza automáticamente el estado de los issues.

5.7.1. Métricas observadas

Indicador	Manual	Automatizado	Mejora (%)
Tiempo total por ciclo	15 min promedio	2 min promedio	≈ 87 %
Casos ejecutados por iteración	3 / 7 (43 %)	7 / 7 (100 %)	+ 57 p.p.
Defectos detectados pre-deploy	1 de 6 (20 %)	6 de 6 (100 %)	+ 80 p.p.
Esfuerzo humano por ciclo	15 min (2 roles)	3 min (revisión)	– 80 %
Corridas reproducibles	Baja (< 50 %)	Alta (100 %)	+ 50 p.p.

Tabla 2 – Métricas (Valores basados en ejecuciones promedio registradas en el pipeline y pruebas manuales controladas.)

5.7.2. Relación con atributos ISO/IEC 25010

Atributo de calidad	Evidencia observada en el prototipo
Adecuación funcional	Validación completa de flujos críticos (login, compra, checkout) con 100 % Pass.

Fiabilidad	Ejecución reproducible en entornos cloud sin variaciones ni falsos positivos.
Eficiencia de desempeño	Reducción ≈ 90 % en tiempos de validación y esfuerzo operativo.
Mantenibilidad	Casos versionados junto al código; actualización automática en pipeline.

En conjunto, las métricas evidencian mejoras sustanciales en la eficiencia y la fiabilidad del proceso de testing, aportando bases objetivas para la evaluación del impacto de la estrategia.

5.8. Síntesis

Los resultados experimentales confirman que la automatización del workflow de pruebas mediante Cypress + GitHub Actions + Projects genera beneficios medibles:

- Reducción de tiempos de validación en más del 85 %.
- Cobertura total de los casos críticos y regresivos.
- Reproducibilidad y trazabilidad completas, con evidencia automática en la nube.
- Disminución del esfuerzo humano y errores por intervención manual.

Estos hallazgos validan que la propuesta cumple con los objetivos planteados y fortalecen los atributos de calidad definidos por la norma ISO/IEC 25010, estableciendo una base objetiva para las conclusiones y recomendaciones del Capítulo 6.

6. Conclusiones y Recomendaciones

6.1. Conclusiones Generales

El presente trabajo de grado permitió abordar de manera integral una problemática actual y recurrente en la industria del software: la falta de estrategias sistemáticas de testing automatizado que garanticen la calidad en sistemas web de alta criticidad, como los entornos de venta online.

A partir del análisis realizado, se logró demostrar que la incorporación de pruebas funcionales y de regresión automatizadas, ejecutadas en entornos cloud y orquestadas mediante herramientas modernas como Cypress, GitHub Actions y GitHub Projects, aporta mejoras objetivas en los atributos de calidad definidos por la norma ISO/IEC 25010.

Los resultados obtenidos evidencian una reducción significativa en los tiempos de validación, un incremento de la cobertura de pruebas, una trazabilidad completa entre el código, la ejecución y la gestión de incidencias, y una disminución del esfuerzo humano en tareas repetitivas.

Estas mejoras fortalecen de manera directa los atributos de adecuación funcional, fiabilidad, eficiencia de desempeño, mantenibilidad y capacidad de interacción.

En síntesis, los hallazgos del proyecto confirman la hipótesis inicial: la implementación de una estrategia de automatización de pruebas funcionales y de regresión, integrada dentro de un flujo CI/CD y soportada en entornos cloud, mejora la calidad del software de forma medible, reproducible y sostenible, validando así su aplicabilidad en contextos reales de ingeniería de software.

6.2. Contribuciones del Trabajo

6.2.1. A nivel académico

El trabajo aporta evidencia concreta sobre cómo los principios de calidad definidos en ISO/IEC 25010 pueden operacionalizarse en un entorno real mediante automatización. Se demuestra de forma práctica cómo integrar teoría y práctica, convirtiendo conceptos abstractos —fiabilidad, adecuación funcional, mantenibilidad— en métricas, reportes y flujos reproducibles.

Además, ofrece un caso de estudio accesible para futuras investigaciones o cátedras que deseen introducir CI/CD y testing E2E dentro del enfoque académico.

Aporte propio:

El trabajo construye un puente claro y documentado entre la norma ISO y su implementación, mostrando un ejemplo completo, didáctico y replicable, inexistente previamente en este contexto académico.

6.2.2. A nivel técnico

Se desarrolló un pipeline CI/CD funcional que integra construcción, despliegue, validación automatizada, generación de reportes y gestión de incidencias. Este pipeline permite ejecutar pruebas de forma determinística, estandarizada y auditable, reduciendo la variabilidad del proceso manual.

Aportes propios concretos:

- Diseño de un flujo CI/CD completo orientado a la calidad, no solo al despliegue.
- Implementación de pruebas E2E con Cypress vinculadas a GitHub Actions y Cypress Cloud.
- Automatización de la apertura y actualización de issues según resultado.
- Generación consistente de evidencia que permite comparar manual vs. automatizado.

El resultado demuestra que herramientas abiertas y gratuitas pueden soportar prácticas del nivel de una ingeniería profesional.

6.2.3. A nivel profesional

El proyecto evidencia que incorporar práctica DevOps + QA reduce tiempos, riesgos y costos asociados a la validación.

La metodología resultante es aplicable a múltiples industrias que operan con sistemas transaccionales (e-commerce, fintech, SaaS, retail, etc.).

Aporte propio:

Se presenta un **modelo replicable**, documentado y simple de adoptar, que puede ser implementado por equipos con diferentes niveles de madurez técnica. El enfoque entrega una base para profesionalizar prácticas de QA en organizaciones pequeñas o medianas.

6.3. Recomendaciones

1. Incorporar la estrategia de automatización propuesta como parte de los procesos formales de desarrollo en organizaciones que requieran validación continua y despliegues frecuentes.

2. Mantener actualizada la suite de pruebas y el pipeline de CI/CD, integrándolos en el ciclo de versionado del producto.
3. Utilizar las métricas obtenidas (tiempo, cobertura, defectos detectados, trazabilidad) como indicadores de control de calidad en auditorías internas o revisiones técnicas.
4. Fomentar la formación del personal en herramientas de automatización y metodologías DevOps, asegurando la correcta adopción y mantenimiento del modelo.
5. Documentar y versionar todas las configuraciones de entorno, garantizando la reproducibilidad y la continuidad operativa ante cambios de equipo o infraestructura.

6.4. Líneas Futuras de Mejora

El trabajo aquí presentado abre nuevas oportunidades de investigación y desarrollo en materia de calidad de software:

- Pruebas no funcionales automatizadas: incorporar validaciones de rendimiento, carga y seguridad en la misma infraestructura cloud.
- Integración con inteligencia artificial: explorar modelos de aprendizaje automático que permitan generar casos de prueba dinámicos o predecir defectos a partir de patrones históricos.
- Monitoreo continuo de calidad: desarrollar paneles de indicadores (dashboards) con métricas en tiempo real sobre fiabilidad, desempeño y defectos detectados.
- Cobertura estática y calidad de código: complementar esta estrategia con herramientas de análisis estático como SonarQube o CodeQL, para integrar métricas de mantenibilidad y complejidad.
- Escalabilidad multi-entorno: extender el pipeline actual a validaciones distribuidas en entornos híbridos (cloud + on-premise), asegurando consistencia en diferentes escenarios de despliegue.

6.5. Cierre Final

En conclusión, este trabajo constituye un aporte tangible al aseguramiento de calidad en la ingeniería de software, demostrando que la integración de procesos automatizados, herramientas cloud y criterios de calidad normalizados puede transformar de manera significativa la validación y entrega de productos digitales.

Los resultados obtenidos evidencian que la automatización E2E y el uso de pipelines CI/CD no solo reducen tiempos y riesgos, sino que permiten estandarizar el proceso, obtener métricas objetivas y asegurar una trazabilidad completa en cada ciclo de ejecución.

Más allá de la solución técnica, el proyecto plantea un cambio cultural: entender la calidad como un proceso continuo, medible y automatizado, incorporado desde el desarrollo hasta la entrega, y no como una etapa posterior sujeta a variabilidad humana.

Finalmente, para garantizar la reproducibilidad de los resultados y la portabilidad del modelo, se desarrolló un paquete de replicación (ver Anexo 5) que reúne los archivos de configuración, scripts y ejemplos necesarios para ejecutar la suite de pruebas de punta a punta tanto en entornos locales como en pipelines cloud (GitHub Actions). Este paquete consolida la principal contribución práctica del trabajo al ofrecer un modelo completo y replicable para futuras implementaciones.

7. Bibliografía

- Capgemini, Sogeti & Micro Focus. (2023). *World Quality Report 2023-2024.* Disponible en https://ebook.capgemini.com/world-quality-report/

- PractiTest. (2023). *State of Testing Report 2023.* Disponible en: https://www.practitest.com/state-of-testing/

- ISTQB. (2023). *Software Testing Survey 2023.* International Software Testing Qualifications Board. Disponible en: https://www.istqb.org/

- Cypress.io. (2024). *Cypress Documentation.* Disponible en: https://docs.cypress.io/

- GitHub. (2024). *GitHub Actions Documentation.* Disponible en: https://docs.github.com/es/actions

- Pressman, R. (2015). *Ingeniería del Software: un enfoque práctico* (7.ª ed.). McGraw-Hill.

- Sommerville, I. (2017). *Ingeniería del Software* (10.ª ed.). Pearson Educación.

- International Organization for Standardization. (2011). *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.* Geneva: ISO.

- International Organization for Standardization. (2023). *ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model and Quality in use model.* Geneva: ISO.

- Shopify Community. (2023). *Problemas con el carrito de compras.* Disponible en: https://community.shopify.com/t/problemas-con-el-carrito-de-compras/188463

- ADSLZone. (2025). *Problemas de acceso a cuentas en Openbank.* Disponible en: https://www.adslzone.net/noticias/fintech/problemas-acceso-cuenta-openbank-solucion/

- Ámbito Financiero. (2023). *Fallas en transferencias de Brubank.* Disponible en: https://www.ambito.com/

- Infobae. (2022-2023). *Reportes de fallas en CyberMonday y Hot Sale Argentina.* Disponible en: https://www.infobae.com/

ANEXO I – ISO/IEC 25010:2011

El modelo de calidad representa la piedra angular en torno a la cual se establece el sistema para la evaluación de la calidad del producto. En este modelo se determinan las características de calidad que se van a tener en cuenta a la hora de evaluar las propiedades de un producto software determinado.

La calidad del producto software se puede interpretar como el grado en que dicho producto satisface los requisitos de sus usuarios aportando de esta manera un valor. Son precisamente estos requisitos (funcionalidad, rendimiento, seguridad, mantenibilidad, etc.) los que se encuentran representados en el modelo de calidad, el cual categoriza la calidad del producto en características y subccaracterísticas.

El modelo de calidad del producto definido por la ISO/IEC 25010 se encuentra compuesto por las nueve características de calidad que se muestran en la siguiente figura:

CALIDAD DEL PRODUCTO SOFTWARE								
ADECUACIÓN FUNCIONAL	EFICIENCIA DE DESEMPEÑO	COMPATIBILIDAD	CAPACIDAD DE INTERACCIÓN	FIABILIDAD	SEGURIDAD	MANTENIBILIDAD	FLEXIBILIDAD	PROTECCIÓN
COMPLETITUD FUNCIONAL	COMPORTAMIENTO TEMPORAL	COEXISTENCIA	RECONOCIBILIDAD DE ADECUACIÓN	AUSENCIA DE FALLOS	CONFIDENCIALIDAD	MODULARIDAD	ADAPTABILIDAD	RESTRICCIÓN OPERATIVA
CORRECCIÓN FUNCIONAL	UTILIZACIÓN DE RECURSOS	INTEROPERABILIDAD	APRENDIZABILIDAD	DISPONIBILIDAD	INTEGRIDAD	REUSABILIDAD	ESCALABILIDAD	IDENTIFICACIÓN DE RIESGOS
PERTINENCIA FUNCIONAL	CAPACIDAD		OPERABILIDAD	TOLERANCIA A FALLOS	NO-REPUDIO	ANALIZABILIDAD	INSTALABILIDAD	PROTECCIÓN ANTE FALLOS
			PROTECCIÓN FRENTE A ERRORES DE USUARIO	RECUPERABILIDAD	RESPONSABILIDAD	CAPACIDAD DE SER MODIFICADO	REEMPLAZABILIDAD	ADVERTENCIA DE PELIGRO
			INVOLUCRACIÓN DEL USUARIO		AUTENTICIDAD	CAPACIDAD DE SER PROBADO		INTEGRACIÓN SEGURA
			INCLUSIVIDAD		RESISTENCIA			
			ASISTENCIA AL USUARIO					
			AUTO-DESCRIPTIVIDAD					
iso25000.com								

Adecuación Funcional

Representa la capacidad del producto software para proporcionar funciones que satisfacen las necesidades declaradas e implícitas de los usuarios cuando el producto se usa en las condiciones especificadas. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

Completitud funcional. Grado en el que el conjunto de funcionalidades del producto cubre todas las tareas y los objetivos de usuario especificados.

Corrección funcional. Capacidad del producto o sistema para proveer resultados exactos cuando es usado por los usuarios especificados.

Pertinencia funcional. Capacidad del producto software para proporcionar un conjunto de funciones que facilitan la consecución de tareas y objetivos de usuario especificados.

Eficiencia de Desempeño

Esta característica representa el desempeño de un producto en la realización de sus funciones dentro de unos parámetros de tiempo y rendimiento especificados y con un uso eficiente de recursos (CPU, memoria, almacenamiento, energía...) utilizados bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

Comportamiento temporal. Grado en que un producto realiza sus funciones de forma que el tiempo de respuesta y el ratio de rendimiento cumple los requisitos especificados.

Utilización de recursos. Grado en que la cantidad y tipos de recursos utilizados por el producto al llevar a cabo su función bajo condiciones determinadas no exceden lo especificado.

Capacidad. Grado en que el producto cumple los requisitos relativos a límites máximos para un parámetro (ítems almacenados, usuarios concurrentes, ancho de banda de comunicaciones...).

Compatibilidad

Capacidad de un producto de intercambiar información con otros productos y/o llevar a cabo sus funciones requeridas cuando comparten un mismo entorno y recursos. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

Coexistencia. Capacidad del producto para coexistir con otro software independiente, en un entorno común, compartiendo recursos comunes sin detrimento.

Interoperabilidad. Capacidad de dos o más sistemas o componentes para intercambiar información y utilizar la información intercambiada.

Capacidad de Interacción

Capacidad del producto software para que el usuario interactúe mediante su interfaz intercambiando información para completar determinadas tareas. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

Reconocibilidad de la adecuación. Capacidad del producto que permite al usuario entender si el software es adecuado para sus necesidades.

Aprendizabilidad. Capacidad del producto que permite al usuario aprender su funcionamiento dentro de un tiempo especificado.

Operabilidad. Capacidad del producto que permite al usuario operarlo y controlarlo con facilidad.

Protección contra errores de usuario. Capacidad del sistema para prevenir errores en su operación.

Involucración del usuario. Capacidad del producto de presentar sus funciones e información de forma atractiva y motivadora, fomentando la interacción continua.

Inclusividad. Capacidad de un producto para ser utilizado por personas con distintos contextos (edad, habilidades, cultura, raza, lenguaje, género...).

Asistencia al usuario. Capacidad del producto que permite que sea utilizado por usuarios con determinadas características y capacidades logrando objetivos específicos.

Auto-descriptividad. Capacidad de un producto para presentar la información adecuada, haciendo su uso inmediatamente evidente para el usuario sin interacciones excesivas con el producto u otros recursos (documentación, *help desks*, otros usuarios...).

Fiabilidad

Capacidad de un sistema o componente para desempeñar las funciones especificadas, cuando se usa bajo unas condiciones y periodo de tiempo determinados sin interrupciones o fallos. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

- Ausencia de fallos. Capacidad del sistema de llevar a cabo sus funciones sin fallos bajo condiciones normales de operación.
- Disponibilidad. Capacidad del sistema o componente de estar operativo y accesible para su uso cuando se requiere.
- Tolerancia a fallos. Capacidad del sistema o componente para operar según lo previsto en presencia de fallos hardware o software.
- Capacidad de recuperación. Capacidad del producto software para recuperar los datos directamente afectados y reestablecer el estado deseado del sistema en caso de interrupción o fallo.

Seguridad

Capacidad de protección de la información y los datos de manera que las personas u otros productos tengan el grado de acceso a los datos adecuado a sus tipos y niveles de autorización, y para defenderse de los patrones de ataque de agentes malintencionados.. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

- Confidencialidad. Capacidad de asegurar que los datos solo son accesibles a aquellos con autorización para ello.
- Integridad. Capacidad de un producto para garantizar que el estado de su sistema y sus datos están protegidos frente a modificaciones o eliminaciones no autorizadas, ya sea por acciones malintencionadas o por errores informáticos.
- No repudio. Capacidad de demostrar las acciones o eventos que han tenido lugar, de manera que dichas acciones o eventos no puedan ser repudiados posteriormente.
- Responsabilidad. Capacidad de rastrear de forma inequívoca las acciones de una entidad.
- Autenticidad. Capacidad de un producto para demostrar que la identidad de un sujeto o recurso es la que se afirma.
- Resistencia. Capacidad de mantener la operación de un producto bajo condiciones de ataque de un actor malicioso.

Mantenibilidad

Esta característica representa la capacidad del producto software para ser modificado efectiva y eficientemente, debido a necesidades evolutivas, correctivas o perfectivas. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

- Modularidad. Capacidad de un producto para evitar que los cambios en un componente afecten a otros componentes.
- Reusabilidad. Capacidad de un activo que permite que sea utilizado en más de un sistema software o en la construcción de otros activos.
- Analizabilidad. Facilidad con la que se puede evaluar el impacto de un determinado cambio sobre el resto del software, diagnosticar las deficiencias o causas de fallos en el software, o identificar las partes a modificar.
- Capacidad para ser modificado. Capacidad del producto que permite que sea modificado de forma efectiva y eficiente sin introducir defectos o degradar su calidad.
- Capacidad para ser probado. Facilidad con la que se pueden establecer criterios de prueba para un sistema o componente y con la que se pueden llevar a cabo las pruebas para determinar si se cumplen dichos criterios.

Flexibilidad

Capacidad del producto para adaptarse a cambios en sus requisitos, contextos de uso o entorno del sistema. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- Adaptabilidad. Capacidad del producto que le permite ser adaptado de forma efectiva y eficiente a diferentes entornos determinados de hardware, software, operacionales o de uso.
- Escalabilidad. Capacidad del producto para gestionar cargas de trabajo crecientes o decrecientes y para adaptar su capacidad a la variabilidad.
- Instalabilidad. Facilidad con la que el producto se puede instalar y/o desinstalar de forma exitosa en un determinado entorno.
- Reemplazabilidad. Capacidad del producto para ser utilizado en lugar de otro producto software determinado con el mismo propósito y en el mismo entorno.

Protección

Esta característica representa la capacidad del producto, en condiciones definidas, de evitar un estado en el que se ponga en peligro la vida humana, la salud, la propiedad o el medio ambiente. Esta característica se subdivide a su vez en las siguientes subccaracterísticas:

- Restricción operativa. Capacidad de un producto para limitar su funcionamiento a unos parámetros o estados seguros cuando se enfrenta a un peligro operativo.
- Identificación de riesgos. Capacidad de un producto para identificar situaciones u operaciones que pueden exponer la vida, la propiedad o el medio ambiente a un riesgo inaceptable.
- Protección ante fallos. Capacidad de un producto para ponerse automáticamente en un modo de funcionamiento seguro o para volver a una condición segura en caso de fallo.
- Advertencia de peligro. Capacidad de un producto para alertar de riesgos inaceptables, de modo que puedan reaccionar con tiempo suficiente para mantener la seguridad de las operaciones.
- Integración segura. Capacidad de un producto para mantener la seguridad durante y después de la integración con uno o varios componentes.

ANEXO II - Plan y Guía de Pruebas – Plataforma de venta de entradas (Manual y Referencia a E2E)

Objetivo académico: demostrar implementación y **validación de resultados mediante pruebas manuales**, con correspondencia (trazabilidad) hacia los casos E2E automatizados existentes para comparación de métricas.

Alcance

- Módulos: Login, Catálogo, Carrito, Checkout, Confirmación.
- Tipos de pruebas: funcionales manuales, validaciones de formularios, guards de navegación, y escenario de pago rechazado - reintento aprobado (simulado).

Entorno

- App ejecutándose localmente (dev o preview).
- Usuario(s) de prueba definidos en usuarios.json (p.ej. qa@example.com / ******).
- Carrito limpio al iniciar cada caso (borrar localStorage o usar navegador en limpio).

Evidencia requerida en cada caso: capturas de pantalla por paso clave y resultado **Pass/Fail**.

Matriz de Trazabilidad (Manual - E2E)

ID Manual	Caso Manual	Spec E2E relacionada
TC-001	Login correcto / inválido (mensajes)	auth.cy.js
TC-002	Happy path: Login → Catálogo → Carrito → Checkout → Confirmación	checkout.happy.cy.js
TC-003	Carrito: agregar, ver totales, vaciar	cart.cy.js
TC-004	Catalogo: Verifica disponible luego de login	cart.cy.js
TC-005	Sesión Expirada: Expiración de sesión y recuperación del flujo de compra	checkout.validations.cy.js
TC-006	Pago rechazado → reintento aprobado	payment.decline-retry.cy.js

Ejecución de Casos de Prueba (Manual)

1. TC-001 — Login correcto / inválido

Precondición: Sin sesión activa.
Datos:

- Usuario válido: user@test.com / 123456
- Usuario inválido: invalido@example.com / 111111

Pasos:

1. Abrir la vista de Login.
2. Ingresar usuario válido y pulsar *Ingresar*.
3. Verificar redirección automática al Catálogo.
4. Cerrar sesión.
5. Repetir con usuario inválido.
6. Verificar mensaje de error “Credenciales incorrectas”.

Resultado Esperado:

- En el caso válido: acceso al catálogo.
- En el caso inválido: mensaje de error visible sin redirección.

Evidencia: Capturas de pantalla del mensaje de error y del Catálogo.
Ejecuciones: 3 veces.

Control de credenciales: OK

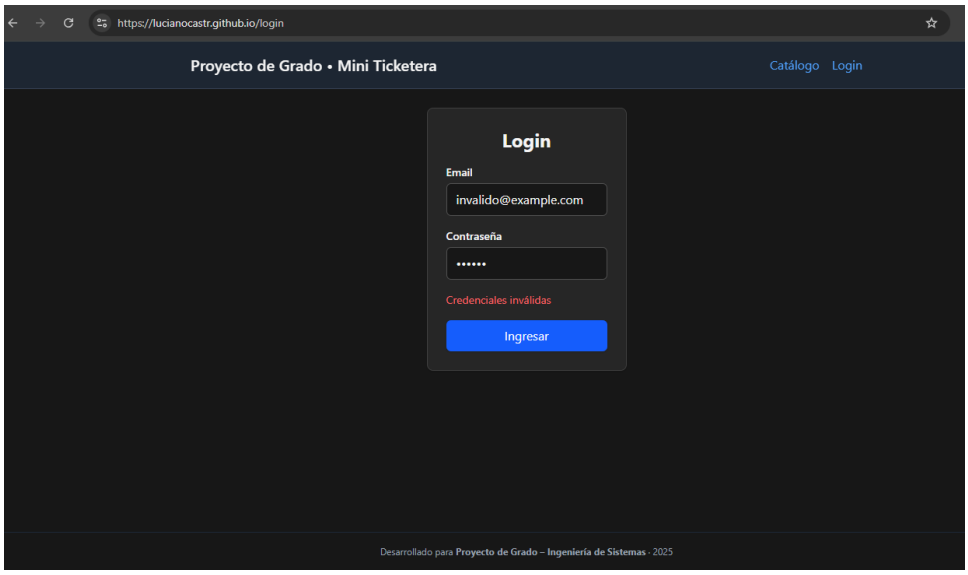

Figura 1 - Anexo II – Evidencia **TC-001**

Login: OK

Figura 2 - Anexo II – Evidencia 2 **TC-001**

2. TC-002 — Happy Path de compra

Precondición: Sin sesión; carrito vacío.
Datos:
Usuario válido; tarjeta simulada válida (Luhn).
Ejemplo: 4242 4242 4242 4242, venc. 12/30, CVC 123.

Pasos:

1. Abrir Login. Ingresar email y contraseña válidos. Pulsar *Ingresar*.
2. En Catálogo, verificar título. Agregar un producto cualquiera (guardar nombre del ticket).
3. Abrir Carrito desde el menú. Verificar que el nombre esté listado y total > 0.
4. Pulsar *Ir a checkout*.
5. En Checkout, verificar que el resumen contenga el nombre del ticket. Completar Nombre, Email, DNI, Número, Vencimiento, CVC.
6. Pulsar *Confirmar compra*.
7. En Confirmación, verificar mensajes de éxito y que Ítems contenga el nombre del ticket.

Resultado Esperado: Navegación fluida entre pantallas y confirmación visible con detalle de orden.
Evidencia: Captura de carrito y confirmación de compra.
Ejecuciones: 5 veces.

1. Login: OK
2. Selección de ticket: Entrada General. OK
3. Verificación de carrito (Captura): OK

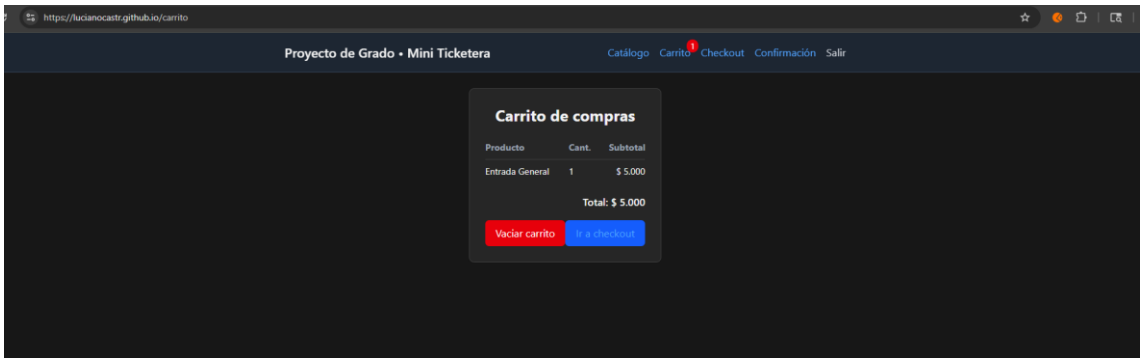

Figura 3 - Anexo II – Evidencia 2 **TC-002**

4. Verificar Checkout y validar funcionamiento. OK
5. Generación de confirmación (Captura). OK

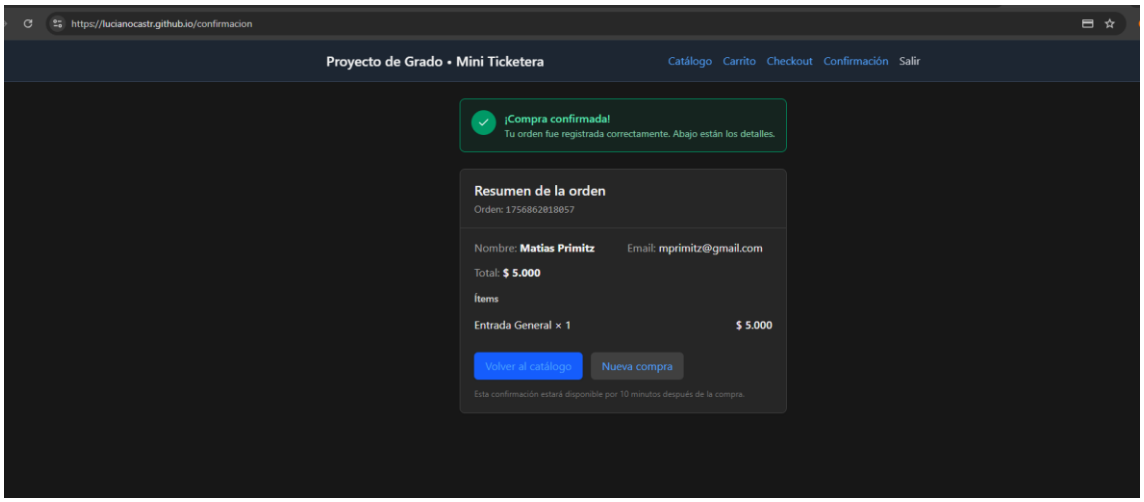

Figura 4 - Anexo II – Evidencia 2 **TC-002**

Tiempo de ejecución promedio 3 minutos.

Tiempo de organización y registro de evidencia promedio 4 minutos 25 segundos.

Cantidad de ejecuciones: 5

Tiempo total: 37 minutos aproximadamente

---

3. TC-003 — Pago rechazado - reintento aprobado

Precondición: Usuario logueado; producto agregado al carrito.
Datos:

- Tarjeta simulada rechazada: 4000 0000 0000 0002
- Tarjeta simulada válida: 4242 4242 4242 4242

Pasos:

1. Acceder al Checkout con carrito listo.
2. Completar datos válidos de cliente.
3. Ingresar la tarjeta rechazada y pulsar *Pagar*.
4. Verificar mensaje de “Pago rechazado”.
5. Reintentar con la tarjeta válida.
6. Verificar redirección a la Confirmación y mensaje de éxito.

Resultado Esperado:
Primer intento rechazado (mensaje visible); segundo intento aprobado con confirmación de compra.

Evidencia: Captura de error y pantalla de confirmación.
Ejecuciones: 5 veces.

1. TC001. OK
2. Número inválido (Captura). OK

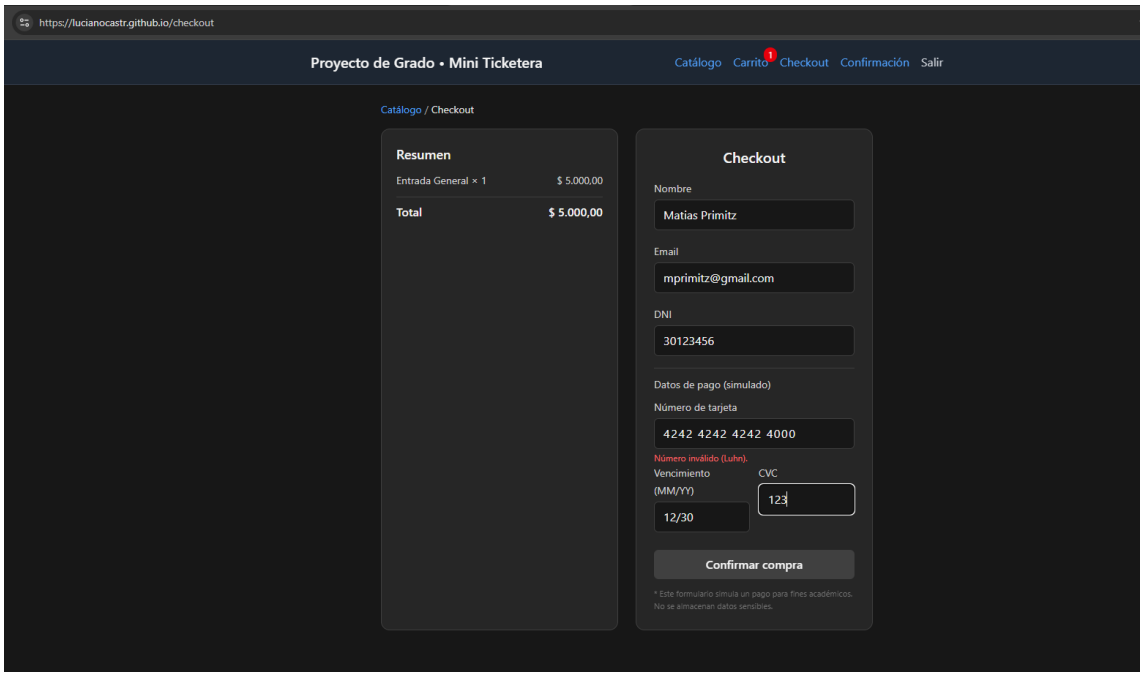

Figura 5 - Anexo II – Evidencia 2 **TC-003**

3. Número válido (Captura). OK

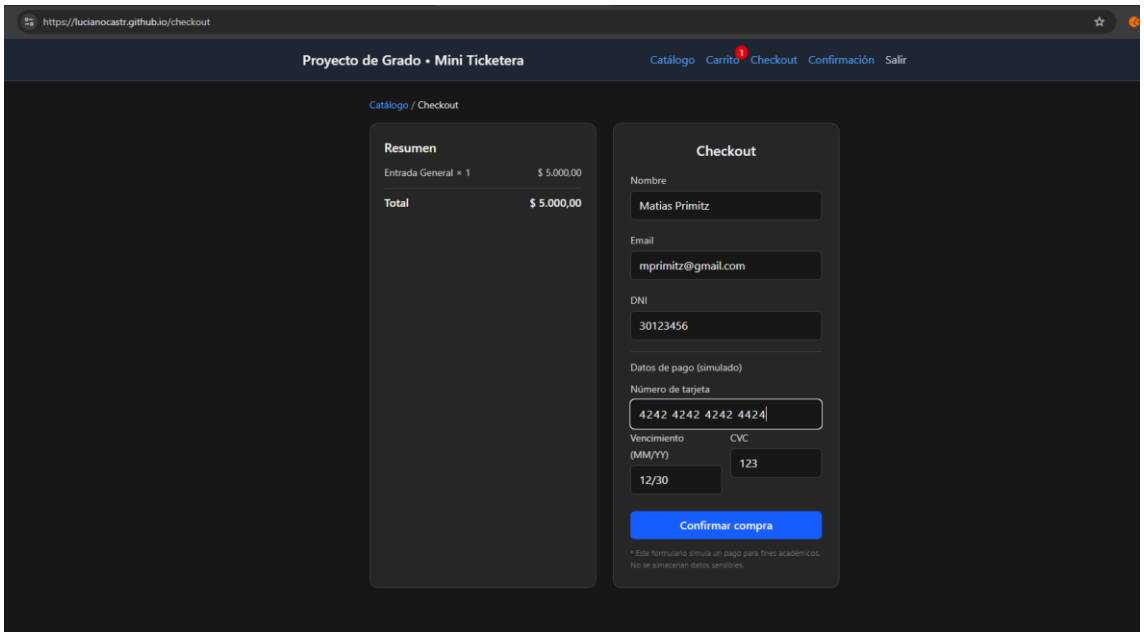

Figura 6 - Anexo II – Evidencia 2 **TC-003**

4. Confirmación (Captura). OK

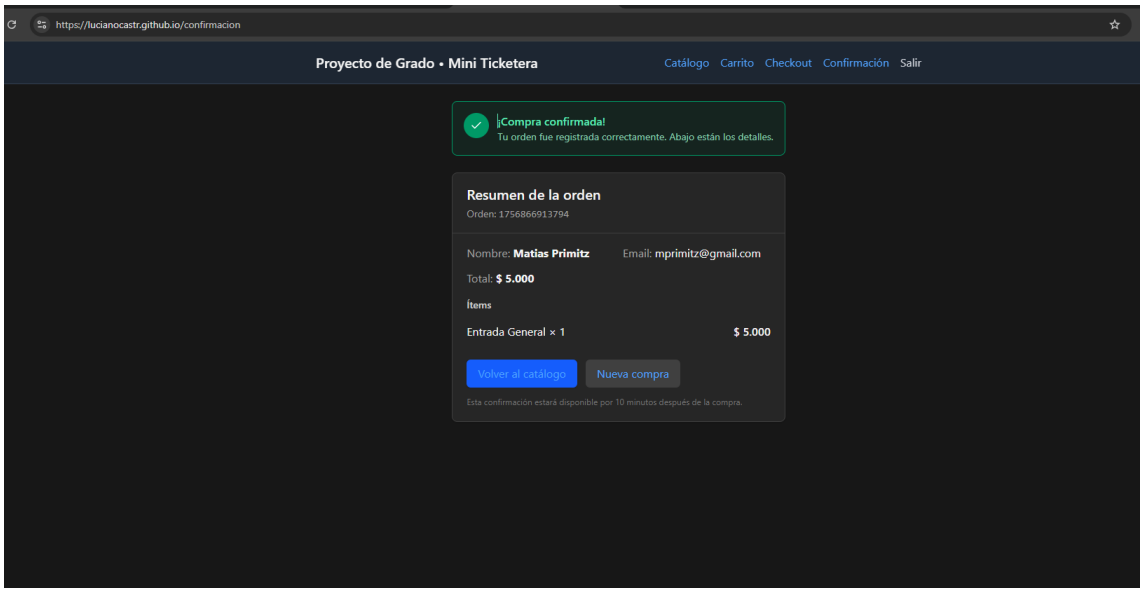

Figura 7 - Anexo II – Evidencia 3 **TC-003**

Tiempo de ejecución promedio 3 minutos 15 segundos.

Tiempo de organización y registro de evidencia promedio 4 minutos 30 segundos.

Cantidad de ejecuciones: 5

Tiempo total: 38 minutos aproximadamente

---

4. TC-004 — Carrito: agregar, ver totales y vaciar

Precondición: Usuario autenticado.
Datos: Producto del catálogo con precio > 0.

Pasos:

1. Abrir Catálogo y agregar un producto al carrito.
2. Abrir Carrito y verificar que el producto esté listado.
3. Validar que el subtotal y total se actualicen correctamente.
4. Pulsar *Vaciar carrito*.
5. Confirmar que el carrito quede vacío.

Resultado Esperado: Ítem agregado correctamente, totales calculados y vaciado exitoso.

Evidencia: Captura de carrito antes y después del vaciado.
Ejecuciones: 4 veces.

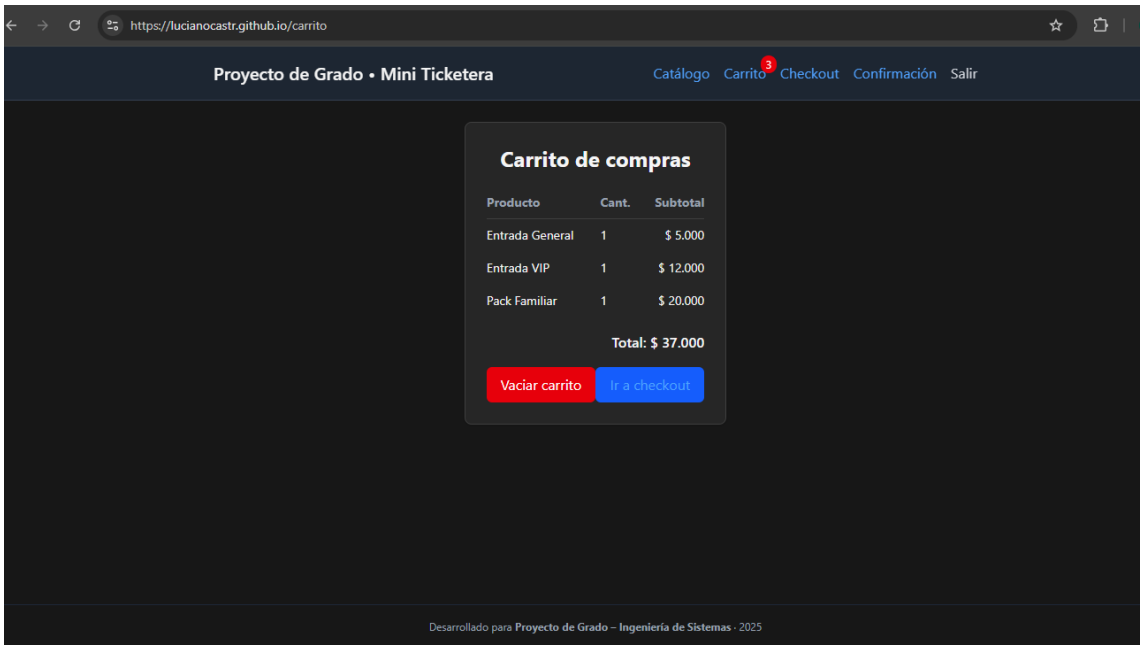

Figura 8 - Anexo II – Evidencia 1 **TC-004**

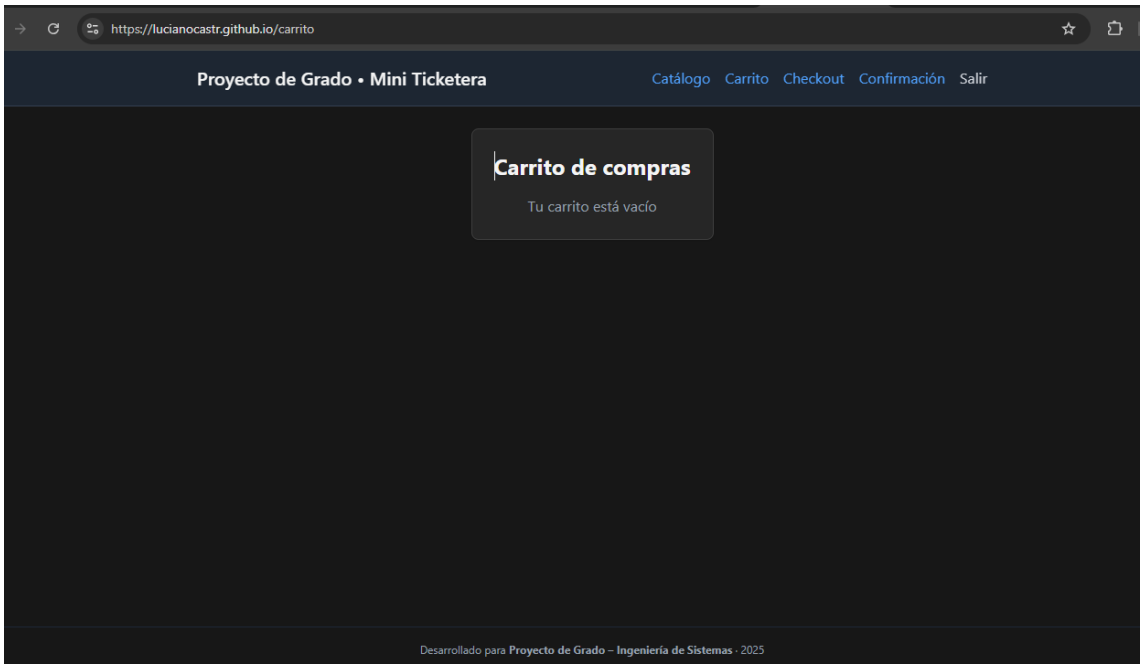

Figura 9 - Anexo II – Evidencia 2 **TC-004**

Tiempo de ejecución promedio 2 minutos 10 segundos.

Tiempo de organización y registro de evidencia promedio 4 minutos 30 segundos.

Cantidad de ejecuciones: 5

Tiempo total: 25 minutos aproximadamente

5. TC-005 — Catálogo disponible luego del login

Precondición: Usuario sin sesión activa.
Datos: Usuario válido (user@test.com / 123456).

Pasos:

1. Acceder a la vista de Login.
2. Ingresar credenciales válidas.
3. Verificar redirección al Catálogo.
4. Confirmar que se muestren tarjetas de producto con título, precio y botón *Agregar al carrito*.

Resultado Esperado: Catálogo visible con productos cargados tras login.
Evidencia: Captura de la vista de Catálogo autenticado.
Ejecuciones: 3 veces.

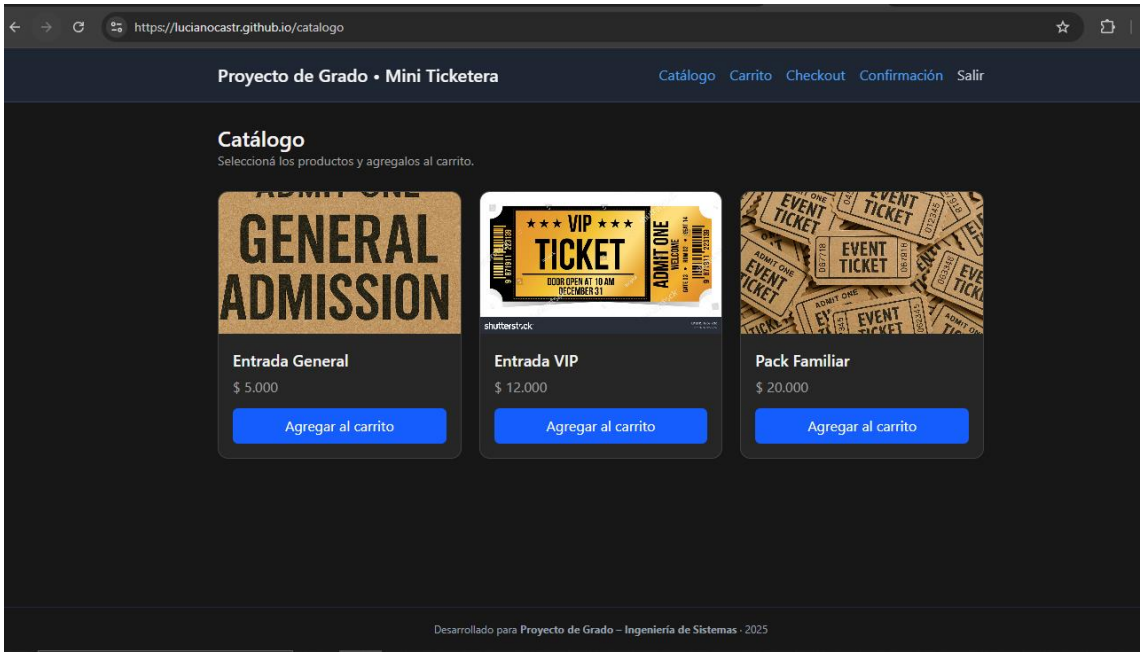

Figura 10 - Anexo II – Evidencia 1 **TC-005**

Tiempo de ejecución promedio 1 minutos 20 segundos.

Tiempo de organización y registro de evidencia promedio 3 minutos 30 segundos.

Cantidad de ejecuciones: 5

Tiempo total: 11 minutos aproximadamente

6. TC-006 — Sesión expirada y recuperación del flujo

Precondición: Usuario logueado y con un producto en carrito.
Datos: Token de sesión con tiempo limitado (simulado o manualmente expirado).

Pasos:

1. Loguearse y agregar un producto al carrito.
2. Avanzar hasta Checkout.
3. Simular expiración de sesión (borrar localStorage.token o expirar cookie).
4. Intentar continuar con el pago.
5. Verificar redirección automática al Login.
6. Reautenticarse.
7. Verificar que el carrito anterior se haya restaurado y continuar al Checkout.

Resultado Esperado: Sistema detecta sesión expirada, redirige al login y conserva el carrito tras nueva autenticación.
Ejecuciones: 3 veces.

Tiempo de ejecución promedio 4 minutos.

Tiempo de organización y registro de evidencia promedio 3 minutos.

Cantidad de ejecuciones: 5

Tiempo total: 23 minutos aproximadamente

ANEXO III - Workflow de Integración Continua en GitHub Actions

A continuación, se presenta el archivo .yml utilizado en el proyecto, con comentarios explicativos en cada sección para ilustrar cómo se integran las tecnologías involucradas en la solución.

```yaml
# Nombre descriptivo del workflow
name: Deploy to GitHub Pages

# Evento disparador: se ejecuta en cada push sobre la rama principal
on:
  push:
    branches: [ main ]

# Control de concurrencia: evita que se acumulen múltiples pipelines
  concurrency:
  group: pages-${{ github.ref }}
  cancel-in-progress: true

# Permisos mínimos para las operaciones que se realizarán
permissions:
  contents: read       # Permite leer contenido del repositorio
  issues: write        # Necesario para actualizar/etiquetar issues
  pages: write         # Permite desplegar en GitHub Pages
  id-token: write      # Autenticación segura en la ejecución
jobs:
  # Primer bloque: compilación de la aplicación
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4    # Descarga el código fuente

      - uses: actions/setup-node@v4  # Configura Node.js
        with:
          node-version: 20.19        # Versión exacta para
                                     reproducibilidad
          cache: 'npm'

      - run: npm ci                  # Instala dependencias limpias
      - run: npm run build           # Genera la build de producción
      - run: cp dist/index.html dist/404.html # Requisito SPA en Pages

      - uses: actions/upload-pages-artifact@v3 # Empaqueta el resultado
        with:
          path: dist

  # Segundo bloque: despliegue en GitHub Pages
```

```yaml
  deploy:
    needs: build                      # Solo corre si build fue exitoso
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    outputs:
      page_url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4 # Publica la app en GitHub Pages

  # Tercer bloque: ejecución de pruebas E2E con Cypress
  e2e:
    needs: deploy                      # Corre después del deploy
    runs-on: ubuntu-latest
    env:
      BASE_URL: ${{ needs.deploy.outputs.page_url }} # URL pública
                                                       desplegada
      CI: true
      CYPRESS_RECORD_KEY: ${{ secrets.CYPRESS_RECORD_KEY }}# Key secreta
                                                             de Cypress
                                                             Cloud
      ISSUE_NUMBER: 2   # Número del issue vinculado
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20.19
          cache: 'npm'

      - run: npm ci

      # Cachea binarios de Cypress para acelerar ejecuciones
      posteriores
      - name: Cache Cypress
        uses: actions/cache@v4
        with:
          path: ~/.cache/Cypress
          key: cypress-${{ runner.os }}-${{ hashFiles('package-
lock.json') }}
          restore-keys: |
            cypress-${{ runner.os }}-

      # Ejecución de pruebas Cypress grabando en Cypress Cloud
      - name: Run Cypress (record)
        run: npm run cy:run:all -- --record --key "$CYPRESS_RECORD_KEY" -
-config baseUrl=${BASE_URL}
```

```
      # Generación de reportes Mochawesome
      - run: mkdir -p reports/html
      - run: npx mochawesome-merge reports/mochawesome/*.json -o
reports/merged.json
      - run: npx mochawesome-report-generator reports/merged.json -o
reports/html --inline -f index

      # Subida de artefactos SIEMPRE, pase o falle la prueba
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: cypress-reports
          path: |
            reports/html
            reports/junit
            cypress/screenshots
            cypress/videos
          retention-days: 14

      # Asegurar etiquetas (labels) en issues, idempotente
      - name: Ensure labels exist
        if: always()
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          gh label create "e2e-pass" --color 0E8A16 --description "E2E
pipeline passed" || true
          gh label create "e2e-fail" --color D93F0B --description "E2E
pipeline failed" || true

      # Etiquetar issue según resultado
      - name: Label issue as PASS
        if: success()
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: gh issue edit $ISSUE_NUMBER --add-label "e2e-pass"

      - name: Label issue as FAIL
        if: failure()
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: gh issue edit $ISSUE_NUMBER --add-label "e2e-fail"

      # Comentar siempre los resultados en el issue
      - name: Comment results on issue
        if: always()
```

```
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          RUN_URL="${{ github.server_url }}/${{ github.repository
}}/actions/runs/${{ github.run_id }}"
          cat << 'EOF' > body.txt
          Ejecución E2E finalizada

          • Workflow (Actions): RUN_URL
          • Base URL del deploy: BASE_URL
          • Cypress Cloud (proyecto):
https://cloud.cypress.io/projects/mfnt8h/runs
          • Artifact Mochawesome: ver "cypress-reports" en la corrida de
Actions

          Commit: ${{ github.sha }}
          EOF
          # Reemplazos simples
          sed -i "s|RUN_URL|$RUN_URL|g" body.txt
          sed -i "s|BASE_URL|$BASE_URL|g" body.txt
          gh issue comment $ISSUE_NUMBER --body-file body.txt

       # Cerrar issue si todas las pruebas pasan (moviendo la tarjeta a
       Done en Projects)
      - name: Close issue when tests pass
        if: success()
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          gh issue close $ISSUE_NUMBER --comment "Pruebas E2E OK en
workflow #${{ github.run_number }}. Tarjeta movida a Done."
```

ANEXO IV – CASOS DE PRUEBA - Cypress

1. Login - auth cy.js

Verificación del proceso de autenticación (login correcto e incorrecto)

Objetivo:

Validar el comportamiento del sistema ante intentos de autenticación con credenciales válidas e inválidas, asegurando la correcta protección de acceso y mensajes de error apropiados.

```
// cypress/e2e/auth.cy.js
describe('Auth: login OK e inválido', () => {
  it('login correcto redirige a /catalogo', () => {
    cy.readFile('src/data/usuarios.json').then(({ 0: u }) => {
      cy.visit('/login')
            cy.get('.bg-blue-600').click()

      cy.get('[data-testid="input-email"]').type(u.email)
      cy.get('[data-testid="input-password"]').type(u.password)
      cy.get('[data-testid="btn-login"]').click()
      cy.url().should('include', '/catalogo')
    })
  })

  it('login inválido muestra error y no redirige', () => {
    cy.visit('/login')
          cy.get('.bg-blue-600').click()

    cy.get('[data-testid="input-email"]').type('no@existe.com')
    cy.get('[data-testid="input-password"]').type('xxx')
    cy.get('[data-testid="btn-login"]').click()
    cy.get('[data-testid="login-error"]').should('contain', 'Credenciales
inválidas')
    cy.url().should('include', '/login')
  })
})
```

2. Checkout Happy - checkout.happy.cy.js

Flujo completo de compra: login - catálogo - carrito - checkout - confirmación

Objetivo:

Validar el correcto funcionamiento del flujo completo de compra, asegurando que un usuario autenticado pueda seleccionar un producto, agregarlo al carrito, completar el proceso de checkout con datos válidos y recibir una confirmación de compra exitosa.

```
// cypress/e2e/checkout.flow.cy.js
describe('Flujo completo: login → catálogo → carrito → checkout →
confirmación', () => {
  it('valida ítems y completa compra aprobada', () => {
    // === LOGIN ===
    cy.readFile('src/data/usuarios.json').then((users) => {
      const { email, password } = users[0];

      cy.visit('/login');
          cy.get('.text-sm > [href="/login"]').click()
      cy.get('body').then($body => {
      // ¿ya está el formulario?
      if ($body.find('[data-testid="input-email"]').length === 0) {
        // plan B: click al link de Login en el header (ancla o Link)
        // - intenta selector por href que termina en 'login'
        cy.get('a[href$="login"]').first().click({ force: true });
      }
    });

      cy.get('[data-testid="input-email"]').type(email);
      cy.get('[data-testid="input-password"]').type(password);
      cy.get('[data-testid="btn-login"]').click();

      // Redirección a Catálogo confirmada por URL o título
      cy.url().should('include', '/catalogo');
      cy.contains('h1', 'Catálogo').should('be.visible');

      // === CATÁLOGO: tomar nombre del primer ticket y agregarlo ===
      cy.get('[data-testid^="card-"]').first().within(() => {
        cy.get('h3')
          .should('be.visible')
          .invoke('text')
          .then((t) => cy.wrap(t.trim()).as('ticketName'));
        cy.get('button[data-testid^="btn-add-"]').click();
```

```
      });

      // toast de agregado
      cy.get('[data-testid="toast-added"]').should('be.visible');

      // === MENÚ: ir a CARRITO (sin visit) ===
      cy.get('a[href="/carrito"]').first().click();

      // === CARRITO: validar que está el producto ===
      cy.get('[data-testid="carrito-page"]').should('be.visible');
      cy.get('@ticketName').then((name) => {
        cy.get('[data-testid="carrito-lista"]').should('contain', name);
      });

      // Ir a checkout por el link "Ir a checkout"
      cy.contains('a', 'Ir a checkout').click();

      // === CHECKOUT: validar resumen y completar datos ===
      cy.get('[data-testid="checkout-form"]').should('be.visible');
      cy.get('@ticketName').then((name) => {
        cy.contains('aside', 'Resumen').should('contain', name);
      });

      // Comprador
      cy.get('[data-testid="inp-nombre"]').type('Luciano QA');
      cy.get('[data-testid="inp-email"]').type('luciano.qa@example.com');
      cy.get('[data-testid="inp-dni"]').type('30123456');

      // Tarjeta (aprobada para llegar a confirmación)
      cy.get('[data-testid="inp-card"]').type('4242 4242 4242 4242');
      cy.get('[data-testid="inp-exp"]').type('12/30');
      cy.get('[data-testid="inp-cvc"]').type('123');

      // Confirmar compra
      cy.get('[data-testid="btn-pay"]').click();

      // === CONFIRMACIÓN: textos y nombre del ticket ===
      cy.get('[data-testid="confirmacion-page"]').should('be.visible');
      cy.contains('¡Compra confirmada!').should('be.visible');
      cy.contains('Tu orden fue registrada correctamente. Abajo están los
detalles.')
        .should('be.visible');
      cy.get('@ticketName').then((name) => {
        cy.get('[data-testid="orden-items"]').should('contain', name);
      });
    });
  });
});
```

3. Carrito -cart.cy.js

Validación de visualización y vaciado del carrito de compras

Objetivo:
Verificar que los datos del producto agregado al carrito se visualicen correctamente y que la funcionalidad de vaciado del carrito opere según lo esperado.

```
describe('Valida que los datos de la compran se vean en el carrito', ()
=> {
it('Agrega ticket y luego vacía el carrito', () => {
   // === LOGIN ===
    cy.readFile('src/data/usuarios.json').then((users) => {
      const { email, password } = users[0];

      cy.visit('/login');
                cy.get('.bg-blue-600').click()

      cy.get('[data-testid="input-email"]').type(email);
      cy.get('[data-testid="input-password"]').type(password);
      cy.get('[data-testid="btn-login"]').click();

      // Redirección a Catálogo confirmada por URL o título
      cy.url().should('include', '/catalogo');
      cy.contains('h1', 'Catálogo').should('be.visible');

      // === CATÁLOGO: tomar nombre del primer ticket y agregarlo ===
      cy.get('[data-testid^="card-"]').first().within(() => {
        cy.get('h3')
          .should('be.visible')
          .invoke('text')
          .then((t) => cy.wrap(t.trim()).as('ticketName'));
        cy.get('button[data-testid^="btn-add-"]').click();
      });

      // toast de agregado
      cy.get('[data-testid="toast-added"]').should('be.visible');

      // === MENÚ: ir a CARRITO (sin visit) ===
      cy.get('a[href="/carrito"]').first().click();

      // === CARRITO: validar que está el producto ===
      cy.get('[data-testid="carrito-page"]').should('be.visible');
      cy.get('@ticketName').then((name) => {
        cy.get('[data-testid="carrito-lista"]').should('contain', name);
      });
```

```
    })
  cy.get('a[href="/carrito"]').click()
  cy.get('[data-testid="carrito-lista"]').should('contain', 'Entrada')
  cy.get('[data-testid="carrito-limpiar"]').click()
  cy.get('[data-testid="carrito-vacio"]').should('be.visible')
})
});
```

4. Catalogo -cart.cy.js

Título: Verificación de disponibilidad del catálogo luego del login

Objetivo:

Comprobar que, tras un inicio de sesión exitoso, el sistema redirige correctamente al catálogo y permite interactuar con sus elementos (por ejemplo, agregar un producto al carrito).

```
describe('Valida que luego del login encuentre el catalogo disponible',
() => {
it('Catalogo disponible luego de login', () => {
   // === LOGIN ===
    cy.readFile('src/data/usuarios.json').then((users) => {
      const { email, password } = users[0];

      cy.visit('/login');
                        cy.get('.bg-blue-600').click()
cy.wait(1000)
      cy.get('[data-testid="input-email"]').type(email);
      cy.get('[data-testid="input-password"]').type(password);
      cy.get('[data-testid="btn-login"]').click();

      // Redirección a Catálogo confirmada por URL o título
      cy.url().should('include', '/catalogo');
      cy.contains('h1', 'Catálogo').should('be.visible');
      });

  cy.get('button[data-testid^="btn-add-"]').first().click()
  cy.get('[data-testid="toast-
added"][role="status"]').should('be.visible')
})
});
```

5. Sesión expirada – sesión.expire.cy.js

Manejo de expiración de sesión y recuperación del flujo de compra

Objetivo

Verificar que, ante la expiración de la sesión del usuario durante el proceso de compra, el sistema:

1. Redirige automáticamente a la pantalla de login.
2. Permite reautenticarse correctamente.
3. Mantiene los ítems previamente agregados al carrito una vez restablecida la sesión.

```
// cypress/e2e/session.expire.cy.js
describe('Sesión expira y recuperación desde catálogo', () => {
  it('redirige a login al expirar y permite continuar desde el catálogo',
() => {
    cy.readFile('src/data/usuarios.json').then((users) => {
      const { email, password } = users[0];
      let savedSession;

      // === LOGIN INICIAL ===


      cy.visit('/login');
      cy.get('.bg-blue-600').click()
      cy.get('[data-testid="input-email"]').type(email);
      cy.get('[data-testid="input-password"]').type(password);
      cy.get('[data-testid="btn-login"]').click();

      // Confirmar acceso al catálogo
      cy.url().should('include', '/catalogo');
      cy.contains('h1', 'Catálogo').should('be.visible');

      // === AGREGAR PRODUCTO ===
      cy.get('[data-testid^="card-"]').should('have.length.at.least',
1).first().within(() => {
        cy.contains('Agregar al carrito').click();
      });
      cy.get('[data-testid="toast-added"]').should('be.visible');

      // Guardar sesión actual
      cy.window().then(win => {
        savedSession = win.localStorage.getItem('sessionUser');
      });
```

```
      // === IR A CHECKOUT ===
      cy.contains('a', 'Checkout').click();
      cy.url().should('include', '/checkout');
      cy.get('[data-testid^="checkout"]').should('exist');

      // === FORZAR EXPIRACIÓN DE SESIÓN ===
      cy.window().then((win) => {
        const u = JSON.parse(win.localStorage.getItem('sessionUser') ||
'{}');
        u.loginAt = 0;
        win.localStorage.setItem('sessionUser', JSON.stringify(u));
      });

      // === REINTENTAR ACCESO AL CHECKOUT ===
      cy.visit('/checkout', {
        onBeforeLoad(win) {
          if (savedSession) {
            win.localStorage.setItem('sessionUser', savedSession);
          }
        },
      });

      // Validar redirección automática a login
                  cy.get('.bg-blue-600').click()

      cy.url().should('include', '/login');
      cy.get('[data-testid="login-form"]').should('be.visible');

      // === REINICIAR SESIÓN ===
      cy.get('[data-testid="input-email"]').type(email);
      cy.get('[data-testid="input-password"]').type(password);
      cy.get('[data-testid="btn-login"]').click();

      // === VALIDAR REDIRECCIÓN AL CATÁLOGO ===
      cy.url().should('include', '/catalogo');
      cy.contains('h1', 'Catálogo').should('be.visible');

      // === CONTINUAR MANUALMENTE AL CHECKOUT ===
      cy.contains('a', 'Checkout').click();
      cy.url().should('include', '/checkout');
      cy.get('[data-testid^="checkout"]').should('be.visible');

      // === VALIDAR QUE EL CARRITO SE CONSERVA ===
cy.get('aside').should('contain', 'Entrada General');
cy.get('aside').invoke('text').should('match', /5\.000/);

    });
  });
});
```

6. Reintento de pago exitoso – sesión.expire.cy.js

Pago rechazado con reintento exitoso a través de la interfaz (UI)

Objetivo:

Verificar el correcto funcionamiento del flujo de pago ante un escenario de rechazo inicial y posterior aprobación en reintento, asegurando que:

1. El sistema reaccione adecuadamente ante una transacción fallida (*status: declined*), mostrando mensaje de error y manteniendo el estado del checkout sin redirección prematura.
2. Al reintentar el pago con los mismos datos corregidos (*status: approved*), se procese exitosamente la transacción y se redirija al usuario a la pantalla de confirmación.
3. Los datos del producto seleccionado, el resumen de compra y los elementos de confirmación (número de orden, detalles del ítem) se muestren correctamente tras la aprobación.
4. La simulación de pasarela de pago mediante stub (inyectada en window.__TEST_PAY__) reproduzca condiciones controladas para evaluar la resiliencia y la capacidad de reintento del flujo sin dependencia externa.

```
// cypress/e2e/payment.decline-retry.cy.js
describe('Pago: rechazado → reintento aprobado (flujo por UI)', () => {
  it('login → catálogo → carrito → checkout: decline y reintento ok', ()
=> {
    // === LOGIN === (usa usuario real del proyecto)
    cy.readFile('src/data/usuarios.json').then(({ 0: { email, password }
}) => {
      cy.visit('/login');
                  cy.get('.bg-blue-600').click()

      cy.get('[data-testid="input-email"]').type(email);        //
Login.jsx
      cy.get('[data-testid="input-password"]').type(password);  //
Login.jsx
      cy.get('[data-testid="btn-login"]').click();              //
Login.jsx
      cy.url().should('include', '/catalogo');                  //
redirige al catálogo

      // === STUB de pagos: 1° declined, 2° approved (usa hook de
Checkout.jsx) ===
      cy.window().then((win) => {
        win.__TEST_PAY__ = {
          charge: cy.stub()
```

```
            .onFirstCall().resolves({ status: 'declined', reason:
'card_declined' })
            .onSecondCall().resolves({ status: 'approved', id: 'pay_123'
}),
        };
      });

      // === CATÁLOGO: guardar nombre del primer ticket y agregarlo ===
      cy.contains('h1', 'Catálogo').should('be.visible');       //
Catalogo.jsx
      cy.get('[data-testid^="card-"]').first().within(() => {
        cy.get('h3').invoke('text').then((t) =>
cy.wrap(t.trim()).as('ticketName'));
        cy.get('button[data-testid^="btn-add-"]').click();      // solo
el botón
      });
      cy.get('[data-testid="toast-added"]').should('be.visible'); //
toast de agregado

      // === IR A CARRITO (link del menú/layout) y validar producto ===
      cy.get('a[href="/carrito"]').click();                     //
Carrito.jsx
      cy.get('[data-testid="carrito-page"]').should('be.visible');
      cy.get('@ticketName').then((name) => {
        cy.get('[data-testid="carrito-lista"]').should('contain', name);
      });

      // === IR A CHECKOUT (link "Ir a checkout") ===
      cy.contains('a', 'Ir a checkout').click();                //
Carrito.jsx
      cy.get('[data-testid="checkout-form"]').should('be.visible'); //
Checkout.jsx
      cy.get('@ticketName').then((name) => {
        cy.contains('aside', 'Resumen').should('contain', name); //
nombre en el resumen
      });

      // === CHECKOUT: completar datos ===
      cy.get('[data-testid="inp-nombre"]').type('Luciano QA');
      cy.get('[data-testid="inp-email"]').type('luciano.qa@example.com');
      cy.get('[data-testid="inp-dni"]').type('30123456');
    // NÚMERO (AHORA): válido por Luhn
    cy.get('[data-testid="inp-card"]').type('4242 4242 4242 4242')

    // asegurate que el botón esté habilitado
    //cy.get('[data-testid="btn-pay"]').should('not.be.disabled').click()
      cy.get('[data-testid="inp-exp"]').type('12/30');
      cy.get('[data-testid="inp-cvc"]').type('123');
```

```
      // PRIMER INTENTO → debe RECHAZAR y quedarse en checkout
      cy.get('[data-testid="btn-pay"]').click();
      cy.location('pathname').should('include', '/checkout');
      cy.get('[role="alert"]').should('be.visible').and('contain', 'Pago
rechazado'); // Checkout.jsx

      // REINTENTO → APRUEBA
      cy.get('[data-testid="inp-card"]').clear().type('4242 4242 4242
4242');
      cy.get('[data-testid="btn-pay"]').click();

      // === CONFIRMACIÓN: textos e ítems ===
      cy.location('pathname').should('include', '/confirmacion'); //
Confirmacion.jsx
      cy.get('[data-testid="confirmacion-page"]').should('be.visible');
      cy.contains('¡Compra confirmada!').should('be.visible');
      cy.contains('Tu orden fue registrada correctamente. Abajo están los
detalles.').should('be.visible');
      cy.get('@ticketName').then((name) => {
        cy.get('[data-testid="orden-items"]').should('contain', name);
      });
    });
  });
});
```

ANEXO V – Paquete de replicación técnica

1. Introducción

Con el propósito de garantizar la reproducibilidad de los resultados, la trazabilidad del proceso de validación y la transparencia metodológica del presente trabajo, se elaboró un paquete técnico que permite replicar la ejecución completa de la suite de pruebas automatizadas desarrollada.
El paquete de replicación contiene todos los elementos necesarios para instalar, configurar y ejecutar las pruebas end-to-end (E2E) en entornos locales y en integración continua (CI) mediante GitHub Actions.

Se diseñó siguiendo las buenas prácticas de ingeniería de software reproducible, asegurando que ningún archivo contenga secretos ni credenciales sensibles.

Archivo / Carpeta	Descripción	Ubicación sugerida
README-Replicacion.md	Guía paso a paso para instalar dependencias, configurar variables y ejecutar la suite de pruebas en modo local o CI.	Carpeta raíz del proyecto
cypress.env.example.json	Plantilla de variables de entorno con valores de ejemplo (sin datos sensibles).	Carpeta raíz del proyecto
.github/workflows/e2e-tests.yml	Archivo de configuración de **GitHub Actions** que define el pipeline de CI/CD: instalación, ejecución de pruebas, publicación de artefactos y registro en Cypress Cloud.	.github/workflows/

2. Características técnicas del entorno

- **Sistema operativo (runner):** Ubuntu 24.04.3 LTS
- **Versión del runner:** 2.328.0
- **Imagen:** ubuntu-24.04 (versión 20250831.1.0)
- **Node.js:** recomendado 20 LTS (mínimo 18.x)
- **Navegador:** Chrome / Electron (headless)
- **Herramienta de pruebas:** Cypress v15.0.0
- **Integración continua:** GitHub Actions (trigger en push y pull request)

- **Registro de ejecuciones:** Cypress Cloud (vía CYPRESS_RECORD_KEY configurado como secreto en el repositorio)

3. Procedimiento de ejecución

a. Clonar el repositorio:

```
git clone <url-del-repo>
cd <proyecto>
npm ci
npx cypress verify
```

b. Configurar las variables de entorno locales (opcional):

```
cp cypress.env.example.json cypress.env.json
```

c. Ejecutar pruebas:

- **Modo interactivo:** npx cypress open
- **Modo headless (CI):** npx cypress run

d. Visualizar resultados:

- Videos y capturas: cypress/videos/ y cypress/screenshots/
- Reportes HTML o XML (si están habilitados en cypress.config.js)
- Ejecuciones en cloud (si el secreto CYPRESS_RECORD_KEY está configurado).

4. Propósito del anexo

Este anexo asegura que los resultados obtenidos en las pruebas automatizadas sean verificables y replicables por terceros evaluadores o investigadores, manteniendo la coherencia entre los entornos local, cloud y CI.
Asimismo, refuerza el cumplimiento del principio de reproducibilidad científica dentro de la ingeniería de software aplicada a la calidad.